



北京大學

北京大学集成电路学院博士研究生选题报告

敏捷芯片设计与创新性编译技术
驱动的软硬件协同

姓名：肖有为

学号：2201111592

专业：集成电路科学与工程

导师：梁云

摘要

人工智能与大语言模型、数据密集型应用和工业级异构架构快速演进，使芯片设计生产力与软件栈适配同时成为系统瓶颈。领域定制硬件只有在软件语义能够被发现为可复用架构能力、这些能力能够被实现为质量可控的硬件、并能够通过编译接口进入可验证的软件生态时，才能从一次性加速单元转化为可持续演化的系统能力。本报告围绕“敏捷芯片设计与创新性编译技术驱动的软硬件协同”，研究敏捷芯片设计与创新性编译技术如何在同一软硬件协同体系中相互定义、相互约束和共同演化。

本报告聚焦三个核心问题。应用需求如何通过架构接口转化为可复用的指令、微架构和加速器能力，架构能力如何高质量落到可综合、可验证、性能与资源可控的硬件实现，编译接口如何在架构约束下组织程序语义、硬件能力和优化过程，并形成可验证、可复用、可供智能体可靠使用的编译协同机制。方法上，本研究以架构接口和编译接口为主线，结合等价图、反合一、等价饱和、多层硬件中间表示、调度建模和可执行判定基准，组织可分析、可实现、可检查的协同过程。

已有研究基础沿三个层次展开，分别是面向领域定制的架构接口与端到端协同、面向高质量硬件实现的前端抽象与综合优化，以及大语言模型与形式化技术驱动的软硬件协同方法。未来研究将进一步围绕验证约束下的架构、硬件与编译协同生成、智能体编译基础设施、面向工业级异构架构的算子优化与智能体化编译协同展开，目标是在跨硬件架构场景中把可复用硬件能力、高质量硬件实现和编译接口支撑的智能体能力提升组织为统一的软硬件协同方法体系。

关键词：敏捷芯片设计，软硬件协同，架构接口，编译接口，等价图，智能体编译基础设施

HARDWARE-SOFTWARE CO-DESIGN DRIVEN BY AGILE CHIP DESIGN AND INNOVATIVE COMPILATION TECHNIQUES

Youwei Xiao (Integrated Circuit Science and Engineering)

Supervised by Yun Liang

ABSTRACT

Rapid advances in AI and LLM workloads, data-intensive applications, and industrial heterogeneous architectures are making chip-design productivity and software-stack adaptation coupled system bottlenecks. Domain-specific hardware can become a sustainable capability only when software semantics can be discovered as reusable architectural capabilities, when those capabilities can be implemented as quality-controlled hardware, and when compiler interfaces can expose them to a verifiable software ecosystem. This proposal studies hardware-software co-design driven by agile chip design and innovative compilation techniques, emphasizing the coordinated relation through which agile hardware capability formation and compiler infrastructure define, constrain, and evolve with each other.

The proposal formulates three research questions: how application demands can be transformed through architecture interfaces into reusable instructions, microarchitectures, and accelerator capabilities; how architectural capabilities can be realized as synthesizable, verifiable, and performance- and resource-controlled hardware implementations; and how compiler interfaces, under architectural constraints, can organize program semantics, hardware capabilities, and optimization processes into verifiable and reusable co-design mechanisms that agents can use reliably. Methodologically, it connects architecture and compiler interfaces with e-graphs, anti-unification, equality saturation, multi-level hardware IRs, scheduling models, and executable oracles.

The existing work is organized into three layers: domain-specialized architecture interfaces and end-to-end co-design, front-end abstractions and synthesis optimization for high-quality hardware implementation, and hardware-software co-design methods that combine LLMs with formal techniques. Future work will integrate these foundations around validation-constrained architecture-hardware-compiler co-generation, agentic compiler infrastructure, and operator optimization with agentic compilation for industrial heterogeneous

ABSTRACT

architectures. The expected contribution is a coherent methodology and set of systems that turn reusable hardware capabilities, quality-controlled hardware implementation, and compiler-mediated agentic optimization into a unified cross-architecture co-design ecology.

KEY WORDS: agile chip design, hardware-software co-design, architecture interface, compiler interface, e-graph, agentic compiler infrastructure

目录

第一章 引言	1
1.1 领域应用演进与领域定制计算趋势	1
1.2 芯片设计生产力瓶颈与敏捷设计路径	2
1.3 跨硬件架构的软硬件协同生态与关键接口	3
1.4 面向协同接口的国内外研究现状	4
1.4.1 应用需求到架构能力：从接口开放到语义复用	5
1.4.2 架构能力到硬件实现：从高层抽象到质量约束	7
1.4.3 架构约束下的编译优化：从语义空间到目标事实	9
1.4.4 智能化方法：从代码生成到受约束协作	11
1.5 核心科学问题与研究挑战	13
1.6 已有研究基础与拟开展研究	14
第二章 面向领域定制的架构接口与端到端协同	17
2.1 引言	17
2.2 应用专用处理器与定制指令的端到端协同	17
2.2.1 统一定制指令接口与端到端工具链	18
2.2.2 面向数据密集指令的软硬件协同优化	21
2.3 可复用定制指令发现	23
2.3.1 研究背景与问题建模	23
2.3.2 结构化表示与等价图反合一	24
2.3.3 分阶段识别流程与可扩展反合一	25
2.3.4 向量化、硬件感知选择与真实系统评估	26
2.4 微架构感知的领域加速器定制	28
2.4.1 研究背景与整体框架	28
2.4.2 全应用表示、性能剖析与程序分析	29
2.4.3 加速器建模与数据访问接口选择	30
2.4.4 候选选择算法与加速器合并	31
2.4.5 实验评估与意义	33
2.5 本章小结	34

第三章 面向高质量硬件实现的前端抽象与综合优化	35
3.1 引言	35
3.2 周期确定型硬件前端	35
3.2.1 研究背景与设计目标	35
3.2.2 周期行为表达与控制子语言	36
3.2.3 周期分析与有限状态机综合	38
3.2.4 实验评估与意义	39
3.3 面向实现质量的前端抽象与综合机制	39
3.3.1 研究背景与设计目标	39
3.3.2 统一指令扩展接口与架构描述语言	40
3.3.3 微架构感知综合与调度	41
3.3.4 硬件实现与接口生成	42
3.3.5 实验评估与意义	43
3.4 变换、映射与调度联合优化	45
3.4.1 研究背景与设计目标	45
3.4.2 等价图设计空间与硬件映射表示	45
3.4.3 时序模型与调度式提取	46
3.4.4 高效求解与实验评估	47
3.4.5 本节小结：从前端表达到联合优化	50
3.5 本章小结	50
第四章 大语言模型与形式化技术驱动的软硬件协同方法	52
4.1 引言	52
4.2 等价图策略自动化	53
4.2.1 研究背景与设计目标	53
4.2.2 可检查的策略表示	53
4.2.3 智能体流程：离线策略合成与在线复用	54
4.2.4 证明驱动的经验记忆	55
4.2.5 可处理性引导与简化提示	56
4.2.6 实验评估与对软硬件协同的意义	57
4.3 本章小结	58
第五章 未来工作	59
5.1 未来工作总体目标	59
5.2 验证约束下的架构、硬件与编译协同生成	60

5.2.1 问题来源与研究问题	60
5.2.2 研究方法与系统设计	60
5.2.3 对全局故事的补全意义	61
5.3 智能体编译基础设施	61
5.3.1 问题来源与研究问题	61
5.3.2 研究方法与系统设计	61
5.3.3 对全局故事的补全意义	62
5.4 面向工业级异构架构的算子优化与智能体化编译协同	62
5.4.1 问题来源与研究问题	62
5.4.2 研究方法与系统设计	63
5.4.3 对全局故事的补全意义	64
5.5 本章小结	64
第六章 结论	65
参考文献	66

第一章 引言

1.1 领域应用演进与领域定制计算趋势

高性能计算系统正在同时承受应用演进和系统构建方式变化带来的压力。人工智能（AI）与大语言模型（LLM）、后量子密码、信号处理、点云处理、图形渲染等领域持续产生新的计算、访存和控制模式，芯片设计、编译适配、验证和软件生态构建成本也在不断上升。二者叠加后，算力瓶颈已不限于单个芯片的峰值性能，而表现为应用需求增长、能效约束、硬件迭代速度和软件生态生产力共同造成的系统性矛盾。

人工智能与大语言模型是当前最强的产业压力之一。以 TPU v4^[1] 为例，Google 将其定位为面向快速变化机器学习模型的领域专用架构，并在系统层面结合可重构互连、嵌入加速和云端部署环境；论文报告 TPU v4 相对 TPU v3 获得 2.1x 性能提升和 2.7x 每瓦性能提升，4096 芯片系统规模也使其成为面向大语言模型训练和推理的系统级基础设施。NVIDIA GB200 NVL72^[2] 将 72 个 Blackwell 图形处理器通过 NVLink/NVLink Switch 组织为机架级系统，Huawei CloudMatrix384^[3] 和 UB-Mesh^[4] 相关研究也把大语言模型服务扩展到跨芯片、跨节点互连和系统级调度问题。面向 3D-DRAM 的大语言模型加速器 ATLAS^[5] 与分布式 3D 堆叠人工智能加速器设计空间探索框架 DeepStack^[6] 进一步表明，架构研究已经必须同时建模存储事务、互连、并行策略、调度和编程原语。这样的工业系统说明，领域定制计算的价值来自架构、编译、互连、系统软件和部署环境的共同设计，而不仅是某个算术单元的局部增强。

在人工智能与大语言模型之外，后量子密码、信号处理、点云处理和图形渲染等场景进一步说明领域定制具有跨应用的长期需求。不同领域的热点结构、访存粒度、控制模式和数据复用方式各不相同。后量子密码强调数论变换、多项式运算和位级/字级操作，点云处理和图形渲染强调不规则数据布局与几何结构，信号处理强调流式计算、向量化和数据通路复用。开放 RISC-V^[7]、应用专用指令集处理器工具链^{[8][9]} 和定制指令/加速器候选发现方法^{[10][11][12]} 为领域定制提供了硬件入口，但真正的挑战在于如何把不同应用中的可复用模式识别出来，并通过架构接口、硬件实现和编译工具链稳定接入系统。

因此，领域定制计算呈现出两个转变。硬件能力从通用核外的单点加速器，转向可通过架构接口表达、可由编译器发现和使用、可在不同平台迁移的系统能力；优化对象从单个内核或单个模块，转向应用语义、存储层次、指令/微架构机制、编译映射和系统

执行环境共同约束的联合空间。这个趋势为“敏捷芯片设计”提出了更高要求。硬件创新不仅要能快速生成，还要能快速验证、映射、复用和迭代。

与此同时，大语言模型与智能体方法正在成为另一条技术革新线索。大语言模型在本选题中具有两重角色。一方面，它作为工作负载推动异构算力系统和领域专用架构发展；另一方面，大语言模型与智能体方法作为工程自动化方式，正在改变软件开发、编译优化和系统构建流程。OpenAI 对 Codex 的系统说明显示，软件工程智能体已经从代码补全扩展到独立沙箱、代码修改、命令执行、测试和日志记录组织^[13]。这一趋势尚不足以证明芯片设计可以完全自动化，但它提示复杂工程流程正在从人手工编辑产物，转向人、模型、工具和记录共同协作的形态。对于芯片设计与编译优化，这种变化只有在后续章节所讨论的架构接口、编译基础设施和验证约束体系支撑下才可能可靠落地。

1.2 芯片设计生产力瓶颈与敏捷设计路径

领域定制计算带来性能和能效机会，也放大了芯片设计生产力瓶颈。传统芯片设计流程高度依赖人工经验。架构设计者需要决定哪些应用模式值得硬化，硬件工程师需要实现和验证寄存器传输级设计，编译器工程师需要提供指令选择、数据布局、调度和系统软件接入路径，系统工程师还要保证这些能力能够在真实软件栈中被稳定调用。随着应用变化加快，硬件设计若仍以长期手工迭代为主，就很难支撑快速演进的领域需求。

本报告所说的“敏捷芯片设计”，强调快速形成、验证、映射和迭代领域硬件能力。它从应用中发现值得硬化或优化的模式，通过架构接口把这些模式表达为可复用硬件能力，再通过硬件前端、综合和验证流程生成质量可控的实现，并通过编译和系统软件机制使软件能够自动发现、调用、评估和改进这些能力。只有这些环节同时成立，领域定制硬件才不会停留在一次性实验模块，而能进入可复用的系统生产力链条。

现有技术路线各有价值，也各有局限。应用专用指令集处理器和定制指令扩展允许设计者在保持处理器可编程性的同时加入领域能力，RISC-V 生态进一步降低了指令扩展和处理器实验门槛；但不同扩展接口、片上系统框架和编译路径之间仍存在割裂，导致同一硬件能力难以跨平台复用^{[7] [14] [15] [16]}。现场可编程门阵列和可重构加速器提供了比专用集成电路更快的验证与部署路径，但底层寄存器传输级开发成本高，而高层综合虽提高抽象层次，却常以编译指示和工具启发式间接控制微架构，生成结果的可预测性和可检查性仍然不足^{[17] [18] [19]}。硬件中间表示、MLIR^[20]、等价图等编译基础设施为统一表示和跨层优化提供了基础，但如果缺少稳定的语义契约、测试记录和基准过程，优化结果仍难以跨任务、跨平台和跨工具链迁移^[21]。

人工智能辅助设计和智能体方法为生产力问题提供了新的可能。AlphaEvolve^[22] 将大语言模型组织为进化式代码智能体，通过直接修改代码并接受评价器结果来迭代改

进算法,在 Google 的计算基础设施优化和算法发现任务中展示了“模型、工具和评价器”组合的潜力。NVIDIA AVO^[23]进一步把类似思想推进到性能敏感的内核与编译优化。它用自主代码智能体替代固定变异、交叉和手工启发式,在 Blackwell B200 图形处理器的注意力内核优化中利用谱系、领域知识和执行结果生成、修复、批判和验证实现修改。这些工作说明,智能体方法的关键不仅是模型能生成代码,更是模型能在可执行评价、领域知识和检查约束中持续改进产物。

然而,芯片设计比一般软件工程和单一内核优化更强调语义正确性、时序约束、接口协议、可综合性和长期维护。直接让模型生成寄存器传输级设计、编译遍或调度策略,容易把设计意图、目标约束和验证结果混在自然语言或临时代码中,难以检查和复用。因此,敏捷芯片设计中的智能体方法需要依托架构接口、硬件中间表示、编译基础设施和验证判定基准,并被放在可检查、可回放的系统中,成为受约束的协同机制。

1.3 跨硬件架构的软硬件协同生态与关键接口

由上述趋势可见,本报告讨论的软硬件协同指向跨硬件架构的协同生态。应用需求通过架构接口转化为硬件能力,软件语义通过编译接口转化为可映射、可优化、可验证的执行形式,硬件实现通过综合和验证流程形成可部署产物,性能剖析、执行轨迹和失败案例再反过来约束下一轮编译与架构设计。这个生态的关键不在单次生成一个模块或单次优化一个程序,而在于能否形成可复用、可检查、可验证、可持续演化的系统能力。

在这个生态中,架构接口首先定义“硬件能够提供什么能力”。它不仅包括指令集架构或定制指令扩展,也包括微架构机制、存储层次、互连、加速器执行模型和软件可见资源抽象。一个领域能力若不能通过架构接口稳定表达,就难以被编译器发现和利用;一个架构接口若只描述指令格式而不描述访存路径、执行约束和系统边界,也很难支撑复杂数据密集型应用。RoCC^[14]、CV-X-IF^[15]、SCAIE-V^[16]、TIGRA^[24]和 Longnail^[9]等研究从不同侧面说明,领域定制处理器需要统一扩展接口、硬件合成机制和编译支持;而面向大语言模型的 TPU v4^[1]、GB200 NVL72^[2]、CloudMatrix384^[3]、ATLAS^[5]和 DeepStack^[6]等系统进一步说明,复杂架构能力还需要把访存、互连、通信、并行策略和服务系统事实纳入协同设计。

编译接口则定义“软件如何发现、使用、优化和验证这些能力”。它覆盖中间表示、逐层降低、模式/重写、调度、代码生成、系统软件接口和性能剖析等层次。编译器在这里承担连接应用语义、架构能力、硬件实现和系统执行环境的中间层角色。LLVM^[25]、MLIR^[20]、Diospyros^[26]、ISARIA^[27]、SEER^[28]和 Lakeroad^[29]等工作共同说明,编译接口不仅决定“能否生成代码”,还决定“哪些硬件能力能被系统性发现、验证和复用”。

随着智能体方法进入工程自动化，编译基础设施还需要承担新的智能体协作媒介角色。目标事实需要进入显式中间表示与语义契约，受约束变换需要生成可执行后端产物，语义检查、测试、基准测试和执行轨迹需要形成可追踪记录。这个判断可以推广到本报告的核心主张。智能体方法只有通过编译器原生对象、编译过程边界、目标契约、语义/可执行检查和基准测试/执行轨迹记录进入流程，才可能可靠服务软硬件协同。否则，模型生成的硬件或编译改动即使局部可运行，也难以说明为什么正确、为什么适配目标硬件、如何跨任务复用、如何在后续系统演化中继续发挥作用。

因此，跨硬件架构软硬件协同的关键在于通过架构接口定义可复用硬件能力，通过编译接口连接软件语义、硬件实现与系统执行环境，并在形式化语义、执行记录和自动化工具链支撑下形成可持续演化的系统能力。后续章节围绕这一判断展开。第二章讨论应用需求如何经由架构接口转化为可复用定制能力，第三章讨论这些能力如何落到高质量硬件实现，第四章讨论大语言模型与形式化技术如何进入可验证优化流程，第五章提出面向未来的系统化研究路线。

图示占位：跨硬件架构软硬件协同生态。正式版本将绘制应用需求、架构接口、编译接口、硬件实现、综合/验证、系统执行环境和验证约束体系之间的关系。

图 1.1 跨硬件架构软硬件协同生态示意图（待绘制）

1.4 面向协同接口的国内外研究现状

围绕敏捷芯片设计与创新性编译技术的软硬件协同，国内外研究在协同链条的关键接口上形成了丰富积累。第一类研究关注应用需求如何进入架构层，开放指令集架构、应用专用指令集处理器、定制指令、协处理器接口和加速器候选选择共同回答“什么硬件能力值得被形成并长期保留”。第二类研究关注架构能力如何落到硬件实现，寄存器传输级设计、嵌入式硬件语言、高层综合、领域硬件语言、硬件中间表示和物理感知综合共同回答“这些能力如何变成可综合、可验证、质量可控的电路”。第三类研究关注编译优化如何受目标架构约束，LLVM、MLIR、TVM、Triton、等价饱和、硬件映射、工业级人工智能编译框架和大规模异构系统共同回答“软件语义如何在目标事实中被发现、映射和优化”。第四类研究关注智能体化方法如何进入严肃工程流程，硬件生成大语言模

型、软件工程智能体、算法搜索智能体和形式化编译策略共同回答“模型如何在工具、检查和结构化对象约束下工作”。这四类接口分别对应从应用到架构、从架构到硬件、从软件到目标架构、从人机协作到智能体能力的必要转化环节。四类研究合在一起说明，现有基础已经相当充分，但也暴露出共同缺口。单点工具可以局部提高生产力，单层优化可以局部提高性能，博士阶段需要进一步研究这些局部能力如何被组织成跨硬件架构、跨编译层次、跨设计阶段的可复用协同机制。

1.4.1 应用需求到架构能力：从接口开放到语义复用

应用需求到架构能力的研究首先来自一个基本矛盾。通用处理器提供可编程性和生态稳定性，但领域应用中的计算热点、访存模式和控制结构常常需要专门硬件才能获得足够性能和能效；专门硬件一旦脱离可编程接口，又容易变成难以复用、难以调用、难以维护的一次性模块。因此，这一方向需要从能否加速某个热点进一步推进到更根本的问题，即什么样的应用模式值得成为架构能力，这个能力通过什么接口暴露给软件，编译器如何识别和调用它，实现代价是否允许它被长期保留。相关研究大致形成三条主线，分别是提供硬件入口的开放处理器和扩展接口、提供处理器生成与编译支持的应用专用指令集处理器和架构描述工具链，以及提供应用到硬件能力筛选机制的候选识别与加速器选择方法。

开放指令集架构尤其是 RISC-V^[7] 降低了架构实验和指令扩展门槛，使研究者可以在真实处理器、片上系统框架和编译工具链中探索定制能力。Rocket Chip^[14] 和 Chipyard^[30] 代表了以生成式片上系统框架支撑处理器、互连、缓存和加速器组合的路径；RoCC^[14] 为 Rocket 系列处理器提供较灵活的协处理器连接接口，适合把较复杂或较长延迟的逻辑作为解耦模块接入；PicoRV32 的 PCPI^[31] 则提供轻量级协处理器接口，体现资源受限处理器中的定制入口；NICE^[32]、CV-X-IF^[15]、TIGRA^[24] 和 SCAIE-V^[16] 分别从不同处理器耦合、流水线交互、可移植扩展和接口生成角度处理定制指令接入问题。这些接口共同说明，架构开放只是协同统一的前提。同样是定制指令或协处理器，可能涉及不同的寄存器读写时机、访存权限、异常处理、流水线阻塞、握手协议和编译器可见性。开放指令集架构解决了“可以扩展”的前提，但没有自动解决“如何跨处理器、跨工具链、跨应用稳定复用”的问题。

应用专用指令集处理器设计长期试图把处理器描述、硬件生成和编译工具生成纳入同一流程。TIE^[8] 将定制指令扩展作为架构描述语言的一部分，强调指令语义、资源和编译支持之间的联系；Tensilica^[33]、Synopsys ASIP Designer^[34] 和 Cudasip Studio^[35] 等工业系统则把处理器建模、指令集扩展、模拟器、调试器和编译工具生成组织为商业化流程；Longnail^[9] 将高层综合用于生成可移植 RISC-V 定制指令扩展，试图降低从算

法描述到处理器扩展的实现成本；ASSIST^[36]面向 RISC-V 生成高质量应用专用指令集处理器，体现了开放指令集架构背景下快速处理器定制的需求。这条主线的价值在于把“指令扩展”从孤立硬件模块推进到处理器工具链问题，但局限也很明显。工业工具通常闭源且绑定特定知识产权核，研究工具往往只覆盖若干耦合方式或有限硬件行为，编译支持常以模式匹配和后端改动为主，难以系统表达复杂访存、状态化行为、多周期执行和目标处理器微架构属性。

候选识别研究则直接回答“什么值得定制”。早期工作多从基本块或数据流图中枚举可硬化子图。Clark 等人的自动定制指令生成^[37]^[10]、Pozzi 等人的精确与近似算法^[38]关注凸子图、输入输出端口约束、面积代价和加速收益，在细粒度定制指令选择中奠定了经典问题形式。Aditya 等人面向 VLIW/EPIC 的自动架构综合^[39]体现了把程序并行性转化为处理器结构的早期努力。随后，研究范围从单个指令扩展扩展到能效核心和粗粒度加速区域。Conservation Cores^[40]和 GreenDroid^[41]关注成熟代码片段的硬化和能效收益，QSCORES^[42]试图对规模更大的程序区域进行专门化，FINDER^[43]面向内联加速器发现，NOVIA^[11]强调从多程序、多区域中发现非常规内联加速器。AccelSeeker^[12]将 LLVM 中间表示分析用于从未改写软件中选择加速候选，Trireme^[44]进一步探索多层次并行性和层次化硬件加速候选。这些方法显示，候选选择已经从局部数据流子图走向程序区域、函数、循环和多应用复用，但它们通常仍依赖语法相似性、剖析热点或抽象性能模型，难以同时处理语义等价、跨程序复用、硬件实现质量和编译调用稳定性。

面向现代应用的架构能力还受到访存、互连和系统规模的强烈约束。Google TPU v4^[1]将矩阵乘单元、片上存储、嵌入加速、可重构光互连和云端系统组织为面向机器学习整体架构；ATLAS^[5]面向 3D-DRAM 大语言模型加速器提出全栈性能评估基础设施，将系统架构抽象和编程原语统一到可复用建模中；DeepStack^[6]面向分布式 3D 堆叠人工智能加速器，把存储事务、存储体约束、缓冲限制、热功耗建模、并行策略和执行调度纳入早期设计空间探索；NVIDIA GB200 NVL72^[2]通过 NVLink 和机架级系统把图形处理器、中央处理器、交换芯片、内存容量和服务部署组织成大语言模型基础设施；Huawei CloudMatrix384^[3]和 UB-Mesh^[4]则从大规模神经网络处理器互连、专家并行、令牌分发、集合通信和数据中心网络局部性说明，面向大语言模型的架构能力已经延伸到跨芯片、跨节点和服务系统边界。这些工业和系统研究使“架构能力”的含义发生变化。它已经涵盖访存通路、互连拓扑、数据布局、通信原语、执行系统接口和编译可见目标事实的组合，而不限于某条定制指令或某个加速核。

由此可见，应用到架构的研究已经提供了开放接口、应用专用指令集处理器工具链和候选选择三类基础，但三者之间仍存在断裂。开放接口给出接入点，却不保证语义复用；应用专用指令集处理器工具链能生成处理器，却不一定覆盖开放、跨平台、可检查

的协同；候选选择能发现热点或区域，却常常无法证明它们在跨程序、跨负载、跨实现中的长期价值。面向软硬件协同，真正需要的是把应用语义、复用范围、耦合方式、访存事实、面积性能代价和编译器可见性放在同一个架构接口问题中处理，使硬件能力成为可由软件生态稳定调用和持续演化的系统能力。

1.4.2 架构能力到硬件实现：从高层抽象到质量约束

架构接口回答“硬件能力应当是什么”，但它并不保证该能力能够以高质量硬件形式实现。硬件实现还受到周期行为、控制结构、资源共享、流水线、时序收敛、接口协议、物理布局和验证成本的共同约束。敏捷芯片设计的困难正是在这里显现。完全使用寄存器传输级设计可以控制细节，但迭代成本高；完全依赖高层综合可以提高抽象层次，却常常失去对微架构结果的直接控制；使用领域专用语言时，效率取决于应用是否落在语言假设内；构建硬件中间表示时，又必须在通用性、可分析性和后端可落地之间取得平衡。因此，本类研究可以按表达方式和质量约束的关系分为寄存器传输级/嵌入式硬件语言、高层综合、领域硬件语言、硬件中间表示与物理感知综合五个层次。

寄存器传输级设计和嵌入式硬件语言提供最直接的结构表达能力。Chisel^[45]通过Scala嵌入式语言支持参数化、生成式硬件和模块组合，Bluespec SystemVerilog^[46]通过规则和类型系统提高并发硬件描述的抽象层次，FIRRTL^[47]则把Chisel生成的硬件引入中间表示，使优化、降低和后端生成可以被编译器化组织。这一路径的优势是保留硬件结构和周期控制，适合复杂微架构和可重用硬件生成器；局限是跨周期行为、握手交错和控制合法性仍需要设计者以硬件思维手工维护。嵌入式硬件描述语言改善了“写硬件”的生产力，但并没有完全解决“以可分析方式表达设计意图和周期约束”的问题。

高层综合通过从C/C++、OpenCL或类似高级描述生成硬件，试图让软件工程师在更高层次描述算法。Vitis HLS^[17]、LegUp^[48]和Dynamatic^[49]代表了静态调度、资源共享、动态调度和控制/数据流硬件生成的不同路径；AutoDSE^[18]使用程序变换和设计空间探索改进高层综合结果，ScaleHLS^[19]则把高层综合优化纳入MLIR体系，借助多层中间表示表达循环、数据流和硬件优化。高层综合的核心贡献是显著降低硬件生成门槛，并能快速探索不同编译指示、流水线和并行化配置；但其核心限制也同样突出。高级软件语义和最终硬件结构之间存在较长的工具链距离，设计者往往只能通过指令、编译指示、调度报告和经验反复试探结果。对于需要确定周期行为、精细资源控制或复杂接口协议的设计，高层综合的抽象收益可能被不可预测的后端决策抵消。

领域硬件语言试图用更受限但更可分析的语言表达特定硬件结构。Dahlia^[50]通过时间敏感仿射类型约束存储访问和时序行为，强调可预测加速器设计；Spatial^[51]面向应用加速器表达并行、流水、存储层次和空间映射；Aetherling^[52]通过类型引导调度描

述流式加速器；HeteroCL^[53] 面向软件定义可重构计算，把算法表达和调度、数据类型、硬件后端选择分离；Allo^[54] 将计算、存储、通信和数据类型定制封装为可组合源语，面向层次化加速器设计保留程序结构；StreamTensor^[55] 则用迭代张量类型表达流式布局，使张量切分、计算内核融合、缓冲分配和资源分配能够共同服务于数据流加速器。这类语言的价值在于用领域结构换取更强的静态分析和更清晰的实现意图，尤其适合流式计算、张量计算和规则数据通路；但其适用性也受领域假设约束。一旦设计涉及通用控制流、状态化行为、多种耦合接口或非规则访存，领域语言往往需要借助更底层的硬件描述或后端工具特性。

硬件中间表示和硬件编译基础设施试图把不同前端和综合路径共享的硬件事实显式化。Calyx^[56] 将控制和数据通路分离，使加速器生成器可以共享优化与后端；Filament^[57] 通过时序类型表达模块接口的时序约束；CIRCT^[58] 试图在 MLIR 生态中组织硬件方言、寄存器传输级降低、SystemVerilog 生成和硬件编译流程；MLIR^[20] 本身则为多层抽象、逐层降低和领域方言提供了通用基础。与单一高层综合或单一硬件描述语言相比，硬件中间表示的意义在于为不同层次保留可检查的事实，包括哪些值在哪些周期可用，哪些组件共享资源，哪些控制路径互斥，哪些操作可以调度到同一硬件原语，哪些接口需要满足握手或时序约束。

在现场可编程门阵列和大规模加速器设计中，实现质量还强烈依赖物理布局、互连和时序收敛。AutoBridge^[59] 将粗粒度布局和流水化耦合起来，说明高层综合设计若忽略物理距离，很难在大型现场可编程门阵列上获得高频率；TAPA^[60] 通过任务并行数据流编程和共同优化支持现代现场可编程门阵列上更可扩展的高层综合设计；RapidStream^[61] 和 RapidStream IR^[62] 把并行物理实现、中间表示基础设施和高层物理综合结合起来；FADO^[63] 进一步强调多芯粒现场可编程门阵列上的布局感知编译指示优化。它们共同说明，硬件质量从任务划分、数据流边界、缓冲、流水、资源映射和物理约束开始就参与设计决策，不能被视为前端功能正确后的后端细节。若前端抽象无法承载这些事实，敏捷设计很容易变成“快速生成低质量硬件”。

因此，架构能力到硬件实现的国内外研究已经形成清晰谱系。寄存器传输级/嵌入式硬件描述语言保留结构控制，高层综合提高算法到硬件的生产力，领域语言用受限表达换取可分析性，硬件中间表示为跨前端和跨后端提供公共事实，物理感知综合把布局和时序纳入高层优化。这些基础共同导向一个结论。高层抽象本身并不保证高质量硬件，真正关键的是周期、控制、资源、耦合、访存、调度、映射和物理约束能否在足够早的层次被显式表达、共同优化并接受检查。只有这样，架构层形成的能力才可能稳定转化为可综合、可验证、性能与资源可控的实现。

1.4.3 架构约束下的编译优化：从语义空间到目标事实

编译优化在软硬件协同中承担接口角色。架构能力如果不能被编译器发现、匹配、调用、验证和优化，就很难进入真实软件生态；硬件实现如果不能把延迟、吞吐、访存、资源和系统边界转化为编译可见事实，也很难被软件稳定使用。因此，架构约束下的编译优化需要同时处理程序语义、控制流、数据流、张量表达和等价变换空间，也需要处理目标硬件的指令、源语、存储层次、互连、内核实现、服务系统和性能代价。相关研究可以沿着编译基础设施、自动向量化与模式匹配、等价饱和、硬件映射和工业级异构编译框架展开。

LLVM^[25]代表了可重用编译基础设施的经典路径，它把多语言前端、优化编译遍、中间表示和多目标后端组织起来，使程序分析和变换可以跨工具复用。MLIR^[20]进一步把这一思想推广到多层中间表示和方言体系，使张量、循环、线性代数、硬件、加速器和目标后端可以在不同抽象层保留各自语义；Polygeist^[64]将C/C++提升到多面体友好的MLIR表示，说明前端提升本身就是保留优化机会的重要步骤。TVM^[65]和Triton^[66]则体现了人工智能编译中的另一种接口化趋势。前者把深度学习算子、调度、自动搜索和多后端代码生成组织为端到端优化编译器，后者用适合块级张量计算的中间语言和编译器让程序员表达接近硬件执行模型的分块、程序实例和内存访问。TileLang^[67]进一步把分块级数据流和线程绑定、布局、张量化、流水等调度空间分离，说明人工智能计算内核编程正在形成更贴近硬件的中间抽象；ACT^[68]则从张量加速器指令集架构描述自动生成编译后端，直接回应新型加速器迭代快而后端构建成本高的问题。这些系统共同说明，现代编译器的价值不仅在于优化已有代码，也在于提供一种能让应用语义、目标约束和优化搜索共同存在的工程基底。

自动向量化和模式匹配研究为“软件如何使用硬件能力”提供了更早的基础。Superword-level parallelism^[69]从标量代码中发现可打包为单指令多数据执行的独立操作，Nuzman等人的交错数据自动向量化^[70]处理更复杂的数据布局，Diospyros^[26]使用等价饱和为数字信号处理自动向量化提供更丰富的重写空间，ISARIA^[27]则通过规则合成生成面向可定制数字信号处理器的向量化编译器。这条线索表明，编译器要使用硬件能力，需要超越语法层面的指令选择，识别程序中的语义模式、数据布局、位宽约束、向量形态和目标源语之间的对应关系。当硬件能力来自定制指令或领域加速器时，这种对应关系更加困难，因为硬件可能没有成熟指令集架构那样稳定的模式库和成本模型。

等价图和等价饱和为语义空间中的搜索提供了重要技术基础。Tate等人的等价饱和^[71]将程序变换从“依次应用并丢弃旧版本”改为“在等价图中累积所有等价版本，再由代价模型选择”，egg^[21]通过高效重建和可扩展接口推动等价饱和成为实用基础设施。Ruler的重写规则推断^[72]、Babble的基于等价图和反合一的抽象学习^[73]、ENumO

[74]、Equality Saturation Theory Exploration à la Carte [75]、Datalog 与等价饱和结合 [76] 等工作，分别从规则生成、抽象归纳、理论组织和关系式表达角度扩展了等价饱和的能力。近期工作还开始把等价饱和推入编译器原生基础设施。eqsat 方言 [77] 将等价图状态表示为 MLIR 代码，使构造性编译遍可以在保持等价图状态的同时继续工作；DialEgg [78] 连接 Egglog 和 MLIR，提供方言无关的表示转换；Mind the Abstraction Gap [79] 直接面向真实机器学习编译器中局部图重写和后续布局、并行、调度、内存管理之间的抽象落差。这些研究说明，编译优化可以把等价关系、重写规则、搜索策略、目标事实和提取目标组织成更显式的对象，而不必完全依赖人工手写的一串编译遍。

硬件相关优化把等价饱和进一步推向目标事实约束。SEER [28] 使用等价图进行高层综合层面的循环和数据通路超优化，E-Syn [80] 将技术感知代价函数纳入逻辑综合中的等价图重写，E-morphic [81] 处理结构探索中的可扩展等价饱和，EqMap [82] 面向现场可编程门阵列查找表映射使用等价关系组织映射空间，Lakeroad [29] 面向复杂硬件原语映射，Churchroad [83] 和 Smoothe [84] 则分别从扩展映射和可微提取角度缓解搜索和提取瓶颈。TensorRight [85] 从另一侧提醒，张量图重写若要进入真实编译器，就需要能证明任意阶数和尺寸张量上的语义保持；Trinity [86] 则把等价饱和推进到分块级张量程序优化，在代数等价、内存输入输出和计算组织三个维度中联合搜索。这些研究说明，等价空间只有和目标硬件事实、语义验证和代价模型结合才具有工程价值。同样的语义表达，面对数字信号处理器、查找表、块随机访问存储器、流水线深度、可用频率和物理布局，最优实现可能完全不同。编译接口因此必须能够表达“哪些变换保持语义”“哪些映射目标可用”“哪些代价和合法性来自硬件”。

工业级人工智能编译框架进一步把目标事实扩展到系统边界。PyTorch torch.compile 和 TorchDynamo [87] 通过 Python 帧求值捕获动态图程序，并在图中断、重新编译、后端选择和即时回退之间管理动态语言与静态编译的边界；TensorRT-LLM [88] 将大语言模型推理中的模型构建、引擎生成、计算内核融合、量化、KV 缓存、分页注意力、连续批处理、低延迟服务和 C++ 执行系统组织在同一优化流程中。PagedAttention [89] 以分块 KV 缓存管理说明服务系统中的内存碎片和共享问题会直接限制批处理规模，FlashInfer [90] 则通过可定制注意力模板、KV 缓存格式和负载均衡调度展示计算内核、缓存布局和服务框架之间的耦合。Triton-distributed [91] 将通信源语引入 Triton 编译模型以表达计算、访存和通信重叠，MSCCL++ [92] 和 NCCL EP [93] 进一步说明人工智能推理和混合专家模型的专家并行需要可移植但保留性能的通信抽象。这里的编译优化已经超出单个中间表示编译遍的问题，需要同时考虑张量图、计算内核选择、内存布局、缓存管理、批处理策略、服务延迟、通信源语和目标硬件能力。ATLAS [5] 和 DeepStack [6] 从评估与设计空间探索角度补充了同一判断。面向大语言模型的架构和编

程原语必须和存储层次、互连、并行策略、调度和执行系统一起建模。NVIDIA GB200 NVL72^[2]、CloudMatrix384^[3] 和 UB-Mesh^[4] 所呈现的多芯片、多互连和大规模服务系统进一步强化了这一点。编译优化面对的目标事实已经覆盖算子内部语义、通信拓扑、数据移动、专家并行、集合通信和资源分配等多个层面。

因此，架构约束下的编译优化研究已经从传统编译遍优化扩展为跨层接口问题。LLVM、MLIR、TVM 和 Triton 说明编译基础设施需要保留并分层组织语义；自动向量化、模式匹配和源语选择说明软件必须能发现和调用硬件能力；等价饱和说明语义空间可以被紧凑表示和系统搜索；硬件映射研究说明目标事实必须进入搜索与提取；PyTorch 和 TensorRT-LLM 等工业框架说明编译优化还要面对动态程序、服务系统和异构硬件边界。这一综述直接支撑后续科学问题。软硬件协同中的编译接口必须同时组织程序语义、硬件能力、目标事实和优化策略，否则架构创新和硬件实现都难以进入可持续的软件生态。

1.4.4 智能化方法：从代码生成到受约束协作

智能化方法正在成为工程自动化的重要趋势。在芯片设计和编译优化中，它需要超越“让大语言模型写寄存器传输级设计”或“让大语言模型写优化代码”的初级形态。硬件和编译任务具有更强的正确性约束。寄存器传输级设计要满足语法、仿真、协议、可综合、时序和面积功耗要求；编译优化要保持程序语义，适配目标架构，并在可重复的基准测试中体现性能收益。因此，本类研究需要关注模型能力、工具调用、结构化对象、检查机制和经验积累如何共同形成智能体能力。相关工作可以分为硬件生成大语言模型、工程智能体与算法搜索智能体、以及面向编译优化的形式化智能体方法三类。

硬件生成大语言模型研究已经从简单的自然语言到 Verilog 生成，逐步走向数据、评测、修复和工具结果共同约束的流程。DAVE^[94] 早期探索从英文描述生成 Verilog，Chip-Chat^[95] 讨论对话式硬件设计的挑战与机会，ChipGPT^[96] 评估自然语言硬件设计能力。VerilogEval^[97] 和 RTLLM^[98] 建立了面向 Verilog/寄存器传输级生成的评测基准，VeriGen^[99]、RTLCoder^[100] 和 BetterV^[101] 则分别从开源模型、代码生成能力和受控生成角度提高寄存器传输级输出质量。RTLFixer^[102] 和 AutoChip^[103] 引入编译错误、仿真结果或多轮修复机制，说明硬件生成必须经由工具检查和迭代改正。ChipNeMo^[104] 和 Data is all you need^[105] 进一步强调芯片设计领域数据、自动化数据增强和领域适配对模型能力的重要性。这条谱系表明，大语言模型硬件生成的关键在于让模型在领域数据、设计任务、编译器、仿真器和评测基准约束下逐步提高可靠性，避免停留在一次性生成看似合理代码的层面。

软件工程智能体和算法搜索智能体提供了更一般的工程形态。SWE-agent^[106]将语言模型放入面向软件工程的计算机接口中,使模型能够阅读仓库、编辑文件、运行命令并根据测试结果修复问题;OpenAI Codex^[13]和 Anthropic Claude Code^[107]等系统进一步把模型置于沙箱、命令执行、代码审查、测试和人工监督构成的流程中。AlphaEvolve^[22]用大语言模型驱动算法发现和基础设施优化,依靠评价器对候选代码进行选择 and 迭代;NVIDIA AVO^[23]将智能体变异算子用于 Blackwell B200 图形处理器注意力内核优化,通过生成、修复、批判、验证、谱系和领域知识组织自主搜索。围绕图形处理器计算内核的后续研究进一步细化了这种趋势。K-Search^[108]将高层算法规划与低层程序实例化分离,用共演化世界模型引导复杂计算内核搜索;ARGUS^[109]用数据流不变量为智能体提供密集、结构化评价信号;面向计算内核优化智能体的领域专用语言与硬件上限引导^[110]强调抽象层次和硬件上限判断会直接影响搜索效率;SOL-ExecBench^[111]则以面向 NVIDIA Blackwell 图形处理器的 235 个真实计算内核优化问题和硬件上限评分重新定义智能体优化的评测目标。它们共同说明,智能体方法的有效性来自模型和工具之间的结构化协作。模型提出修改,工具给出可执行结果,评价过程选择方向,经验记录影响后续搜索。芯片设计与编译优化可以借鉴这种形态,但必须把“工具”和“评价”替换为更严格的硬件/编译对象、语义检查、综合约束和可执行判定基准。

形式化和编译基础设施为智能体化提供了更适合软硬件协同的行动边界。等价饱和和基础设施已经将重写规则、等价类、提取目标和代价模型显式化^{[71][21]};规则推断^[72]、抽象学习^[73]、规则合成^[27]和 ASPEN^[112]等工作进一步说明,模型或自动化系统可以围绕规则、策略表示和搜索空间工作,避免直接输出不可检查的最终代码。对编译优化而言,智能体方法更合理的介入方式是围绕中间表示、重写规则、等价饱和阶段、提取目标、硬件代价、基准测试和可执行判定基准提出受约束行动。这样,模型生成的是可以被编译器执行、被形式化语义限制、被测试和基准检查的优化过程,避免停留在难以验证的自然语言建议。

这一趋势对软硬件协同有两层意义。智能体方法可以提高复杂工程流程中的探索能力,在候选架构、硬件实现、编译策略、测试失败和性能异常之间进行长程搜索,弥补人工难以覆盖的大空间;同时,它也放大了对结构化基础设施的需求。如果没有清晰的架构接口、硬件中间表示、编译中间表示、目标契约、测试体系和可执行判定基准,模型的行动就难以被限定、复现和复用。智能体能力必须通过编译器原生对象、硬件实现对象和工程检查对象进入协同体系,才能服务于严肃的软硬件协同研究。

综上,国内外研究在四个层次上已经给出丰富基础。应用到架构研究提供了开放处理器接口、应用专用指令集处理器工具链和候选选择方法;架构到硬件研究提供了寄存器传输级/嵌入式硬件描述语言、高层综合、硬件领域语言、硬件中间表示和物理感知综

合路径；架构约束下的编译优化研究提供了多层中间表示、自动向量化、等价饱和、硬件映射和工业级异构编译框架；智能体化方法研究则提供了模型、工具、检查和结构化行动对象共同工作的工程范式。它们共同支持一个判断。敏捷芯片设计与创新性编译技术的关键在于建立协同机制，使应用语义形成架构能力，使架构能力形成高质量硬件，使硬件能力进入编译和软件生态，并使智能体在受约束对象上持续工作。下一节将在这一综述基础上进一步收束核心科学问题。

1.5 核心科学问题与研究挑战

从上述背景与研究现状可以看到，敏捷芯片设计与创新性编译技术驱动的软硬件协同，需要把硬件生成、编译优化和自动化代码生成纳入同一问题框架。应用与异构架构快速演进造成了设计生产力和软件栈适配的双重压力，硬件能力必须通过可编程、可综合、可验证、可由软件稳定使用的接口进入软件生态，而软件、编译器、系统软件和验证系统本身也在持续演化。因此，本报告关注的核心问题是如何在应用、架构、硬件和编译之间建立可复用、可检查、可验证的协同接口。

第一个核心科学问题关注应用需求如何通过架构接口转化为可复用的指令、微架构和加速器能力。领域应用中存在大量值得硬化或专门优化的结构，例如后量子密码中的数论变换和多项式运算，信号处理中的流式数据通路，图形和点云处理中的几何结构与不规则访存，以及大语言模型推理中的低精度算子、KV 缓存与注意力计算模式。但应用热点并不天然等价于好的硬件能力。若只从局部热点出发进行人工定制，容易得到面积代价高、适用范围窄、难以被编译器稳定调用的专用单元；若只从指令集架构格式或加速器接口出发，又可能忽略应用语义、访存路径、耦合方式和端到端工具链需求。因而，真正的挑战在于抽取具有跨程序、跨工作负载和跨平台价值的架构能力，并把它表达为编译器可识别、硬件可实现、系统软件可调用的接口对象。第二章将围绕这一问题展开，讨论端到端应用专用指令集处理器/定制指令扩展协同、可复用定制指令识别和微架构感知定制等研究基础。

第二个核心科学问题关注架构能力如何高质量落到可综合、可验证、性能与资源可控的硬件实现。架构接口只定义“应当提供什么能力”，并不自动保证该能力能够以高质量硬件形式实现。现实硬件设计还受到时序、资源、流水线、存储层次、接口协议、验证覆盖和后端工具行为的共同制约。传统寄存器传输级设计能表达细节但生产力低，高层综合能提高抽象层次却常缺少对微架构结果的直接控制；硬件领域语言和硬件中间表示能改善抽象与可组合性，但必须同时保留周期行为、控制结构、调度策略、耦合方式和目标实现属性。不同综合方法若缺少共同可检查的中间表示和可组合流程，也难以把高层综合、硬件生成器、静态/动态调度和后端综合机制组织起来。因此，该问题的关

键在于建立能够把架构意图、时序结构、资源约束、综合结果和实现质量同时纳入设计过程的硬件前端与优化机制，避免把问题简化为在若干描述层次之间作取舍。第三章将围绕这一问题展开，梳理从多层硬件综合基础设施、周期确定型前端，到硬件实现质量优化和变换-映射-调度联合优化的研究积累。

第三个核心科学问题关注编译接口如何组织程序语义、硬件能力和智能体行为，形成跨硬件架构的可验证、可复用编译协同机制。随着硬件能力日益异构，编译器已经成为软硬件协同生态中的中心接口，承担保留程序语义、表达目标硬件约束、组织优化搜索、调用验证和测试工具、利用性能剖析和执行轨迹支持下一轮设计的职责。大语言模型和智能体方法的出现进一步强化了这一点。模型可以提出策略、生成候选实现或执行长程搜索，但其输出必须被中间表示、编译过程边界、目标契约、语义检查和执行记录约束。编译基础设施因此应当成为智能体方法的协作媒介，使自动化过程有对象可操作、有边界可检查、有记录可回查。第四章将围绕这一问题展开，重点讨论大语言模型与形式化技术结合后如何支撑受约束的智能体编译优化与软硬件协同方法。

三个问题共同构成本报告的主逻辑链条。第一个问题决定领域需求能否进入架构层；第二个问题决定架构能力能否变成可靠硬件；第三个问题决定这些硬件能力能否被软件系统发现、验证、调用、改进并进一步支撑智能体方法。如果缺少第一个问题，硬件定制会失去应用来源；如果缺少第二个问题，架构想法难以形成可部署实现；如果缺少第三个问题，已有硬件能力无法进入可持续的软件生态，也无法支撑智能体方法在严肃工程场景中的可靠使用。

由此，本报告后续研究以跨硬件架构的软硬件协同生态为目标。研究将从应用模式中抽取可复用架构能力，通过高质量硬件前端和综合机制实现这些能力，再通过编译接口、形式化语义和执行记录把它们纳入可验证、可迭代的系统能力。第五章将在已有研究基础上进一步提出面向未来博士阶段的研究计划，重点围绕验证约束下的架构、硬件与编译协同生成，智能体编译基础设施，以及**面向工业级异构架构的算子优化与智能体化编译协同**三个方向展开。

1.6 已有研究基础与拟开展研究

围绕上述三个核心问题，本报告已有研究基础和拟开展研究形成一条连续主线。已有工作从应用需求出发，经由架构接口、硬件实现、编译接口和智能体化方法逐步展开，构成系统积累。它们共同支撑本报告的基本判断，即敏捷芯片设计与创新性编译技术在同一个软硬件协同体系中相互定义、相互约束、相互验证和共同演化。

第二章首先围绕“应用需求如何通过架构接口转化为可复用的指令、微架构和加速器能力”展开。APS^[113]和 Aquas^[114]关注应用专用指令集处理器与定制指令扩展的端到

端协同, 分别从开放 RISC-V 扩展接口、硬件合成、编译支持和复杂数据密集指令的访存接口属性、缓存影响、鲁棒编译映射等方面, 回答领域能力如何被接入完整工具链。ISAMORE^[115] 则进一步把问题推进到“定制什么”的层面, 通过等价图反合一从程序等价空间中发现可复用定制指令模式, 使定制指令从局部热点加速走向可跨程序复用的架构机制。Cayman^[116] 将在这一章作为领域定制与端到端协同的重要组成部分, 补充微架构约束、访存接口选择、候选选择和加速器合并之间的联系。第二章的主问题是第一个科学问题, 同时也为第三个问题提供编译可见的架构能力来源。

第三章围绕“架构能力如何高质量落到可综合、可验证、性能与资源可控的硬件实现”展开。HECTOR^[117] 作为早期高层综合与 MLIR 硬件基础设施积累, 说明硬件实现方法需要公共中间表示、可组合综合流程和对静态/动态调度的统一表达; 它在本报告中作为技术框架积累被强调, 但不按一作代表工作篇幅展开。Cement^[118] 关注周期确定型硬件前端, 试图在寄存器传输级精细控制与高层综合抽象之间建立更可预测的表达方式。Clay^[119] 将作为硬件前端与综合优化的重要工作, 重点讨论硬件实现质量、综合机制和设计约束之间的关系。SkyEgg^[120] 则进一步把代数变换、实现选择和调度放入统一等价图设计空间中, 体现从单阶段综合优化转向变换-映射-调度联合优化的思路。第三章主要回应第二个科学问题, 同时通过 SkyEgg 等工作连接到第三个问题中的编译接口和形式化优化空间。

第四章围绕“编译接口如何组织程序语义、硬件能力和智能体行为”展开, 并突出大语言模型与形式化技术结合对软硬件生态的作用。OriGen^[121] 作为开放寄存器传输级生成与修复方向的探索, 将被用于概括大语言模型进入硬件生成流程时对数据、语法、编译结果和自反思机制的需求。EggMind^[122] 则是本章重点, 它将大语言模型的策略生成能力放入 EqSatL、等价图和等价饱和语义约束下, 强调自动化优化必须输出可检查、可追踪、可复用的编译策略表示, 避免产生不可追踪的自然语言建议。第四章主要回应第三个科学问题, 也反向支撑前两个问题。只有编译接口能够表达和验证程序语义, 架构定制和硬件实现才有可能进入长期可维护的软件生态。

第五章在已有研究基础上提出未来工作。未来工作把三个核心科学问题合成为更完整的系统研究路线。第一, 研究验证约束下的架构、硬件与编译协同生成, 使架构接口、硬件生成和编译支持能够在形式化约束与执行记录下共同演化。第二, 研究智能体编译基础设施, 把编译器作为智能体方法的协作媒介, 使大语言模型和智能体能够围绕中间表示、编译过程、目标契约、语义检查和经验记录进行受约束协作。第三, 研究面向工业级异构架构的算子优化与智能体化编译协同, 把 PyTorch/Dynamo^[87]、TensorRT-LLM^[88] 等框架所体现的架构约束、系统软件边界和长程优化问题, 推进到更复杂的跨硬件架构场景中。

图 1.2 概括了上述关系。第二至第四章分别以已有研究基础回应三个核心科学问题，并在相邻层次之间形成支撑关系；第五章则把这些问题重新组合为面向博士阶段后续研究的三个方向。这样的组织方式强调每项工作在整体研究链条中的位置，也明确未来研究将从单点方法走向跨层协同系统。

研究问题与章节映射。问题 1（应用需求到架构接口）对应第二章；问题 2（架构能力到高质量硬件实现）对应第三章；问题 3（编译接口组织程序语义、硬件能力和智能体行为）对应第四章；第五章将三者合成为验证约束生成、智能体编译基础设施、工业级异构架构算子与智能体化编译协同三个未来方向。

图 1.2 研究问题、已有工作章节与未来研究方向的对应关系

第二章 面向领域定制的架构接口与端到端协同

2.1 引言

第一章指出，领域定制计算的核心难点在于把应用需求转化为可以被软硬件系统长期使用的架构能力。本章围绕这一问题展开，讨论应用需求如何通过架构接口转化为可复用的指令、微架构和加速器能力。这里的“架构接口”既包括处理器与定制指令单元之间的硬件协议，也包括指令语义、存储访问路径、编译器识别机制、软件可见行为和设计者能够迭代使用的工具链产物。

围绕这一问题，本章讨论端到端协同框架、可复用能力发现和微架构约束三个层次。APS 和 Aquas 关注领域定制能力如何接入完整应用专用指令集处理器/定制指令扩展工具链。APS 解决开放 RISC-V 生态中接口分裂、硬件合成和编译支持割裂的问题；Aquas 进一步面向复杂数据密集定制指令扩展，把访存接口属性、缓存效应、MLIR/Aquas-IR、等价图重写和骨架-组件匹配结合起来，使硬件侧访存机制选择和软件侧自动卸载能够共同优化。随后，ISAMORE 将问题推进到“应该定制什么”本身，强调定制指令面积昂贵，必须从跨程序语义等价空间中发现可复用指令模式。最后，Cayman 补充微架构感知定制，使领域能力选择与具体微架构约束形成更紧密的端到端协同。

这三个层次共同回应第一个科学问题。APS/Aquas 说明领域能力如何进入可运行、可评估、可综合的工具链；ISAMORE 说明领域能力如何避免局部热点定制，转向跨工作负载可复用的架构机制；Cayman 则说明架构能力需要结合微架构结构、接口代价和实现约束进行选择。本章把这些工作作为同一研究链条的不同阶段，先建立可接入系统的协同框架，再解决可复用定制能力发现，最后把定制能力与微架构事实对齐。

2.2 应用专用处理器与定制指令的端到端协同

应用专用指令集处理器（ASIP）和定制指令扩展（ISAX）是领域定制计算中兼顾可编程性和专用硬件效率的重要形态。与传统固定加速器相比，定制指令和专用执行单元能够保持通用处理器的软件入口，使应用在普通控制流、库函数和系统运行环境中调用领域能力；与纯软件优化相比，它们又能把频繁出现的计算、访存或控制模式硬化到专用数据通路中。因此，ASIP/ISAX 是连接应用需求与架构能力的自然接口。但这一接口真正可用，需要同时解决硬件协议、指令语义、合成实现、编译器调用和系统级评估问题。

2.2.1 统一定制指令接口与端到端工具链

APS^[113] 的出发点是开放 RISC-V ASIP 生态中的端到端割裂。RoCC 和 CV-X-IF 等接口都采用处理器与定制单元解耦的握手式交互，但二者在 commit/recall、issue/response、内存访问和控制信号组织上存在差异。RoCC 偏向简化加速器设计，并由处理器侧保证被卸载指令的提交；CV-X-IF 则显式暴露 commit channel，使带副作用的操作必须等待确认后才能执行。二者具有相近的交易语义，却因为协议细节、实现语言和平台生态不同而难以复用同一 ISAX 设计。APS 因此提出基于事务的统一 ISAX 接口抽象 APS-itfc，把 issue、GPR request、recall、memory read/write、memory response 和 result response 等交互统一为有效-就绪事务，并通过后端适配器自动映射到 RoCC 或 CV-X-IF。

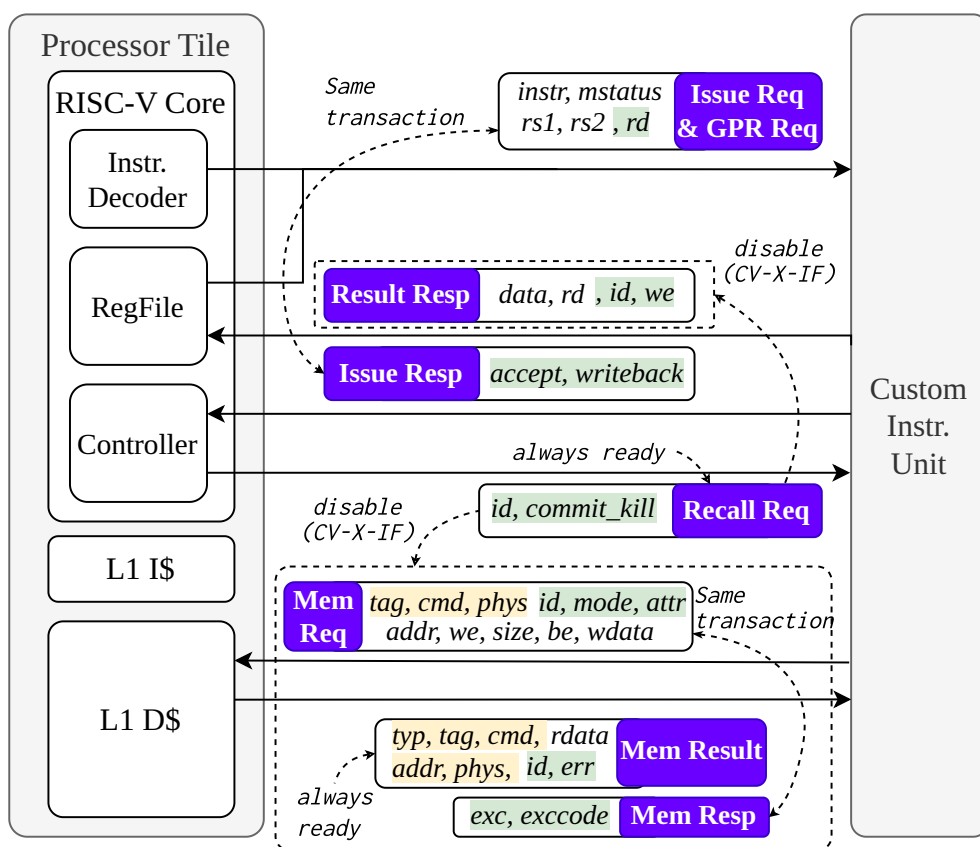


图 2.1 APS 对 RoCC 与 CV-X-IF 处理器-定制指令接口的统一抽象。图中核心关系是把平台相关信号组织为统一事务语义，再通过后端适配器接入不同 RISC-V SoC。

APS-itfc 的意义在于为后续硬件合成和编译支持建立共同边界，超越对接口信号差异的表层隐藏。定制指令单元只需要面向统一事务语义描述与处理器的交互，RoCC 或 CV-X-IF 的 `tag`、`size`、`commit/recall` 等平台细节由适配器处理。APS 在 Rocket/Chipyard 与 CV32E40X/Croc 两个开源 RISC-V 平台上实现这一抽象，并补充参数化缓存、Verilator 周期精确仿真、Yosys/OpenROAD 专用集成电路流程等工具链环节，使同一类定制能力

能够在不同片上系统框架中完成集成、仿真和物理实现评估。这一设计回答了 ASIP 研究中的第一层问题。领域能力必须有一个跨平台、可落地的架构接口，否则后续合成和编译优化会被具体处理器协议锁死。

在硬件侧，APS^[113] 通过 CADL 和 APS-synth 提供定制指令扩展专用综合。CADL 允许设计者描述定制指令的高层行为，同时直接访问统一 APS-itfc 事务和低层硬件组件；其描述内容被降低为结构化中间表示（SIR）和调度后结构化中间表示（SSIR）。SIR 保留序列、块、循环、循环携带变量、方法调用等结构，便于类型推导、函数解释和程序优化；SSIR 则把调度结果表达为流水线/阶段结构，用于估计性能并生成硬件。APS-synth 在调度中加入定制指令特有约束，例如寄存器文件读取必须位于触发阶段、延迟不敏感操作不能错误并列以避免死锁，并最终构造基于事务的动态流水线。这使 APS 不仅能生成组合逻辑式指令，也能描述带状态、循环、流水和定制组件的更复杂定制指令扩展行为。

在软件侧，APS^[113] 提供 APS-c 编译框架，把 CADL 语义导出为编译器可用的匹配信息，并在 LLVM 中生成类似源语的 C 包装函数、基于语义的匹配器、基于性能剖析的匹配器和位宽感知向量化。基于语义的匹配器面向控制较简单的中间表示模式，自动识别等价指令序列并替换为定制指令扩展调用；基于性能剖析的匹配器面向复杂循环或控制区域，通过解释执行和输入输出行为比较辅助发现可以卸载的代码区域。位宽感知向量化则针对 RV32 等架构中低位宽操作无法充分利用寄存器位宽的问题，把多个低位宽标量定制指令扩展调用打包为单指令多数据风格调用，并在超过寄存器带宽时自动插入辅助 memory/config 操作。由此，APS-c 把定制指令从“需要程序员手写源语”推进到“编译器根据硬件语义自动发现和使用”的形态。

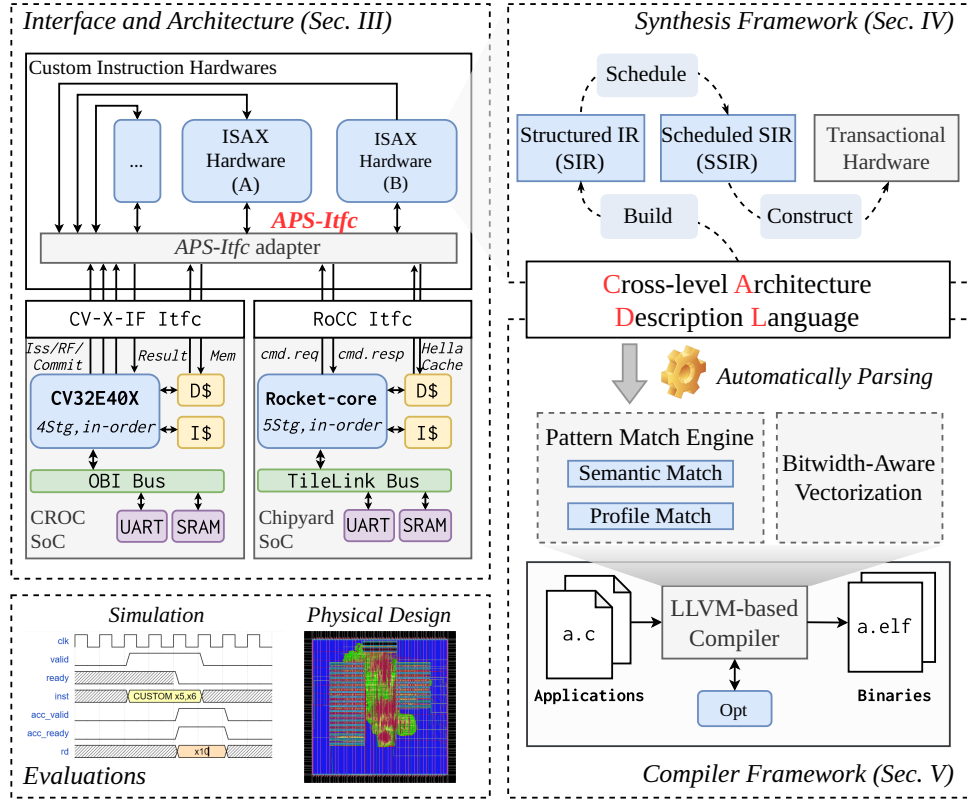


图 2.2 APS 端到端流程。统一接口、CADL/APS-synth、APS-c 编译支持和 SoC/ASIC 评估共同构成敏捷 ASIP 迭代链条。

APS^[113] 的评估覆盖后量子密码、量化大语言模型和数字信号处理三个领域，展示了端到端工具链的实际效果。在 CRYSTALS-KYBER 中，NTT butterfly 与 PWM/Karatsuba 等定制指令扩展用约 175 行 CADL 描述，并在两个 RISC-V 平台上获得明显加速；Butterflyx2 在两个平台上最高达到 10.16x，Karatsuba 对 PWM 基线最高达到 14.99x。在 BitNet b1.58 的 BitLinear 场景中，dotprodW2A8x4 定制指令扩展针对 8 位激活与 2 位权重的点积，完整 BitLinear 层获得 2.03x 和 2.29x 加速，并可由基于性能剖析的匹配器自动利用。在数字信号处理场景中，SOS 与 IIR 定制指令扩展支撑 DPLL 中的 IIR filter 加速，IIR 指令通过深流水和硬件循环消除软件调用与循环开销，在 CROC 和 Rocket 上分别获得 5.51x 和 5.18x 加速。这些结果说明 APS 的价值不仅是某个指令加速，也在于把指令设计、硬件合成、编译插入、仿真评估和专用集成电路性能、功耗、面积结果连成完整敏捷迭代流程。

然而，APS 也暴露出下一层挑战。它已经把定制指令接入统一接口和编译流程，但在复杂数据密集应用中，性能往往受限于存储层次、接口带宽、缓存行为和更复杂的软件变体。APS 的 CADL 和编译器能够描述较一般的行为，但对块级内存访问、暂存存储使用、突发传输、缓存层次不匹配、复杂循环变换和鲁棒定制指令扩展匹配的支持仍不充分。APS 建立了端到端协同框架，但当领域定制从标量/类单指令多数据指令推进

到更复杂的访存中心型定制指令扩展时，架构接口必须进一步显式表达访存机制和缓存效应，编译器也必须在更大的程序等价空间中进行稳健匹配。

2.2.2 面向数据密集指令的软硬件协同优化

Aquas^[114]正是沿着这一方向推进。它仍以应用专用指令集处理器/定制指令扩展为对象，但把问题从“统一接口和基本端到端流程”扩展为“复杂数据密集定制指令扩展的硬件-软件整体协同优化”。在硬件侧，Aquas 首先建立 core-ISAX 访存接口模型，用宽度、最大节拍数、在途事务数、读启动延迟、写完成代价和缓存行大小等参数刻画不同访问路径。这个模型说明，接口选择具有架构级影响。窄低延迟接口和宽高延迟接口在不同访问粒度、突发传输能力和缓存层次下会产生不同性能结果，错误选择可能引入 7 到 9 个周期的局部延迟惩罚，并在更大设计中放大为系统性能损失。

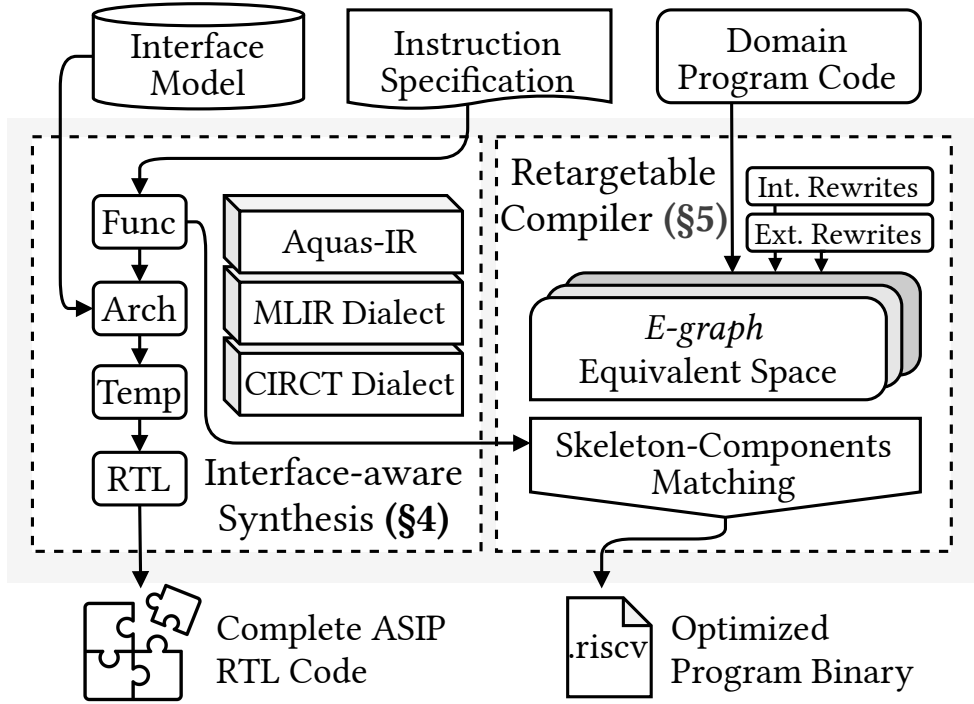


图 2.3 Aquas 的硬件-软件协同流程。硬件侧以访存接口模型和 Aquas-IR 支撑合成，软件侧以 MLIR、等价图重写和骨架-组件匹配支撑自动卸载。

为了把这些硬件事实纳入合成，Aquas^[114]设计了三层 Aquas-IR。功能层接近高层软件语义，描述 `transfer`、`fetch` 等与具体访问机制无关的操作；架构层引入 `memitfc` 符号，把访存操作绑定到具体接口，并使其受到对齐、事务粒度、未完成请求数量限制等微架构约束；时序层则用异步 `issue/wait` 和顺序属性表达事务顺序、缓存层次和有限并发下的具体时序。基于这一中间表示，Aquas 进行暂存缓冲消除、接口选择与规范化、事务调度与排序，以及最终硬件生成。其关键意义在于，访存接口选择成为可分析、可搜索、可在中间表示中回放的架构决策，而不再依赖手工经验或后端偶然结果。

在软件侧，Aquas^[114]进一步解决复杂定制指令扩展卸载的鲁棒编译问题。复杂定制指令扩展的语义常与应用代码存在抽象层次差异。软件表示关注值、控制流和访存效应，而硬件描述中可能包含暂存存储、寄存器交互和接口传输语义。Aquas 首先把软件代码通过 Polygeist^[64]转到 MLIR 基础方言，并把 Aquas-IR 功能层中的硬件语义规范化到同一抽象层次。随后，它把 MLIR 与等价图结合：MLIR 保留结构化控制流和副作用顺序，等价图非破坏性地累积等价程序变体；内部重写用于代数和数据流变换，外部重写则通过调用 MLIR 循环变换编译遍实现分块、展开等控制流重构。为了避免等价图爆炸，Aquas 根据目标定制指令扩展的循环特征有选择地触发外部重写，最后通过骨架-组件匹配检查控制结构、数据流组件、支配关系、排序和效应约束，自动插入定制指令扩展调用。

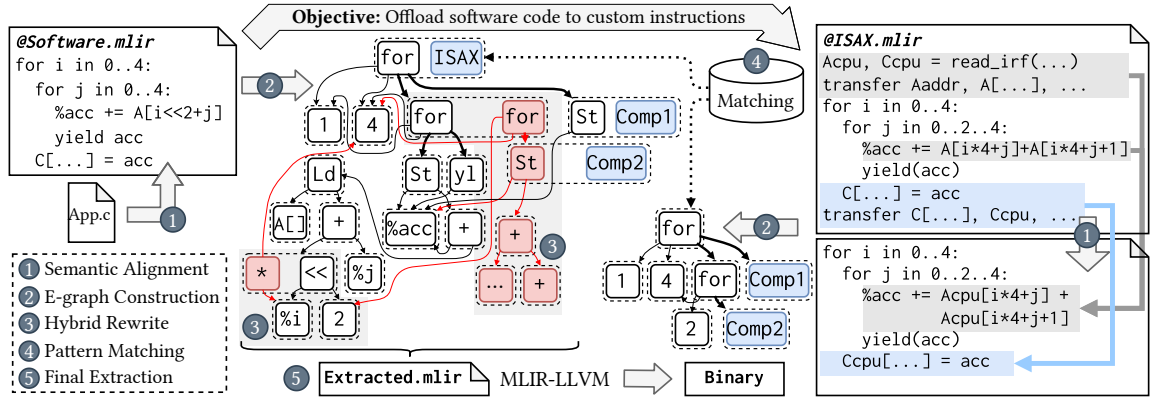


图 2.4 Aquas 编译器流程。MLIR 负责结构化程序表示与外部循环变换，等价图负责累积等价变体，最终通过骨架-组件匹配插入复杂定制指令扩展调用。

这种设计把 APS 中的自动源语插入扩展到更复杂的语义空间。APS 的模式匹配已经能够处理部分语义和基于性能剖析的场景，但 Aquas 更强调同一个定制指令扩展可能对应许多语法不同、控制流形态不同、但语义等价的软件实现；编译器若只匹配单一模式，会被循环分块、展开、代数变换、冗余语句和非仿射表达轻易破坏。因此，Aquas 把“匹配一个模式”转化为“在受约束的等价空间中寻找与目标定制指令扩展骨架/组件对齐的程序变体”。这使编译接口更接近第一章所说的协作媒介，目标事实、中间表示、重写、匹配、代价模型和检查记录共同约束卸载决策。

Aquas^[114]的评估进一步说明了这一复杂协同的必要性。它在后量子密码、点云处理、图形渲染和中央处理器大语言模型推理四类场景中评估真实工作负载。对于基于编码的后量子密码伴随式计算，Aquas 为位流解包和 $GF(2^n)$ 矩阵乘法设计定制指令扩展，并在 Rocket 基线上获得 7.59x 和 3.29x 内核加速比，端到端获得 1.42x 加速比，面积开销小于 5.3% 且无频率下降。在点云 ICP 场景中，Aquas 针对欧氏距离、协方差矩阵、最大值比较和矩阵-向量乘法等操作设计定制指令扩展，内核加速比达到 1.46x 到 9.27x，

端到端达到 1.96x；其优势来自对不规则访存、矩阵/向量混合访问和扩展总线带宽的更好利用。在图形渲染中，Aquas 与 RISC-V 向量单元 Saturn 比较，多个内核获得 9.47x 到 15.61x 加速，并在性能面积权衡上优于通用向量单元。在中央处理器大语言模型推理的现场可编程门阵列原型中，Aquas 针对注意力计算设计定制指令扩展，在 Llama 2 110M 8-bit 量化模型上获得 9.30x 首词延迟和 9.13x token 间延迟加速。这些结果说明，当应用具有复杂访存和控制形态时，端到端协同必须同时考虑硬件访存接口、编译等价空间和真实系统评估。

从研究链条看，APS 和 Aquas 共同回答了“如何把定制能力端到端接入工具链”这一问题。APS 建立了跨 RISC-V 平台的统一接口、定制指令扩展专用综合和自动编译插入，使领域能力能够从 CADL 和 C 程序进入可仿真、可综合、可评估的片上系统。Aquas 则进一步把复杂数据密集场景中的访存接口、缓存层次、MLIR/Aquas-IR、等价图重写和骨架-组件匹配纳入同一流程，使定制能力不仅能接入工具链，还能在更复杂硬件事实和软件变体中被正确、鲁棒、有效地使用。二者为后续 ISAMORE 提供了基础。端到端接入问题被系统化以后，下一步关键就变成如何从应用集合中自动发现真正值得接入这些接口的可复用架构能力。

2.3 可复用定制指令发现

2.3.1 研究背景与问题建模

APS 和 Aquas 解决的是“如何把定制能力接入端到端工具链”的问题，但它们仍然预设设计者已经知道哪些能力值得定制。对于敏捷芯片设计而言，这个前提并不总成立。定制指令会占用面积、引入验证负担，并影响处理器接口、编译器后端和系统维护；如果一个指令只服务于少数局部热点，它即使在单个内核上有效，也很难成为长期可复用的架构能力。因此，在端到端接入工具链之上，还需要回答一个更前置的问题，即如何从一组领域代表程序中自动发现具有跨程序、跨工作负载价值的可复用定制指令。

传统自动指令发现大多以程序热点和语法结构为中心。细粒度方法枚举基本块内的凸子图，能够找到局部可替换片段，但受限於基本块范围和较小模式，容易错过更大粒度的优化机会；粗粒度方法把基本块或代码区域整体合并为加速单元，能够覆盖更多操作，却常生成过大的专用硬件，复用次数低，甚至包含在通用处理器上更适合执行的指令序列。二者共同的问题是过度依赖语法分析。如果两个代码片段语法不同但语义等价，它们往往被视为不同模式，最终产生多个相似但不共享的硬件单元。ISAMORE^[115]的问题意识正来自这一点。真正适合作为指令集架构扩展的能力，应当是语义上可

泛化、在多个位置反复出现、且硬件代价与性能收益匹配的模式，局部热点中的语法片段只提供初始线索。

ISAMORE 将可复用定制指令识别建模为跨程序模式发现问题，并提出 RIMT 方法。其输入是一组领域代表程序，输出是一组在性能和面积之间形成帕累托权衡的定制指令候选。这个过程首先把 LLVM 中间表示转换为结构化领域专用语言，再编码为等价图；随后通过分阶段等价饱和和等价图反合一发现语义等价空间中的公共模式；最后用硬件感知代价模型选择值得实现的模式，并通过高层综合和 RoCC 包装后端生成可评估硬件。与 APS/Aquas 相比，ISAMORE 的位置更靠前。它不仅让编译器使用已有定制指令扩展，也帮助设计者决定哪些定制指令扩展值得进入后续工具链。

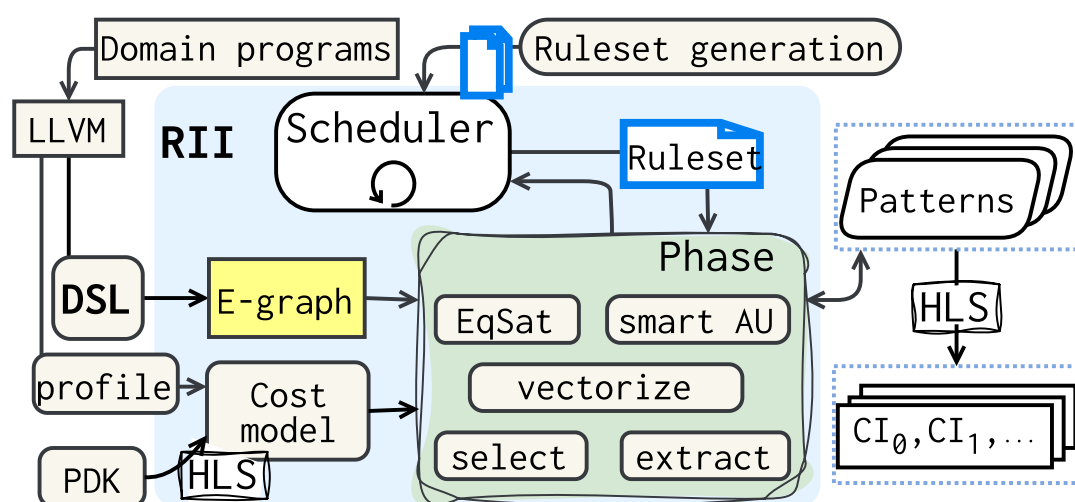


图 2.5 ISAMORE/RIMT 总体流程。领域程序先进入结构化领域专用语言和等价图表示，再经过分阶段等价饱和、反合一、向量化、硬件感知选择和后端生成，形成可复用定制指令集合。

2.3.2 结构化表示与等价图反合一

为了让一般程序能够进入等价图反合一，ISAMORE^[115] 首先设计结构化领域专用语言。该语言覆盖算术、逻辑、访存等常规操作，同时引入 Loop 和 If 表达控制流，并提供 Vec、向量一元/二元/三元运算、模式变量和 App 等结构。Loop 以类似 do-while 的风格表达循环携带变量、循环条件和循环体结果，使控制流能够以结构化、数据流中心的方式进入等价图。该语言还具有强类型系统，利用等价类分析推导每个等价类的结果类型，并对 If 和 Loop 的输入输出元组施加结构约束。这个设计解决了直接把一般程序放入等价图的第一层障碍，使程序在进入等价图后仍能保留循环、条件、类型和向量结构等可泛化语义，而不局限于若干代数表达式。

在此基础上，ISAMORE 的核心方法是等价图反合一。等价图用等价类表示等价项集合，等价饱和通过重写规则暴露语义等价形式；反合一则在两个项之间求最小公共泛

化, 从而发现公共结构。例如, 两个语法不同但可重写到同一等价形式的表达式, 可以被泛化为带模式变量的候选指令模式。这个思路把“复用”从语法匹配提升到语义等价空间: 同一个定制指令可以覆盖多个表面形态不同的程序片段, 只要它们在等价图中共享可泛化结构。

2.3.3 分阶段识别流程与可扩展反合一

直接使用朴素等价图反合一难以处理真实程序, 因为等价类数量和候选反合一模式数量会迅速爆炸。ISAMORE^[115] 因此提出分阶段迭代。它不追求一次性对所有重写规则做完整饱和, 而将规则按饱和型/非饱和型、整数/浮点、向量/标量等维度组织为基础规则集, 并由阶段调度器逐步选择规则集。早期阶段先应用不会显著增加等价类数量的饱和型规则, 尽可能暴露低成本等价形式; 后续阶段再以受限次数应用非饱和型规则, 以避免等价图规模失控。每个阶段中, RIMT 会把以前识别的模式作为模式应用重写重新注入等价图, 使后续阶段能够在已有模式的基础上进一步泛化。这样做既控制了搜索规模, 又允许指令模式从局部公共结构逐渐演化为更高复用度的候选。

RIMT 的第二个关键是智能反合一。朴素反合一需要枚举大量等价类对, 并对每对等价节点递归生成候选模式, 实际复杂度很快失控。智能反合一用两类启发式降低复杂度。第一是基于相似度的等价类配对。ISAMORE^[115] 用类型信息排除结果类型不一致的等价类, 再通过结构哈希估计等价类的结构相似度, 只对相似度超过阈值的配对做反合一。结构哈希对字面量、参数和模式变量做统一处理, 重点刻画结构并弱化具体常量差异, 从而更适合寻找可复用模式。第二是启发式反合一模式采样。当某个等价节点对的反合一候选过多时, ISAMORE 用边界或 kd-tree 策略根据延迟/面积特征选取代表性模式, 避免保留全部笛卡尔积。前者保留特征值极端的模式以提高效率, 后者在特征空间中取更广覆盖以改善质量。二者共同使等价图反合一从理论上有吸引力但不可用的方法, 变成可以处理真实程序的指令识别机制。

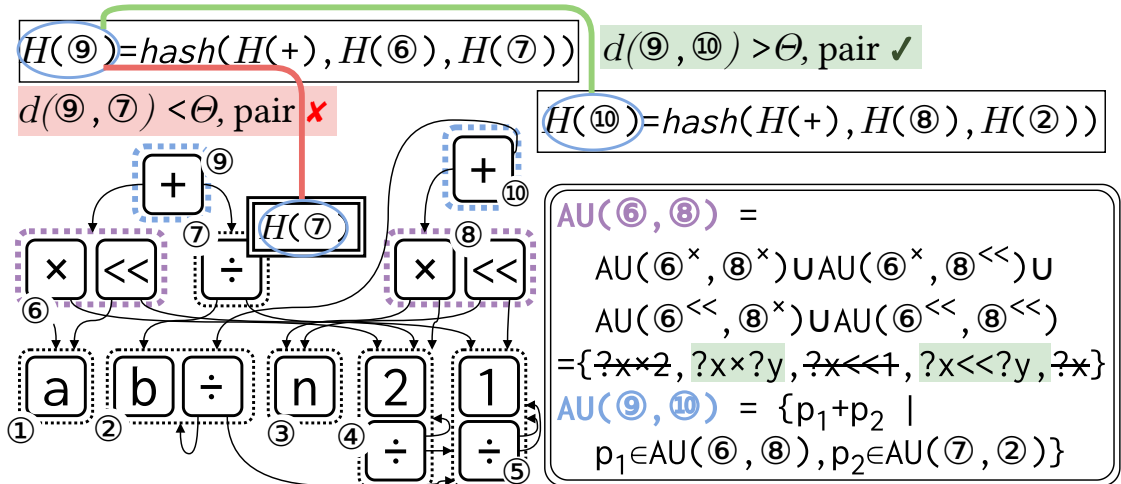


图 2.6 ISAMORE 的智能反合一机制。相似度筛选和候选采样共同控制等价图反合一的搜索规模，使可复用指令模式发现能够扩展到真实程序。

2.3.4 向量化、硬件感知选择与真实系统评估

除了语义复用，ISAMORE^[115]还把数据级并行纳入指令发现过程。传统指令发现往往把标量模式作为主要对象，即使程序中存在多路相似操作，也不一定生成向量化定制指令。ISAMORE 的模式向量化把一个向量化模式看作同一个模式在多个通道上的应用。其流程先通过智能反合一找到可重复出现的标量种子，再把同一基本块中的多个种子打包成 Vec 节点；随后通过提升重写恢复 VecAdd、VecMul 等向量构造，并用耦合重写建立向量到标量通道的数据流；最后用贪心提取和压缩式剪枝消除 Get-Vec-Get 环与组合式打包爆炸，只保留高数据级并行度的非重叠向量化方案。这使定制指令发现不局限于“找到一个可复用标量片段”，而能同时探索向量化、硬件循环和更高吞吐的数据通路。

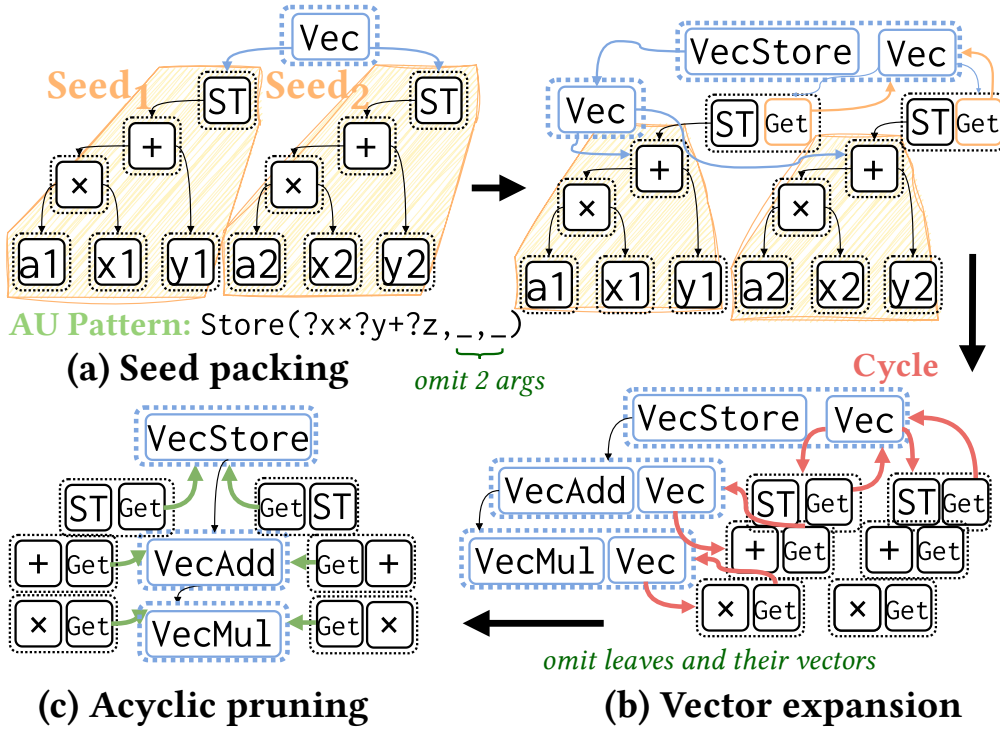


图 2.7 ISAMORE 的模式向量化。多个可复用标量模式被组合为向量化模式，从而把数据级并行性纳入定制指令发现。

在选择指令候选时，ISAMORE^[115]使用基于性能剖析、硬件感知的代价模型，避免依赖抽象语法树规模这类硬件无关目标。对每个模式，它估计该模式在所有使用位置替换为硬件单元后节省的总延迟。软件延迟来自剖析得到的单操作周期数，硬件延迟来自轻量高层综合对模式行为的调度，面积来自高层综合/OpenROAD 估计。RIMT 通过等价类分析在等价图中传播帕累托前沿，把模式集合的加速比与面积开销作为多目标

选择依据，并在提取后重新计算更准确的性能面积指标进行保真度修正。这个设计保证 ISAMORE 发现的模式具有实际性能收益和可接受硬件代价，语法紧凑或项数更少本身不足以成为选择依据。

ISAMORE 源文中的硬件感知代价模型将模式 p 的多次复用收益、模式集合 $\mathcal{P}$ 的整体加速比和面积开销写为：

$$\Delta_{L(p)} = \sum_{u \in \text{use}(p)} \left(\sum_{o \in p} \text{CPO}(\text{bb}(o, u)) - L_{\text{HLS}(p)} \right) \quad (2.1)$$

$$S(\mathcal{P}) = \frac{L_{\text{cpu}}}{L(\mathcal{P})} = \frac{L_{\text{cpu}}}{L_{\text{cpu}} - \sum_{p \in \mathcal{P}} \Delta_{L(p)}} \quad (2.2)$$

$$A(\mathcal{P}) = \sum_{p \in \mathcal{P}} A_{\text{HLS}(p)} \quad (2.3)$$

等价类分析在等价图中传播帕累托前沿。对普通构造子、模式应用节点和等价类，源文分别给出如下递推：

$$\mathcal{F}(s(a_1, \dots, a_k)) = \prod_i \mathcal{F}(a_i) = \{\cup_i \mathcal{P}_i \mid \mathcal{P}_i \in \mathcal{F}(a_i)\} \quad (2.4)$$

$$\mathcal{F}(\text{App}(p, a_1, \dots, a_k)) = \left\{ \mathcal{P} \cup \{p\} \mid \mathcal{P} \in \prod_i \mathcal{F}(a_i) \right\} \quad (2.5)$$

$$\mathcal{F}(a) = \text{prune}\left(\bigcup_{n \in M(a)} \mathcal{F}(n)\right) \quad (2.6)$$

ISAMORE^[115] 的基准测试评估展示了这种方法的可扩展性和收益。在九个来自计算机视觉、机器感知、数字信号处理和密码学的内核上，朴素等价图反合一因候选模式爆炸而在所有案例中超过 30GB 内存限制；启用 RIMT 后，所有基准测试均在 145 秒内完成，内存不超过 799MB，峰值等价图规模相比朴素方法降低 6 到 39 倍。在性能上，ISAMORE 相比 NOVIA 等语法合并方法获得更高加速比和更低面积开销；启用向量化后，ISAMORE 相对 NOVIA 的优势平均达到 1.76x，最高达到 2.69x，并在多个基准测试中展示模式向量化对性能的进一步贡献。MatChain 案例还表明，结构化领域专用语言编码控制流后，ISAMORE 可以识别可复用硬件循环，生成循环流水加速器，获得远超基本块级方法的优化粒度。

更重要的是，ISAMORE^[115] 在真实开源库和具体硬件实例中体现了“可复用架构能力”的价值。对 liquid-dsp、CImg 和 PCL 等库的研究显示，语法合并方法往往生成巨大且低复用的硬件单元，而 ISAMORE 能识别多个更小、更高复用的定制指令。在 CImg 中，NOVIA 生成一个规模为 167 的大硬件单元且只合并少量基本块，ISAMORE 则识别八条平均复用 93 次的指令，获得 1.18x 加速比，同时面积仅为 975 平方微米，相比 NOVIA 节省 90.5% 面积。在 PCL 模块中，ISAMORE 相比 NOVIA 平均获得 1.64x 加速比，最高 2.73x，并节省 93.2% 面积。这些数据说明，可复用性是决定定制指令是否适合进入指令集架构的核心标准。

ISAMORE^[115] 还在量化大语言模型和后量子密码中展示了从自动识别到真实硬件集成的可行性。对 BitNet b1.58 的 BitLinear 推理实现，ISAMORE 从 bitnet.cpp 风格实

现中识别出打包低比特点积的向量化模式，并生成带 RoCC 包装器的定制硬件单元；在 Rocket tile 上通过 Verilator 寄存器传输级仿真获得 2.15x BitLinear 加速比，OpenROAD 报告 4.81% 面积开销且无频率下降。对 CRYSTALS-KYBER 的 NTT 实现，ISAMORE 自动识别正向 NTT 和逆向 NTT 复用的蝶形定制指令，并在 Rocket tile 上获得 5.15x 加速比，面积开销主要来自硬件乘法器。这些案例证明，ISAMORE 能够与 RISC-V/片上系统、高层综合、寄存器传输级仿真和物理设计流程连接，具备进入实际敏捷芯片设计流程的能力，已经超出静态分析工具的范围。

从本章主线看，ISAMORE 对 APS/Aquas 形成了关键补充。APS/Aquas 使一个已知定制能力能够被架构接口承载、被硬件工具实现、被编译器调用并被系统评估；ISAMORE 则反过来从领域程序集合中发现哪些能力值得进入这一流程。其方法创新在于把等价图等价饱和、反合一、结构化控制流领域专用语言、分阶段搜索、智能反合一、模式向量化和硬件感知选择结合起来，使“可复用定制指令”成为可自动发现、可量化权衡、可硬件落地的敏捷芯片设计机制。由此，领域定制可以由程序语义、硬件代价和跨工作负载复用共同驱动，逐步减少对专家经验挑选热点的依赖。

2.4 微架构感知的领域加速器定制

2.4.1 研究背景与整体框架

APS、Aquas 和 ISAMORE 分别从端到端接入、复杂访存协同和可复用指令发现三个角度推进了领域定制处理器研究。Cayman^[116] 则把这一链条进一步推向微架构感知的定制加速器生成。与定制指令扩展更强调处理器指令接口不同，Cayman 面向的是从应用程序中自动选择适合硬件加速的内核，并生成高性能定制加速器；它关注代码片段是否包含复杂控制流、如何组织数据访问、如何通过处理器-加速器接口降低数据移动代价，以及多个加速器之间是否可以合并和复用。

Cayman 的问题背景与本章前两节自然衔接。APS 说明，如果缺少统一接口和合成/编译工具链，定制能力难以跨平台落地；Aquas 说明，如果不显式建模访存接口和缓存效应，复杂数据密集定制指令扩展会被错误的内存路径和低效数据移动限制；ISAMORE 说明，如果只从局部热点做语法合并，定制硬件会过度专用、复用度不足。Cayman 面对的是同一矛盾在加速器生成场景中的体现。高层综合可以从给定内核合成硬件，但内核的选择和抽取仍常依赖人工；真实应用中的候选区域可能包含一般控制流、复杂数据访问和多个可复用加速单元，需要结合程序结构、性能剖析和微架构约束共同判断。

Cayman^[116] 的整体流程从应用程序出发，将程序编译到 LLVM 中间表示，并构建全应用程序结构树（whole-application program structure tree, wPST）表示；同时，系统

通过性能剖析和 LLVM 程序分析收集每个候选区域的执行时间、执行次数、内存依赖、访问模式和访存足迹。随后, Cayman 基于 wPST 和分析结果进行候选选择, 生成由多个内核与加速器配置组成的候选解; 对每个候选解, 系统进一步执行加速器合并, 得到可复用加速器, 并最终综合硬件设计。这个流程把“候选区域选择”和“硬件实现质量”放入同一框架, 避免让高层综合后端被动接受人工选择的内核。

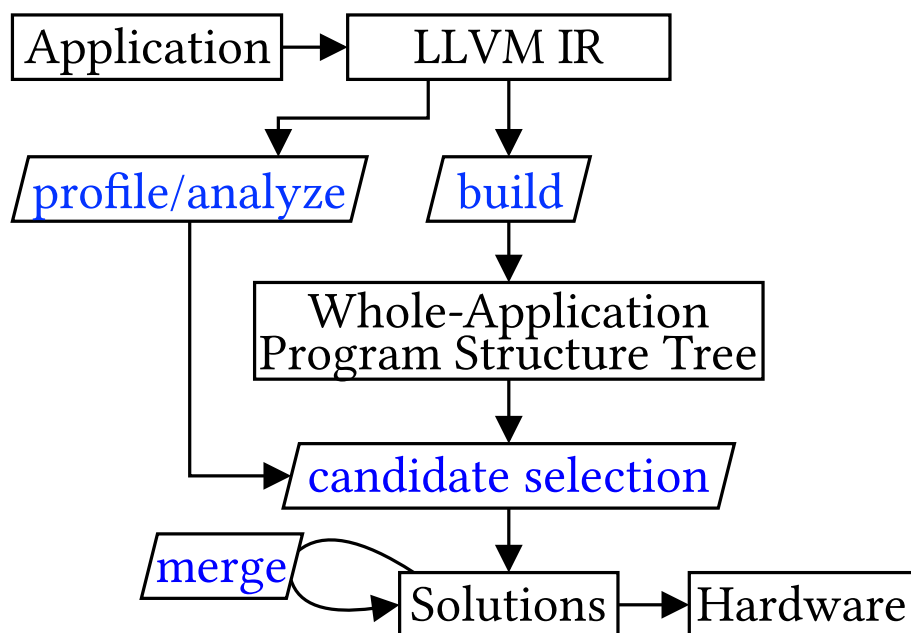


图 2.8 Cayman 总体框架。应用程序经全应用结构表示、性能剖析和程序分析进入候选选择, 再经接口建模、加速器合并和硬件综合形成可复用加速器。

2.4.2 全应用表示、性能剖析与程序分析

Cayman^[116] 引入 wPST 的原因在于, 传统数据流图级定制硬件生成难以处理真实程序中的控制流和跨函数结构。程序结构树 (program structure tree, PST) 本来用于识别函数内单入口单出口区域; Cayman 将其扩展为全应用程序结构树, 在根节点下加入函数节点, 并把每个函数中的单入口单出口区域表示为候选区域。单入口单出口约束很关键: 只有具有单一入口和单一出口的区域, 才能在卸载后通过入口/出口处的同步与主处理器隔离执行, 从而形成清晰的处理器-加速器边界。wPST 中的候选区域包括基本块区域和包含循环/条件的控制流区域, 使候选选择不再局限于平坦数据流图。

仅有区域树仍不足以决定哪些区域值得加速。Cayman^[116] 因此对应用进行性能剖析, 记录每个区域的持续时间和执行次数, 用于定位热点并估算潜在性能收益; 同时执行程序分析, 收集内存依赖、访存模式和访存足迹。具体而言, 系统识别循环携带依赖, 以判断循环流水化或展开是否可行; 识别流式访问模式, 以判断地址序列是否可静态生成; 分析每个访存操作的访问范围, 以估计暂存存储大小和缓存收益。这些分析结果直接进入后续加速器配置, 成为候选生成和接口选择的依据。

2.4.3 加速器建模与数据访问接口选择

Cayman^[116]的关键贡献之一是把处理器-加速器数据访问接口显式纳入建模。其模型包含三类接口。耦合接口使用加载/存储单元访问存储系统,加速器在发出访问请求后等待响应,面积开销较低但容易产生停顿。解耦接口为加载/存储单元配置独立地址生成单元,使读取可以提前发起、写入可以延后提交,从而减少加速器等待;其代价是需要地址生成单元和先进先出缓冲,并且只适用于地址可静态生成的流式访问。暂存接口在加速器内部保留专用缓冲,通过 DMA 在加速器执行前后与存储系统同步,适合访问足迹小且重复访问多的内存区域,但需要暂存缓冲、DMA 引擎和静态访存足迹信息。

这些接口直接影响控制流优化是否能转化为性能收益。例如在顺序循环中,解耦接口可以减少读取停顿;在循环流水化中,耦合接口可能使启动间隔受限,而解耦接口能使启动间隔接近理想值;在循环展开中,暂存存储可以为多个并行访问提供本地带宽。Cayman^[116]因此为选定内核合成多种加速器配置,组合循环展开、循环流水化和数据访问接口策略。其启发式策略是对无循环携带依赖的循环尝试展开,对最内层循环做流水化;当访问总次数显著大于访存足迹时使用暂存存储,当流水化循环中需要提前/延后访问时使用解耦接口,其余情况选择耦合接口以节省面积。

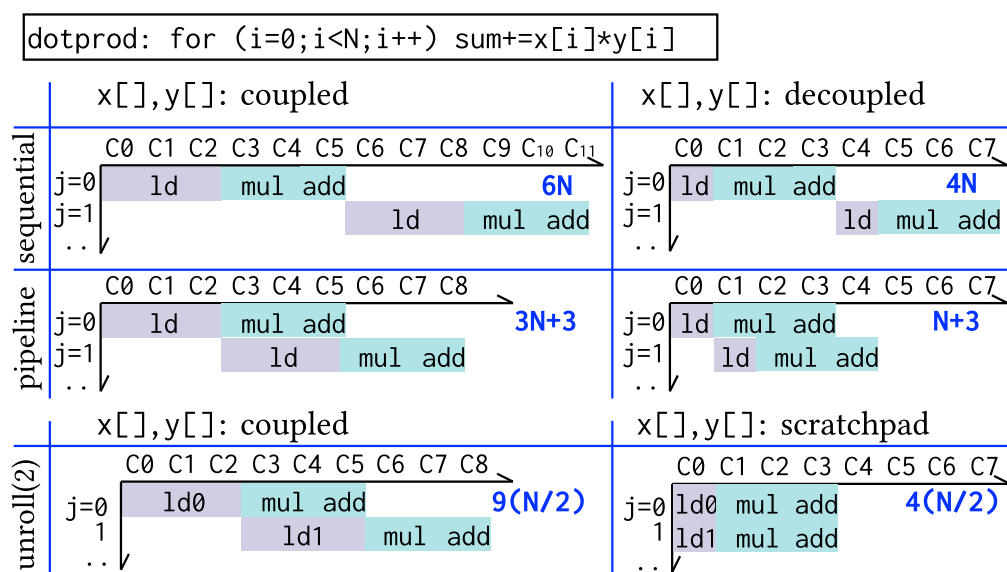


图 2.9 Cayman 数据访问接口对调度的影响。耦合接口、解耦接口和暂存接口在顺序循环、流水循环和展开循环中的延迟/启动间隔差异直接决定加速器收益。

为了避免对每个候选都完整综合硬件, Cayman^[116]还构建性能和面积估计模型。该模型先根据配置对内核做循环展开,再只综合流水化循环区域和顺序基本块区域,最后自底向上估计外层区域的周期数和面积。对于每个基本调度单元,周期数来自调度延迟与性能剖析得到的执行次数,面积来自综合报告;外层区域的延迟则由子区域累加

并加上控制逻辑开销。结合原始程序总时间、被加速候选区域的剖析时间和目标频率，Cayman 可以快速估算整体加速比，从而支持候选选择中的大规模探索。

2.4.4 候选选择算法与加速器合并

基于 wPST，Cayman^[116] 将候选选择建模为带有树形互斥约束的背包问题。每个 wPST 区域是一个候选项，其收益是加速后的性能收益，权重是面积开销；约束是如果选择某个区域，其所有后代区域不能再被选择，以避免同一程序区域被重复卸载。Cayman 使用动态规划求解该问题：对每个 wPST 节点 v ，维护其子树内帕累托最优解序列 $F[v]$ ；若 v 是基本块，则候选来自该基本块的加速配置；若 v 是控制流区域，则在“直接加速该区域”和“组合其子节点解”之间取帕累托前沿；若 v 是根节点，则组合不同函数的解。为了控制复杂度，Cayman 还使用基于性能剖析和程序分析的剪枝跳过不值得加速的区域，并通过解过滤移除面积过近的帕累托解，使每个子问题的解数量保持在可控范围内。

Cayman 源文中的性能估计、候选选择递推和过滤规则

收益估计将候选区域的剖析时间、估计硬件周期数和目标频率统一到端到端加速比中：

$$\text{Speedup} = \frac{T_{\text{all}}}{T_{\text{all}} - T_{\text{cand}} + \frac{C_{\text{cand}}}{f}} \quad (2.7)$$

其中 T_{all} 为原程序总时间, T_{cand} 为候选区域剖析时间, C_{cand} 为候选加速器估计周期数, f 为目标频率。

对 wPST 节点 v , 动态规划状态 $F[v]$ 表示其子树内的帕累托最优候选序列：

$$F[v] \leftarrow \text{筛选}(\text{帕累托}(\text{加速}(v, R))) \quad \text{若 } v \text{ 为基本块} \quad (2.8)$$

$$F[v] \leftarrow \text{筛选}(\text{帕累托}(\text{加速}(v, R) \cup \text{组合}_{u \in \text{子节点}(v)} F[u])) \quad \text{若 } v \text{ 为控制流区域} \quad (2.9)$$

$$F[v] \leftarrow \text{筛选}(\text{帕累托}(\text{组合}_{u \in \text{子节点}(v)} F[u])) \quad \text{其他情况} \quad (2.10)$$

兄弟子树通过张量组合表达互不重叠区域的联合选择：

$$F[u_1] \otimes F[u_2] = \text{帕累托}(\{\varphi_1 \cup \varphi_2 \mid \varphi_1 \in F[u_1], \varphi_2 \in F[u_2]\}) \quad (2.11)$$

解过滤规则移除面积过近的中间解。若 φ_j 是首个满足 $a_j > \alpha a_i$ 的解, 则删除 $\varphi_{i+1}, \dots, \varphi_{j-1}$, 使候选序列规模从 A 降到 $\log_{\alpha} A$ 量级; 总复杂度为

$$O(N(\log_{\alpha} A)^2 + E) \quad (2.12)$$

算法 2.1 Candidate Selection

```

1: function DP( $v, R$ )
2:   if prune( $v, R$ ) then
3:     return  $\emptyset$ 
4:   end
5:   if is-basic-block( $v$ ) then
6:      $F[v] \leftarrow \text{filter}(\text{pareto}(\text{accel}(v, R)))$ 
7:   end
8:   else
9:      $F[v] \leftarrow \emptyset$ 
10:    for  $u \in \text{children}(v)$  do
11:       $F[v] \leftarrow \text{filter}(F[v] \otimes F[u])$ 
12:    end
13:    if is-control-region( $v$ ) then
14:       $F[v] \leftarrow \text{filter}(F[v] \cup \text{pareto}(\text{accel}(v, R)))$ 
15:    end
16:  end
17: end
18: function Candidate-Selection( $T, R$ )
19:    $F[\text{root}] \leftarrow \text{DP}(T.\text{root}, R)$ 
20:   return  $F[T.\text{root}]$ 
21: end
    
```

候选选择解决“哪些区域加速”的问题, 但如果每个区域都生成独立加速器, 面积可能迅速膨胀。Cayman^[116] 因此提出加速器合并。其核心观察是, 多个区域即使控制流不同, 也可能包含相似的基本块数据流; 数据通路中的浮点运算和访存往往占主要面积, 而控制有限状态机相对较小。因此, Cayman 将基本块中共享操作合并为可重构数

据通路单元, 通过多路选择器和重配置位寄存器选择不同数据路径; 每个原始区域保留独立有限状态机, 执行时由全局控制单元发送配置并触发对应控制逻辑。系统以启发式方式估计每对基本块合并后的面积节省, 反复合并节省最大的候选对, 直到没有进一步收益。

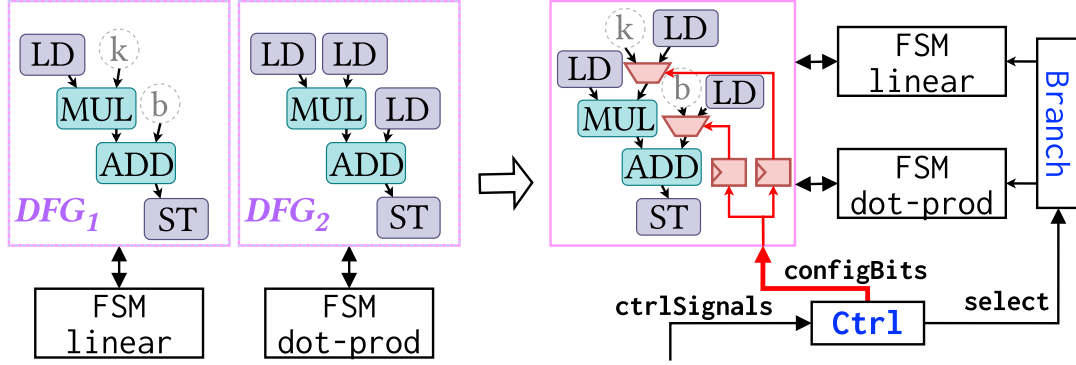


图 2.10 Cayman 加速器合并。不同控制流区域可共享相似基本块数据通路, 保留独立有限状态机并通过可重构数据通路降低面积。

2.4.5 实验评估与意义

Cayman^[116] 的评估使用 PolyBench、MachSuite、MediaBench 和 CoreMark-Pro 等多类基准测试, 并与 NOVA 和 QsCores 两个自动化加速器综合框架比较。实验设置中, 目标频率为 500 MHz, 面积以 CVA6 RISC-V tile 为归一化基准, 并评估小面积预算 (25%) 和大面积预算 (65%) 两种情况。在 25% 面积预算下, Cayman 相对 NOVA 和 QsCores 分别取得平均 14.4x 和 8.0x 加速比; 在 65% 预算下, 优势扩大到 27.2x 和 15.0x。这一结果来自 Cayman 对控制流优化和数据访问接口专门化的联合支持, 而 NOVA 不支持控制流/访存, QsCores 则主要生成顺序控制流和较慢访问接口。

更细的结果说明 Cayman^[116] 的收益来自机制本身, 单纯增加面积预算无法解释这些现象。对于热点分布均匀的程序, Cayman 在面积预算增大时选择更多内核; 对于热点集中的程序, 则把更多面积集中用于关键区域。接口选择结果显示, 解耦接口和暂存接口被广泛采用, 说明数据访问接口专门化是性能来源之一。加速器合并两个面积预算下分别节省约 36% 和 35% 面积, 每个可复用加速器平均加速多个程序区域; 在包含相似基本块的 3mm 基准测试中, 面积节省可达 70% 以上, 而在只有单个热点的 doitgen 中收益较低。这些现象与 Cayman 的设计预期一致。当程序中存在丰富控制流、重复数据通路和复杂访存时, wPST、接口建模、候选选择和加速器合并的联合设计才能发挥作用。

从本报告的角度看，Cayman 的意义在于把“架构接口”从指令级扩展到更完整的处理器-加速器协同边界。定制指令通常受寄存器操作数、指令编码和处理器流水线接口约束；定制加速器则更强调区域/内核级语义、数据搬运、接口带宽、控制流调度和多个调用点之间的硬件复用。Cayman 补足了领域定制链条中的第三个层次：APS/Aquas 解决“如何接入”，ISAMORE 解决“定制什么”，Cayman 则强调“在何种微架构和接口约束下生成可复用加速器”。这使本章从指令扩展走向更一般的领域定制架构能力，为第三章讨论高质量硬件实现奠定了过渡基础。

2.5 本章小结

本章围绕第一章提出的第一个科学问题，讨论了应用需求如何通过架构接口转化为可复用的指令、微架构和加速器能力。APS 和 Aquas 从端到端工具链角度说明，领域定制能力必须同时具有硬件接口、合成实现、编译映射、系统评估和性能-功耗-面积记录，才能从应用想法变成可迭代的应用专用处理器与定制指令扩展机制。ISAMORE 从可复用指令发现角度说明，定制能力应由程序语义、跨工作负载复用和硬件代价共同决定，局部热点只是候选来源之一。Cayman 则把这一逻辑推进到定制加速器生成，强调控制流、数据访问、处理器-加速器接口和加速器合并对领域定制能力选择的决定性作用。

这些工作共同表明，领域定制是应用语义、架构接口、微架构约束、编译支持和硬件实现之间的协同过程。一个定制能力是否值得进入系统，取决于它能否跨应用复用、能否被编译器稳定识别、能否通过接口高效传输数据、能否在硬件中以合理面积和时序实现，并能否在真实工作负载中获得端到端收益。因此，本章既回答了“定制什么”和“如何接入”的问题，也自然引出下一章。架构能力已经被识别并表达出来以后，还必须高质量地落到可综合、可验证、性能与资源可控的硬件实现中。

第三章 面向高质量硬件实现的前端抽象与综合优化

3.1 引言

第二章讨论了领域能力如何被识别、表达并接入架构接口，但架构能力本身并不等于高质量硬件实现。一个定制指令、加速器区域或硬件前端描述，只有在时序、面积、资源、接口协议、控制逻辑和验证条件都满足目标约束时，才能真正成为可部署的系统能力。因此，本章围绕第一章提出的第二个核心科学问题展开，讨论架构能力如何高质量落到可综合、可验证、性能与资源可控的硬件实现。

这一问题的难点在于硬件实现质量来自多个层次的决定。寄存器传输级设计能提供细粒度控制，但抽象层次低、生产率不足；高层综合能从软件描述生成硬件，却常把关键微架构决策隐藏在编译指示和启发式调度之后；硬件领域语言与中间表示试图提高表达能力和可分析性，但如果不能携带时序、资源和映射信息，仍难以形成可预测结果。因此，本章沿着基础设施视角、前端表达、微架构约束和联合优化逐层展开硬件实现问题。

HECTOR^[17]是本人较早参与的硬件综合基础设施工作，在本章中作为引言中的重要背景展开。它把硬件综合问题从单一高层综合工具实现推进到基于多层中间表示的方法学表达。高层中间表示将计算与带时序信息的控制图绑定，低层中间表示描述硬件模块和弹性互连，并可进一步转换为可综合寄存器传输级设计。其意义在于说明硬件综合可以被组织成多层中间表示、转换流程、调度信息和寄存器传输级降低的组合，而不必只能依赖黑盒高层综合或手写寄存器传输级代码。

这一早期积累为后续工作提供了两个判断。第一，硬件实现质量需要中间表示层支撑，只有把控制、时序、模块连接和降低边界显式化，后续方法才有可分析对象。第二，多层中间表示本身还不够，其中携带哪些时序、资源、接口和映射信息，决定了工具能优化到什么程度。基于这一判断，本章后文依次讨论 Cement、Clay 和 SkyEgg。Cement 关注周期确定型硬件行为表达，Clay 关注面向应用专用指令集处理器的微架构感知前端与综合机制，SkyEgg 则把代数变换、硬件映射和调度放入统一等价图设计空间，形成对高质量硬件实现问题的进一步推进。

3.2 周期确定型硬件前端

3.2.1 研究背景与设计目标

Cement^[118] 面向现场可编程门阵列加速器设计中的一个长期矛盾。设计者需要硬件级别的微架构表达和时序控制，但寄存器传输级设计生产率低、高层综合结果难预测、许多领域语言又只覆盖特定结构。硬件描述语言能直接描述连接和寄存器传输，却缺少对跨周期行为的显式表达，复杂有限状态机和流水线控制需要大量手工设计；高层综合从 C/C++ 等软件语言出发，依赖编译指示与启发式调度补足微架构信息，导致同一设计在不同工具版本或配置下可能产生不可预测结果；部分领域语言提升了安全性或特定并行模式表达，但往往牺牲一般微架构能力。Cement 因此提出周期确定型嵌入式硬件描述语言 CmtHDL 与 CmtC 编译器，目标是在不牺牲微架构表达力的前提下，提高硬件前端生产率和结果可预测性。

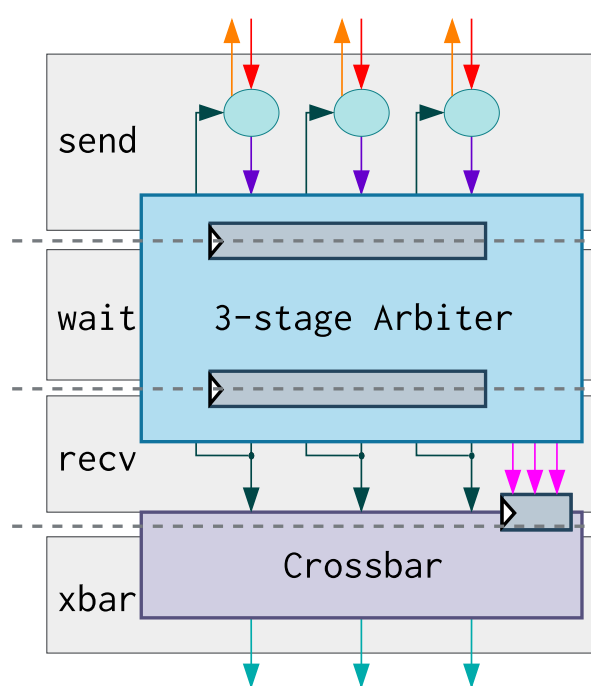
表 3.1 Cement 与代表性现场可编程门阵列编程框架的对比。该表对应源文 Table 1，重点说明 Cement 同时保留一般微架构表达、过程式控制描述、周期确定性和时序感知。

框架	设计入口	产物	控制逻辑描述	微架构表达能力	预设协议	周期确定	时序感知
Chisel	Scala eHDL	RTL	人工描述	一般表达	无	是	否
BSV + Stmt	HDL	RTL	人工 + 过程式	一般表达	ready-enable	否	否
Filament	HDL	Calyx	时间线类型	受限静态流水	无	是	是
HLS 工具	C/C++ 等	RTL	软件程序 + 指令	受工具指令限制	预设接口	否	是
Dahlia	DSL	HLS/ Calyx	软件程序 + 仿射类型	顺序控制器	固定接口	否	是
Spatial	DSL	RTL	软件程序 + 模板	受模板限制	固定接口	否	是
Aetherling	DSL	RTL	空间-时间类型	静态流式结构	固定接口	是	是
Calyx	IR	RTL	人工 + 过程式	受 go-done 协议限制	go-done	部分	部分
Cement	Rust eHDL	RTL	人工 + 过程式	一般表达	无	是	是

注：“过程式”表示控制逻辑可由 if/for 等过程式语句描述并自动生成；“周期确定”表示描述直接决定特定周期发生的硬件操作；“时序感知”表示综合或编译后可获得每周期操作的时序报告。

3.2.2 周期行为表达与控制子语言

CmtHDL 嵌入 Rust，先利用 Rust trait 和宏系统定义数据类型、数据束、接口、模块实例化、运算和端口连接，再引入事件层与控制子语言。事件层用事件表示一组带守卫条件的硬件行为，并显式记录这些行为发生的周期序列；控制子语言用 seq、par、if、for、while、step 等过程式语句描述事件的时序关系，使设计者能够直接表达确定的跨周期行为。与普通寄存器传输级设计只描述“每周期连接如何生效”不同，CmtHDL 允许设计者把“哪些行为在哪些周期发生”作为一等对象写入前端表示^[118]。



(a) 微架构

```
// Chisel code
class Shuffler extends Module {
  val io = IO(..)
  val arb = Module(..)
  val state = RegInit(0.U(4.W))
  val nextState = WireInit(state)
  state := nextState
  when (..) {
    nextState := ..
  } elsewhen // other situations
  val S1 = getStage(state, 1)
  when (S1) {
    // resolve resend
    // send to arbiter
  }
  // other stages
}
```

(b) Chisel 描述

```
// HLS code
void shuffler() {
  while (!loop_exit) {
    #pragma HLS pipeline II=1
    #pragma HLS dependence ..
    for (..) {
      #pragma HLS unroll
      // resolve resend
    }
    // call arbiter
    arbiter(..);
    for (..) {
      #pragma HLS unroll
      // xbar
    }
  }
}
```

(c) HLS 描述

```
// CMT-HDL code
shuffler(io) {
  let arbiter = instance!(..);
  let send = event! {
    // resolve resend
    // send to arbiter
  };
  let wait = event! {..};
  // other events: wait, xbar
  let pipeline = stmt! {
    // events as stages
    seq {
      send; wait; receive; xbar;
    }
  }
  synth!(pipeline, /*II=1*/);
}
```

(d) CmtHDL 描述

图 3.1 Cement 源文 Figure 1 的四级流水 shuffler 完整组图。(a) 给出微架构；(b)、(c)、(d) 分别给出 Chisel、HLS 和 CmtHDL 对同一结构的描述，体现 CmtHDL 用事件和过程式控制直接表达流水阶段。

这一抽象对复杂控制逻辑尤其重要。以 shuffler 和 arbiter pipeline 为例，硬件描述语言设计者需要手工保证发送、仲裁、接收和 resend 之间的周期对齐；高层综合版本则可能因为软件循环依赖模型和工具启发式调度而无法稳定达到 $II=1$ 。CmtHDL 可以用 seq statement 明确流水线不同阶段的时序，并由 CmtC 检查时序违规。这种周期确定性强调在静态或动态输入条件下保证事件的执行周期可推导、可检查、可综合，从而降低手工

有限状态机和流水线控制中的隐性错误；它关注的是可推导的周期语义，重点并不在固定延迟本身。

表 3.2 Cement 控制子语言语句。该表重排自源文 Table 3，将 `step`、`seq`、`par`、`if`、`for`、`while` 的定义与宏语法、时序语义、延迟和周期推导放在同一行，便于在单栏报告中阅读。

语句	定义与宏语法	时序语义	延迟与周期推导
<code>step</code>	<code>StepStmt</code> 包含事件集合、入口事件和出口事件；宏语法为 <code>[x] e0, e1 [y]</code> 。	等待入口事件 x 发生；在同一周期触发事件 e_0, e_1 ；随后等待出口事件 y 。	$L = 1 + \text{入口等待周期} + \text{出口等待周期}$ $sc[e_0] = sc[e_1] = sc[s] + \text{入口等待周期}$
<code>seq</code>	<code>SeqStmt</code> 包含语句序列；宏语法为 <code>seq { s0; s1 }</code> 。	无额外间隔顺序触发 s_0 与 s_1 。	$L = L[s_0] + L[s_1]$ $sc[s_0] = sc[s]$; $sc[s_1] = sc[s] + L[s_0]$
<code>par</code>	<code>ParStmt</code> 包含语句序列；宏语法为 <code>par { s0; s1 }</code> 。	立即并行触发 s_0 与 s_1 ，等待二者完成。	$L = \max(L[s_0], L[s_1])$ $sc[s_0] = sc[s]$; $sc[s_1] = sc[s]$
<code>if</code>	<code>IfStmt</code> 包含条件事件、then 分支和 else 分支；宏语法为 <code>if { [cond] t_stmt else e_stmt }</code> 。	根据条件事件 <code>cond</code> 是否发生，立即触发 then 或 else 分支。	$L = L[t_stmt]$ 或 $L[e_stmt]$ $sc[t_stmt] = sc[s]$ ，若 <code>cond</code> 发生
<code>for</code>	<code>ForStmt</code> 包含归纳变量、范围和循环体；宏语法为 <code>for { [indvar in range] do_stmt }</code> 。	按 <code>range</code> 无额外间隔重复执行循环体。	$L = L[do_stmt] \cdot \text{trip-count}$ $sc[do_stmt] = sc[s] + k \cdot L[do_stmt]$
<code>while</code>	<code>WhileStmt</code> 包含条件事件和循环体；宏语法为 <code>while { [cond] do_stmt }</code> 。	条件成立时无额外间隔重复执行循环体，直到条件失败。	$L = L[do_stmt] \cdot \text{trip-count}$ $sc[do_stmt] = sc[s] + k \cdot L[do_stmt]$

3.2.3 周期分析与有限状态机综合

CmtC 的核心编译能力包括周期分析和有限状态机综合。周期分析分为静态分析和动态监测。前者对展开阶段可确定的静态事件/语句推导周期序列，在仿真前发现数据收发周期不匹配；后者在仿真中插入监测信号和断言，用于捕获数据相关行为下的时序违规。有限状态机综合则从控制子语言生成严格满足时序规格的控制逻辑，并用状态树表示、转移表和编码优化减少触发器/查找表开销。由此，Cement^[118] 在前端语法之外，把语言、分析和综合贯通起来。设计者显式写出时序，编译器检查时序并生成资源优化的控制电路。

具体而言，静态周期分析首先识别周期序列可在展开阶段确定的静态事件/语句，然后自底向上推导语句延迟，再自顶向下推导每个事件的执行周期，并比较每个端口/连线的发送与接收周期。如果发送周期与接收周期不一致，系统可以在仿真前报告 `invalid-data` 或 `data-loss` 风险。动态监测则把事件是否在当前周期发生编码为布尔信号，为端口收发生成监测信号并插入 SystemVerilog 断言，从而处理数据相关控制流下的时序违规。有限状态机综合使用状态树表示表达控制子语言的状态空间：叶节点对应状态，边携带状态编码片段，转移表描述输入条件与下一状态；随后通过编码优化在 FF 数、LUT

数和解码延迟之间折中。这些机制使 CmtHDL 的“周期确定性”落到编译器可检查、可综合、可优化的具体对象上。

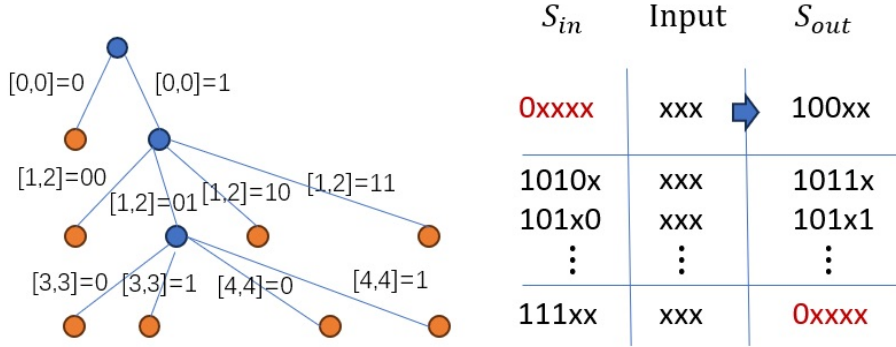


图 3.2 Cement 有限状态机综合中的状态树表示。控制子语言被转化为可编码、可优化的状态空间，避免由设计者手写控制器。

3.2.4 实验评估与意义

实验上，Cement^[118] 在 PolyBench、脉动阵列和稀疏加速器等案例中展示了周期确定前端的实际价值。相对 Vitis HLS，Cement 在未流水和流水配置下分别获得 1.41x 和 1.52x 几何平均加速，并平均节省 23%/47% 查找表和 68%/78% 触发器；相对 Dahlia-Calyx 流程，Cement 获得 3.49x 几何平均加速，并节省 54% 查找表和 82% 触发器。脉动阵列案例研究中，CmtHDL 通过显式周期对齐降低开发成本，并相对 EMS 节省 49% 查找表和 23% 数字信号处理器资源，频率提升 7%，单数字信号处理器吞吐率提升 33%。稀疏加速器案例研究中，CmtHDL 用更少代码描述 shuffler，并相对 Vitis HLS 方案提升频率、显著节省查找表和触发器。这些结果说明，硬件前端的价值不仅在于提高编写效率，也在于使时序行为可预测、可检查、可综合优化。

3.3 面向实现质量的前端抽象与综合机制

3.3.1 研究背景与设计目标

Clay^[119] 与 Cement 面向不同硬件实现问题，二者形成互补关系。Cement 关注现场可编程门阵列设计前端如何显式表达周期行为和控制逻辑；Clay 则回到应用专用指令集处理器和定制指令场景，关注一条指令扩展如何在不同处理器微架构、不同耦合策略和不同寄存器/访存路径下获得灵活、高质量实现。RISC-V 生态降低了指令扩展实验门槛，但现有应用专用指令集处理器框架往往把指令描述、耦合位置和微架构约束绑定在特定处理器模板中，使“能写出一条定制指令”和“能把它高质量接入不同微架构”之间仍存在明显距离。

这一距离首先体现在流水线内耦合的限制上。流水线内耦合能够复用主处理器已有控制和数据通路,适合组合逻辑型指令扩展,因此许多工具优先选择这种方式;但它也会把定制指令限制在处理器流水线已有资源和时序边界内。Clay^[119]源文指出,这类方法难以支持任意状态化行为和复杂控制流,循环往往需要常数循环次数或提前展开;同时,寄存器文件端口数、访存通道和执行阶段位置等约束也会限制一条指令中可表达的寄存器与内存交互。对于机器学习、信号处理和流式数据处理中的指令扩展,这些限制会直接影响设计者能否把多次访存、循环和私有状态封装成一个可复用架构能力。

Clay^[119]的研究目标正是把这些原本隐含在处理器模板中的接口事实显式化。它提出一个高层应用专用指令集处理器框架,以统一指令扩展接口连接 CADL 前端和微架构感知综合流程。设计者在 CADL 中描述指令编码、私有状态和无时序行为语义;工具再根据目标处理器提供的耦合策略、接口动作、阶段范围、延迟和资源限制,自动选择流水线内耦合或协处理器耦合,并生成对应硬件实现。这个设计把第二章讨论的“架构接口”进一步推进到实现层。同一个指令语义是否适合放进流水线、是否需要协处理器、每次寄存器/访存动作应该排在哪个阶段,都是影响指令扩展可移植性和性能的核心综合问题。

3.3.2 统一指令扩展接口与架构描述语言

Clay^[119]的关键抽象是统一指令扩展接口。源文把不同处理器耦合机制抽象成与微架构无关的接口原语,例如寄存器读请求/响应、寄存器写请求、内存读请求/响应、内存写请求和程序计数器更新;每个接口原语再由目标微架构给出延迟、可放置阶段范围和并发资源上限等属性。这样,流水线内耦合和协处理器耦合成为同一指令语义在不同接口属性下的不同实现候选。对综合器而言,接口原语描述“需要发生什么接口行为”,微架构属性描述“这些行为何时、何地、以多少资源发生才合法”。

CADL 则在这个接口之上提供面向设计者的行为描述语言。它包含指令编码、私有状态和行为三部分:编码描述指令格式和译码逻辑,私有状态可以是定制寄存器或暂存存储,行为部分用接口操作和高层控制流描述定制指令语义^[119]。重要的是,CADL 中的行为是无时序的,设计者不需要在前端手写请求-响应握手、流水线阶段或协处理器状态机;这些时序细节由后续综合流程根据目标微架构属性推导。与此同时,CADL 保留了 `if`、`for`、`while` 等控制流,循环边界可以来自解码变量或控制寄存器,因此能够表达传统流水线内定制指令工具难以支持的状态化行为和变长循环。

在编译流程中,CADL 先被转换为高层中间表示。高层中间表示包含指令识别/译码逻辑和对应行为的控制数据流图,随后经过条件转换、用户指定循环展开等变换,进入面向调度和硬件生成的后续表示^[119]。这个层次划分的意义在于,Clay 先保留高层语义,

再把目标处理器事实作为综合约束逐步引入，避免把“指令是什么”和“指令如何接入某个处理器”混在同一个描述中。由此，CADL 既能保持应用专用指令集处理器前端的可写性，又能为后端生成微架构相关实现留下足够信息。

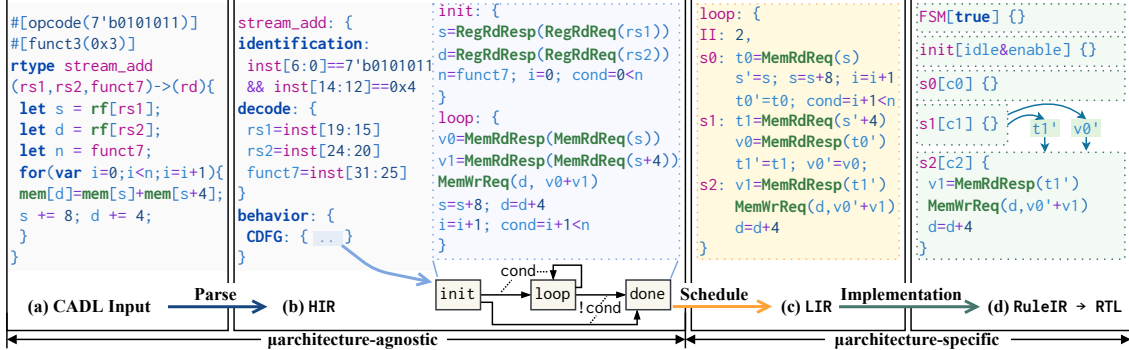


图 3.3 Clay 源文 Fig. 4 的代码转换示例。stream_add 的 CADL 输入经解析形成 HIR，再经过微架构感知综合降低为 LIR 和 RuleIR，体现从无序行为描述到可实现规则的多层表示链条。

3.3.3 微架构感知综合与调度

Clay^[119] 的综合流程以定制指令集合和候选耦合策略为输入。对于每条指令，系统按优先级尝试候选策略，通常先尝试开销较低的流水线内耦合，再尝试表达能力更强的协处理器耦合；每次尝试前先进行合法性检查，例如多基本块的状态化指令不能直接作为流水线内耦合实现，而需要程序计数器更新的指令也不能放入普通协处理器接口。通过这种先筛选、再调度、再生成的流程，Clay 把“选择哪类接口”从人工经验变成可由工具检查和搜索的综合步骤。

调度是 Clay^[119] 实现质量的核心。源文将微架构属性转化为整数线性规划中的约束：每个接口原语有延迟和可放置阶段范围，每类资源有并发上限，每个操作消耗相应资源；请求-响应式接口原语的依赖关系由请求延迟和响应动作共同约束。对于不含复杂控制流的指令，综合器进行普通调度；对于状态化、多基本块指令，综合器先对基本块调度，再生成有限状态机；对于协处理器中包含循环的指令，内层循环可以采用模调度，并逐步增大启动间隔直到找到合法调度。这样，Clay 可以在同一框架内覆盖简单组合指令、状态化控制流、复杂访存和协处理器长延迟行为。

Clay 源文 TABLE I 将调度目标和约束以整数线性规划形式给出。目标函数同时惩罚总体延迟 M 和结果生命周期 l_i :

$$\min \alpha M + \beta \sum_{i \in V} w_i l_i \quad (3.1)$$

依赖约束、资源约束、生命周期约束、阶段区间约束、唯一调度约束和总体延迟约束分别为:

$$s_j + p \cdot \text{dist}_{i,j} \cdot \Pi \geq s_i + L_i \quad (3.2)$$

$$\sum_{kt \in S} \sum_{i \in V} u_{i,r} x_{kt,i} \leq c_r \quad \text{若启用模调度} \quad (3.3)$$

$$\sum_{i \in V} u_{i,r} x_{s,i} \leq c_r \quad \text{若未启用模调度} \quad (3.4)$$

$$s_j + p \cdot \text{dist}_{i,j} \cdot \Pi - s_i - L_i \leq l_i \quad (3.5)$$

$$s_i \in R_i, \sum_{s \in S} x_{s,i} = 1, M \geq s_i + L_i \quad (3.6)$$

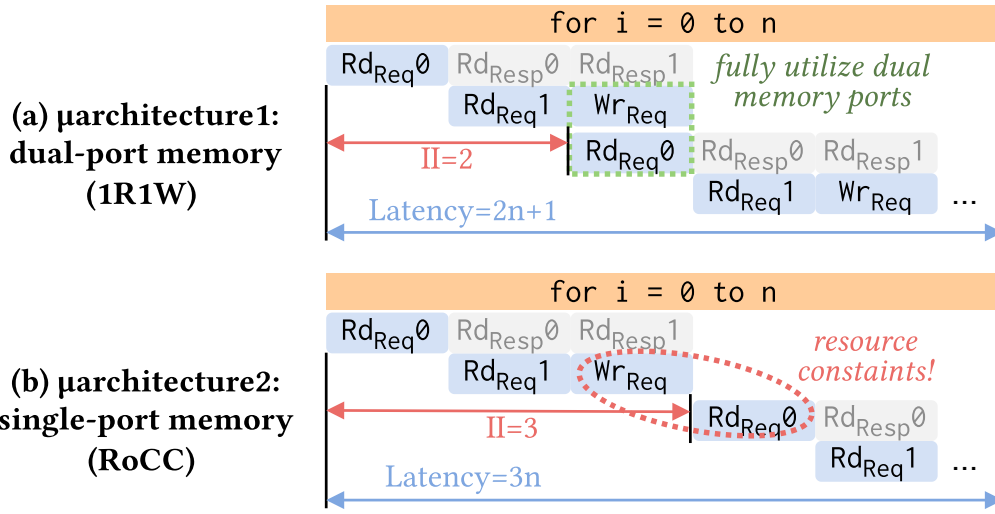


图 3.4 Clay 源文 Fig. 5 的 `stream_add` 微架构感知调度。双端口存储接口允许读写并行而得到更小启动间隔；单端口接口因读写共享资源而产生更长调度。

`stream_add` 示例清楚体现了这种微架构感知的必要性。该指令包含循环、连续内存读取和写回，传统流水线内定制指令工具难以直接支持。Clay^[119] 在双端口存储接口下可以得到启动间隔为 2、总延迟约为 $2n + 1$ 的调度；在单端口存储接口下，由于读写资源冲突，启动间隔增加到 3、总延迟约为 $3n$ 。同一 CADL 行为在不同微架构上产生不同调度结果，说明高层指令语义、接口资源和最终性能之间存在直接耦合。如果工具只把定制指令看作一段待综合逻辑，而不把寄存器/访存端口、流水线阶段和响应延迟显式纳入约束，就很难得到可移植且质量可控的实现。

3.3.4 硬件实现与接口生成

在硬件生成阶段，Clay^[119] 将调度后的 HIR 降低到 LIR 和 RuleIR。LIR 由已经排好周期的基本块组成，RuleIR 则进一步把每个周期或阶段中的行为组织成规则；当数据跨规则使用时，工具自动插入流水寄存器。对于包含多个基本块的状态化指令，Clay 根

据控制流生成有限状态机，用状态守卫条件控制规则触发，并用状态转换逻辑连接不同状态。这个过程把 CADL 中的高层控制流转化为明确的硬件状态和时序行为，使状态化定制指令不再依赖设计者手工编写协处理器控制器。

不同耦合策略在这一阶段被具体化。对于流水线内耦合，规则绑定到目标处理器的流水线阶段，并由阶段范围约束保证译码、寄存器读、执行、访存和写回等动作落在合法位置。对于协处理器耦合，Clay^[119]生成请求-响应接口、内部状态机和必要的通路；当响应延迟不可预测时，例如 RoCC 接口后接缓存层次结构，生成逻辑采用延迟不敏感的停顿机制等待数据返回。最终实现通过 cmt2 生成 SystemVerilog。这一流程使 Clay 既能为简单指令生成轻量级流水线内扩展，也能为复杂状态化指令生成协处理器式实现。

3.3.5 实验评估与意义

Clay^[119]的评估覆盖两类 RISC-V 处理器：Clay-core 是 RV32I、五级全旁路、无缓存、读写通道分离的处理器，支持流水线内耦合和协处理器耦合；Rocket-core 是 RV32IMAC 五级处理器，带缓存层次结构和 RoCC 协处理器接口，实验中主要使用协处理器耦合。评估的定制指令包括 `simd`、`complex_mul`、`sbox`、`autoinc`、`crc`、`cordic`、`gemm2x2` 和 `stream_add`。除查表型 `sbox` 外，多数 CADL 描述不超过 30 行，寄存器传输级代码生成时间低于 1 秒；这说明 Clay 的前端在保持描述成本较低的同时，把复杂接口和时序问题交给综合器系统处理。

表 3.3 Clay 源文 TABLE II 的定制指令集合。该表说明 CADL 同时覆盖组合数据流、私有状态、循环控制流和访存密集指令。

指令	行为描述	使用的 CADL 特性	行数
<code>simd</code>	向量加法等 SIMD 指令	位选择、拼接、寄存器访问	10
<code>complex_mul</code>	四个乘法器组成的复数乘法	算术运算、位选择、拼接、寄存器访问	11
<code>sbox</code>	AES 加解密的 S-box	常量查表、寄存器访问	262
<code>autoinc</code>	load/store 自动递增寻址	定制寄存器、访存、寄存器访问	7
<code>crc</code>	CRC 哈希算法	控制流、位运算、寄存器访问	10
<code>cordic</code>	CORDIC 定点三角函数	条件/循环控制流、有符号算术、移位、寄存器访问	26
<code>gemm2x2</code>	内存读写的 2x2 矩阵乘法	算术运算、访存、寄存器访问	21
<code>stream_add</code>	流式 pairwise reduction	循环控制流、访存、寄存器访问	11

单指令实验展示了不同机制的作用。简单组合指令可以通过流水线内耦合获得低开销实现，例如 `simd` 在 Clay-core 上周期数下降 95%，对应 20.0x 加速；`complex_mul` 在 Clay-core 上获得 203.2x 加速，但也带来更长时钟周期，这说明高性能定制算术逻辑需要同时考察周期数和时序代价^[119]。状态化指令则展示了 Clay 超出传统流水线内工具的能力：`cordic` 若完全展开会造成严重时序压力，而迭代式协处理器实现能够避免组合路径过长；`crc` 因位操作延迟较低，循环展开反而可能更合适；`stream_add` 的变长循环

和多次访存也需要借助协处理器与调度约束处理^[119]。这些结果说明，耦合策略不能由固定规则决定，而应由指令行为和目标微架构共同决定。

表 3.4 Clay 源文 TABLE III 的周期与加速结果重排。该表保留 ClayC 与 Rocket 两个处理器上的原始周期、优化周期和加速比，把源文宽表拆解为适合单栏报告阅读的性能部分。

指令	形态	ClayC			Rocket		
		原始周期	优化周期	加速比	原始周期	优化周期	加速比
<code>simd</code>		20	1	20.0x	18	6	2.8x
<code>complex_mul</code>		283	1	203.2x	38	4	9.3x
<code>sbox</code>		4	1	4.0x	8	6	1.3x
<code>autoinc</code>		72	14	5.1x	301	116	2.5x
<code>crc</code>	组合	73	1	72.9x	61	6	9.4x
<code>crc</code>	迭代	73	8	9.1x	61	24	2.5x
<code>cordic</code>	组合	125	1	30.9x	85	4	7.5x
<code>cordic</code>	多周期	125	9	13.6x	85	16	5.3x
<code>cordic</code>	迭代	125	9	13.7x	85	16	5.2x
<code>gemm2x2</code>		305	12	14.6x	61	31	1.5x
<code>stream_add</code>		71	18	3.9x	101	46	2.1x

表 3.5 Clay 源文 TABLE III 的实现代价与耦合策略重排。时钟周期和面积列表示相对基础处理器的变化；ip 表示流水线内耦合，cop 表示协处理器耦合。

指令	形态	ClayC			Rocket		
		时钟周期	面积	耦合	时钟周期	面积	耦合
Base		1.16 ns	20137um ²	/	1.68 ns	50941um ²	/
<code>simd</code>		-0.2%	+2.5%	ip	+5.7%	+10.6%	cop
<code>complex_mul</code>		+39.3%	+19.2%	ip	+2.5%	+23.7%	cop
<code>sbox</code>		-0.3%	+5.7%	ip	+2.5%	+7.7%	cop
<code>autoinc</code>		+0.4%	+0.3%	ip	+4.4%	+14.5%	cop
<code>crc</code>	组合	+0.2%	+2.5%	ip	+8.7%	+8.5%	cop
<code>crc</code>	迭代	+0.6%	+3.7%	cop	+3.2%	+6.3%	cop
<code>cordic</code>	组合	+305.0%	+39.8%	ip	+182.6%	+33.6%	cop
<code>cordic</code>	多周期	+1.8%	+62.6%	cop	+0.7%	+38.3%	cop
<code>cordic</code>	迭代	+1.7%	+16.8%	cop	+1.8%	+23.1%	cop
<code>gemm2x2</code>		+73.5%	+44.5%	cop	+32.8%	+45.8%	cop
<code>stream_add</code>		+0.5%	+12.1%	cop	+3.0%	+21.1%	cop

端到端工作负载进一步验证了 Clay^[119] 的实用性。EdgeDet 将 `rgba2gray`、`3x3 Sobel`、`3x3 erode` 和 `3x3 dilate` 等图像处理操作定制为指令，4 条定制指令共 87 行 CADL，优化后程序仅含 17 条指令，并在面积增加约 39% 且频率基本保持的情况下获得端到端收益。CFOEst 则定制 `complex_mean`、`complex_mul` 和 `cordic`，3 条定制指令共 50 行 CADL，将动态指令数从 1113 条降至 9 条，并获得 33.9x 加速；其面积和时钟周期增加主要来自乘法器、CORDIC 数据通路和定制状态。相较只报告单个内核加速，这些工作负载结果更能说明 Clay 面向的是可落地的应用专用指令集处理器设计流程。设计者用较少前端代码描述复杂指令行为，工具根据目标微架构生成不同实现，并通过周期、时序和面积共同评价结果。

从本报告角度看，Clay 在第三章中承担连接第二章与第三章的关键角色。第二章强调应用需求和可复用指令/微架构能力如何通过架构接口被发现和表达；Clay 进一步回

答,当一条能力已经成为定制指令后,如何让它在不同处理器微架构上以合适的耦合策略、时序安排和资源使用落到硬件实现。它也与 Cement 形成清晰分工。Cement 解决一般硬件前端如何表达周期确定行为,Clay 解决应用专用指令集处理器指令扩展如何把微架构属性纳入综合约束。二者共同说明,高质量硬件实现需要在前端抽象、接口原语、时序属性、资源约束和综合算法之间形成可检查的闭环,不能仅作为后端黑盒综合的附属结果。

3.4 变换、映射与调度联合优化

3.4.1 研究背景与设计目标

前面三个小节分别从基础设施、前端时序表达和微架构感知应用专用指令集处理器综合角度讨论硬件实现质量。SkyEgg^[120]则进一步指出,许多硬件综合质量问题来自代数变换、硬件映射和调度被人为拆成固定顺序的阶段,单个前端或调度器难以独立解决。传统高层综合通常先进行前端优化,再把操作调度到时钟周期,最后选择查找表、数字信号处理器或知识产权核等硬件实现;但调度需要知道硬件映射的精确延迟和资源行为,映射选择也依赖调度决定操作能否同周期串接,代数变换又可能改变可用映射模式。把这些阶段分开,会错过必须跨层联合选择才能发现的高质量实现。

SkyEgg^[120]的目标是把“表达式如何变换、操作如何映射、映射如何调度”放到同一个设计空间中处理。它以 MLIR 程序为输入,构造初始等价图;随后从目标现场可编程门阵列平台库生成映射重写规则,把软件操作和查找表、数字信号处理器、浮点知识产权核等硬件映射联系起来;等价饱和阶段同时应用一般代数重写规则和硬件映射规则,扩展出包含软件等价形式与硬件映射候选的饱和等价图;最后, SkyEgg 把传统的低成本表达式提取改写为“变换-映射感知”的调度问题,用精确整数线性规划或 ASAP 启发式求解,并生成 SystemVerilog 硬件实现。

3.4.2 等价图设计空间与硬件映射表示

SkyEgg^[120]首先把 MLIR 静态单赋值形式中的每个值转换为等价图节点,节点构造子来自对应 MLIR 操作,节点参数指向操作数所在的等价类。以源文中的 $-(a + b) \cdot c$ 为例,初始等价图只表达软件计算结构,并不包含任何硬件实现信息。为了让等价图同时承载硬件映射空间, SkyEgg 定义了一类专门的映射等价节点,形式为 $\text{id}(\text{Vec}(\text{arg}_1, \text{arg}_2, \dots))$ 。其中 id 表示一个具体硬件映射,例如 DSP48E2 或查找表; Vec 中的参数表示该硬件映射的输入。

硬件映射通过映射规则插入等价图。每条映射规则由三部分组成。匹配器描述硬件单元能够实现的软件代数模式;应用器在匹配位置插入相应映射等价节点;条件约束数

据类型、位宽或硬件原语限制^[120]。以 DSP48E2 为例，它的输入 A、D、B 分别受到不同比特宽度限制，因此同一个代数表达式能否映射到数字信号处理器，不仅取决于操作种类，还取决于数据类型和位宽。SkyEgg 把这些约束作为映射规则条件显式放入等价图扩展过程，使硬件原语的功能、配置和限制不再隐藏在后端绑定阶段。

SkyEgg^[120] 还区分三类硬件映射配置。第一类是基本逻辑，即用 Verilog 操作符描述、没有配置变体的查找表逻辑；第二类是硬件原语，例如 DSP48E2，其内部流水寄存器可以独立打开或旁路；第三类是参数化知识产权核，例如浮点知识产权核，其整体延迟和资源预配置可以配置。对于每个硬件映射，SkyEgg 会枚举可用配置，并把配置参数编码进映射等价节点的 id 中。这样，后续调度面对的是具有确定延迟、时序属性和资源行为的映射-配置对，已经具备比抽象“乘法”或“数字信号处理器”更具体的硬件含义。

3.4.3 时序模型与调度式提取

为了让映射选择真正影响调度，SkyEgg^[120] 为每个目标映射建立时序属性模型。一个映射包含延迟、输入延迟、输出延迟和寄存器到寄存器延迟等属性。对于组合映射，输入延迟描述输入端口到输出端口的组合路径；对于含流水寄存器的映射，输入延迟描述输入到第一级寄存器的路径，输出延迟描述末级寄存器到输出的路径，寄存器到寄存器延迟则约束内部流水线可达到的最小时钟周期。由于现场可编程门阵列原语和知识产权核的供应商时序数据通常不公开，SkyEgg 预先用综合工具实例化每个映射及配置，施加时序约束并提取综合报告，从而建立时序属性数据库。该数据库不追求替代最终布局布线结果，但足以指导调度阶段的映射选择和同周期串接判断。

在等价图上进行调度还需要判断映射之间的结构路径延迟。若从一个映射到另一个映射之间只经过组合映射，则这条路径可能在同一时钟周期内串接；若路径延迟超过目标时钟周期，则必须插入流水寄存器切断组合路径。SkyEgg^[120] 源文将路径延迟定义为源映射输出延迟、中间映射输入延迟、目标映射输入延迟和连线延迟的累积，并用所需切分寄存器数量把时序闭合要求转化为调度约束。这个模型把“能否同周期串接”从高层综合后端的隐式启发式，变成了等价图提取阶段可显式推理的约束。

在问题形式化上，SkyEgg 将传统等价图提取转化为变换-映射感知调度的整数线性规划。源文 Table 1 首先定义了该问题中的核心符号，包括饱和等价图中的映射等价节点集合、等价类集合、根等价类、等价节点的子等价类、等价类和等价节点的完成时间/开始时间，以及二者是否被选中的布尔变量。这张表的意义在于把“选择哪个表达式”和“何时执行哪个硬件映射”放入同一组变量，避免先提取表达式、再交给调度器处理。

表 3.6 SkyEgg 源文 Table 1 的问题建模符号。该表把等价图提取对象和调度对象统一到同一个优化问题中。

符号	含义
$\{m_i\}$	饱和等价图中的映射等价节点集合
$\{cls_j\}$	饱和等价图中的等价类集合
cls_{root}	表示整个程序输出的根等价类
$m_k \in cls_j$	等价类 cls_j 包含映射等价节点 m_k
$child(m_i)$	映射等价节点 m_i 的子等价类集合
$\frac{f_{cls,j}}{b_{cls,j}}$	等价类的完成时间 / 是否被选中
$\frac{f_{m,i}}{b_{m,i}}$	映射等价节点的开始时间 / 完成时间 / 是否被选中

基于这些符号, SkyEgg^[120] 的目标函数最小化根等价类完成时间, 并用较小惩罚项减少不必要的映射选择。约束分为两类。完整性约束保证根等价类被计算、被选中的等价类至少选择一个映射等价节点、被选中的映射等价节点的子等价类也被计算; 调度约束保证依赖关系、延迟关系、等价类完成时间和时序串接约束成立。其核心建模可概括如下。

表 3.7 SkyEgg 的变换-映射感知调度建模。该问题同时决定表达式形态、硬件映射选择和时钟周期安排。

目标函数同时最小化根等价类完成时间, 并用小系数 α 抑制不必要的映射选择:

$$\min_{f,b,s} f_{cls,root} + \alpha \sum_{i=1}^N b_{m,i} \quad (3.7)$$

完整性约束保证最终表达式语义完整: 根等价类必须被计算; 被选择的等价类至少选择一个内部映射等价节点; 被选择的映射节点必须递归计算其所有子等价类:

$$b_{cls,root} = 1 \quad (3.8)$$

$$\forall j, b_{cls,j} \leq \sum_{m_i \in cls_j} b_{m,i} \quad (3.9)$$

$$\forall i, \forall cls_j \in child_{m_i}, b_{m,i} \leq b_{cls,j} \quad (3.10)$$

调度约束把表达式选择和时钟周期安排绑定在一起。映射节点只有在其子等价类完成后才能开始, 完成时间由开始时间和映射延迟 $L_{m,i}$ 决定; 若映射节点被选择, 则其所在等价类的完成时间不得早于该节点完成时间:

$$\forall i, \forall cls_j \in child_{m_i}, s_{m,i} \geq f_{cls,j} \quad (3.11)$$

$$\forall i, f_{m,i} = s_{m,i} + L_{m,i} \quad (3.12)$$

$$\forall j, \forall m_i \in cls_j, f_{cls,j} \geq f_{m,i} - M(1 - b_{m,i}) \quad (3.13)$$

串接约束把时序闭合转化为所需流水寄存器数。对从 m_i 到 m_j 的结构路径 π , 若路径上的映射节点均被选择, 则 m_j 的开始时间至少要晚于 m_i 的完成时间加上切分寄存器数:

$$\forall \pi \in \Pi(m_i, m_j), s_{m,j} \geq f_{m,i} + cuts(t_{path(\pi)}) - M \sum_{m \in \pi} (1 - b_m) \quad (3.14)$$

$$t_{path(\pi)} + q(t_{su} + t_{cq} + t_{net}) \leq (q + 1)T_{clk} \quad (3.15)$$

$$cuts(t_{path(\pi)}) = \left\lceil \frac{t_{path(\pi)} - T_{clk}}{T_{clk} - (t_{su} + t_{cq} + t_{net})} \right\rceil \quad (3.16)$$

3.4.4 高效求解与实验评估

SkyEgg^[120] 的求解难点主要来自串接约束，因为饱和等价图中许多映射等价节点对之间存在大量可行路径。若只保留最大路径延迟，最大路径上的中间映射等价节点可能最终没有被选中，实际被选路径反而没有得到约束；若保留全部路径，整数线性规划规模又难以扩展。SkyEgg 因此提出前 k 条路径延迟，在每对映射等价节点之间保留若干最长路径作为约束，默认 $k = 3$ 即可使实验设计满足目标时钟周期。这个选择反映了 SkyEgg 的工程取舍，既不能把时序闭合简化为单一路径，也不能让完整路径集合压垮求解器。

为了进一步扩展问题规模，SkyEgg^[120] 还提出 ASAP 启发式调度器。它按拓扑顺序遍历等价类，为每个等价类选择最早完成的映射等价节点，并在确定开始时间时同时考虑依赖约束和以该节点为终点的前 k 条串接约束。该调度器属于启发式方法，不保证全局最优，因为当前等价类中“最早完成”的映射可能给后续等价类带来较长组合路径；但源文实验显示，它在多数情况下接近整数线性规划解，同时把求解时间降低到适合大规模任务的范围。

算法 3.2 ASAP Scheduler

```

1: function ASAP-Schedule( $G, P_k$ )
2:   for  $c_j \in \text{topo-order}(G.\text{eclases})$  do
3:     for  $m_i \in c_j$  do
4:        $s(m_i) \leftarrow \max_{c \in \text{child}(m_i)} f(c)$ 
5:       for  $\pi \in P_{k(m_i)}$  do
6:         if active( $\pi$ ) then
7:            $s(m_i) \leftarrow \max(s(m_i), f(\text{src}(\pi)) + \text{cuts}(\text{delay}(\pi)))$ 
8:         end
9:       end
10:       $f(m_i) \leftarrow s(m_i) + L(m_i)$ 
11:    end
12:     $m^* \leftarrow \arg \min_{m_i \in c_j} f(m_i)$ 
13:     $b(m^*) \leftarrow 1$ 
14:     $f(c_j) \leftarrow f(m^*)$ 
15:  end
16:  return  $f(c_{\text{root}})$ 
17: end
    
```

实验上，SkyEgg^[120] 使用 HECTOR 将输入程序降低到 MLIR 并提取基本块，再将其转为 C++ 内核，基线为 Vitis HLS 2024.1，目标平台为 Xilinx xcku3p 现场可编程门阵列；SkyEgg 的 Verilog 输出使用 Vivado 2024.1 综合，整数线性规划求解使用 OR-Tools 并设置 1 小时超时。映射库覆盖整数和浮点操作。整数侧包含查找表逻辑、DSP48E2 和除法知识产权核，浮点侧使用 Xilinx 浮点知识产权核并考虑不同流水配置。基准包含 liquid-

dsp 数学内核、ggml 神经网络内核、PolyBench 科学计算内核、Vitis Library 信号处理内核，以及用于可扩展性分析的 100 到 600 操作合成基准。

结果显示，SkyEgg^[120] 的变换-映射感知调度相对 Vitis HLS 在全部基准和频率目标上平均获得 3.10x 加速，其中整数线性规划平均 3.12x，ASAP 平均 3.08x；浮点基准平均 3.64x、最高 5.22x，整数基准平均 2.38x、最高 4.33x。源文中的分组分析比单张性能图更能说明机制来源。在 `rmsnorm` 中，平方根映射的 10–13 周期配置都能达到 431 MHz，SkyEgg 选择满足 400 MHz 约束的 10 周期配置，而 Vitis HLS 使用保守的 28 周期映射，使整体延迟从 73 周期降至 25 周期；在 `randnf_pdf` 中，Vivado 浮点指数知识产权核的全数字信号处理器配置为 30 周期、中等数字信号处理器配置可在相同频率下达到 20 周期，SkyEgg 进一步选择 8 周期中等数字信号处理器配置并达到 450 MHz，使整体延迟从 154 周期降至 52 周期。这些数据直接支持前文的判断。性能收益来自等价变换、映射配置和时序调度共同进入求解空间，不能归结为单一后端优化。

时序和资源结果同样以结论数据呈现即可。SkyEgg^[120] 的整数线性规划和 ASAP 设计在所有频率目标下均满足时序约束，而 Vitis HLS 在高频目标下有 48% 的设计无法满足时序；资源方面，整数线性规划/ASAP 的触发器使用量分别为 Vitis HLS 的 0.87x/0.95x，查找表使用量分别为 1.51x/1.28x。可扩展性实验中，ASAP 在 100 到 600 操作的整数和浮点合成基准上均能在 1 秒内完成，而整数线性规划在大规模浮点问题上频繁超时；这说明 SkyEgg 同时提供了精确优化路径和可扩展工程路径。

表 3.8 SkyEgg 源文 Table 3 的资源利用率结果重排。FF/LUT 表示 SkyEgg 相对 Vitis HLS 的资源使用比例；I/A 分别表示 ILP/ASAP；绿色表示不高于 Vitis，红色表示高于 Vitis。

Benchmark	FF _I	FF _A	LUT _I	LUT _A
bairstow	0.79	0.90	0.88	0.91
invgauss	0.67	0.92	1.52	1.57
landenf	1.46	1.52	2.29	2.30
lngammaf	0.85	0.93	1.44	1.45
randnf_pdf	0.97	1.11	1.89	1.80
gelu	0.84	0.87	1.07	1.04
geglu	0.85	0.85	1.03	1.03
silu	1.21	1.21	1.33	1.33
swiglu	1.00	1.00	1.25	1.25
softmax	1.16	1.16	0.81	0.81
layernorm	1.14	1.35	2.05	2.11
rmsnorm	2.06	2.06	5.11	5.11
durbin	0.80	0.81	0.83	0.85
jacobi-1d	0.85	0.85	0.83	0.83
jacobi-2d	0.86	0.86	0.82	0.82
bicg16	0.30	0.31	0.85	0.79
gemm16	0.34	0.84	0.00	0.00
gemver16	0.51	0.76	4.26	0.64
lerp	0.66	0.68	0.79	0.79
webp_enc	0.26	0.21	1.82	0.67
rope	0.72	0.72	0.81	0.78
Mean	0.87	0.95	1.51	1.28

表 3.9 SkyEgg 源文 Table 3 的时序闭合结果重排。源文逐基准给出 T_I 、 T_A 、 T_V ；这里按方法汇总，以突出 SkyEgg ILP/ASAP 全部满足时序而 Vitis HLS 在高频目标下失败较多这一结论。

方法	满足时序比例	说明
SkyEgg-ILP	21/21	所有基准均满足目标时序。
SkyEgg-ASAP	21/21	所有基准均满足目标时序，同时求解时间更适合大规模任务。
Vitis HLS	11/21	invgauss、landenfn、lngammaf、randnf_pdf、gelu、geglu、silu、swiglu、layernorm、rmsnorm 未满足时序。

3.4.5 本节小结：从前端表达到联合优化

SkyEgg 是本章的方法高潮，因为它把前三节强调的几个判断合到同一个优化框架中。HECTOR 说明硬件综合需要中间表示化和多层方法学支撑；Cement 说明时序行为必须显式表达并可检查；Clay 说明微架构属性和耦合策略必须进入综合约束；SkyEgg 则说明一旦这些事实进入中间表示和设计空间，工具还需要避免固定编译遍顺序和阶段割裂，把变换、映射与调度作为联合问题求解。由此，本章从“如何表达硬件”推进到“如何在语义等价空间和硬件映射空间中选择高质量实现”。

这一点也自然引出第四章。SkyEgg 仍主要依赖预定义重写规则、硬件映射候选、整数线性规划/启发式调度器和工程化代价形式化；随着设计空间变大，哪些规则应当被应用、哪些策略能在不同任务上复用、如何把专家经验和历史证明转化为可检查策略，成为新的问题。第四章中的 EggMind 正是在 SkyEgg/ISAMORE 所代表的等价图优化基础上，把大语言模型的策略生成能力纳入形式化可控的等价饱和策略表示中，进一步讨论大语言模型与形式化技术如何共同服务软硬件协同。

3.5 本章小结

本章回应第一章提出的第二个科学问题，即架构能力如何高质量落到可综合、可验证、性能与资源可控的硬件实现。HECTOR 提供了基于 MLIR 多层中间表示的早期硬件综合基础设施视角，说明硬件实现质量需要结构化中间表示和降低流程支撑。Cement 强调前端必须显式表达周期行为，并通过周期分析与有限状态机综合保证硬件行为可预测、可检查、可综合优化。Clay 将这一问题带回应用专用指令集处理器/定制指令扩展场景，强调耦合策略、微架构属性和综合约束对定制指令实现质量的决定性作用。SkyEgg 则进一步把代数变换、操作映射和调度统一到等价图设计空间中，避免固定阶段顺序造成的次优实现。

这些工作共同说明，高质量硬件实现需要从前端表达、中间表示设计、时序语义、微架构属性、映射选择和调度求解共同组织，不能被视为后端工具的附属结果。它们也为下一章奠定了方法基础。等价图、MLIR、硬件中间表示和综合结果成为软硬件协同

的核心对象后,大语言模型和智能体方法若要进入这一流程,就必须操作这些结构化对象,并受到形式化语义、验证检查和执行记录约束。

第四章 大语言模型与形式化技术驱动的软硬件协同方法

4.1 引言

前两章分别讨论了领域能力如何进入架构接口，以及这些能力如何落到高质量硬件实现。本章进一步讨论一个新的方法层问题，即大语言模型和智能体方法进入代码生成、硬件生成、编译优化和工程自动化流程后，如何可靠服务软硬件协同。第一章已经指出，大语言模型与智能体方法是一条重要技术趋势，但芯片设计与编译优化对语义正确性、接口协议、时序、资源、验证和长期维护提出了远高于一般代码补全的要求。因此，本章重点说明大语言模型能力必须通过形式化对象、编译基础设施、工具检查结果和经验记录进入协同流程。

OriGen^[121]是这一问题的重要背景。它面向开放寄存器传输级代码生成，指出商业大语言模型在软件代码生成上已经表现出较强能力，也能生成一定质量的寄存器传输级代码，但闭源模型带来隐私、安全、可定制性和研究可复现问题；开源代码模型虽然便于部署和微调，却受到高质量寄存器传输级数据不足的限制。硬件设计中的寄存器传输级代码往往包含知识产权核和专有设计知识，不能简单依赖商业模型远程生成；同时，寄存器传输级语法、时序、综合风格和验证要求又明显不同于普通软件代码，通用代码数据难以充分覆盖。

OriGen 的启示在于说明大语言模型硬件生成必须进入工具约束流程。寄存器传输级生成在本章中作为背景讨论，不作为重点展开。它通过代码到代码数据增强提升寄存器传输级数据质量，并构建 VerilogFixEval 评测模型基于编译错误信息修复寄存器传输级代码的能力。训练数据中包含自然语言指令、错误代码、编译错误信息和修正代码，使开源模型学习从工具检查结果到代码修复的映射。对本研究而言，这表明硬件生成中的大语言模型能力必须与编译、语法检查、仿真和修复样例结合，后续面向验证约束的架构、硬件与编译协同生成也需要进一步把这种约束扩展到接口协议、等价性检查、综合结果和跨层执行轨迹。

本章以 EggMind 为重点，讨论大语言模型能力如何通过 EqSatL、等价图、证明派生 motif 记忆和可处理性引导进入形式化优化框架。OriGen 在引言中提供大语言模型硬件生成侧的背景，EggMind 则作为本章完整展开的现有工作，展示大语言模型与形式化技术结合后如何把策略搜索、经验组织和智能体化迭代纳入可检查的工具链产物。

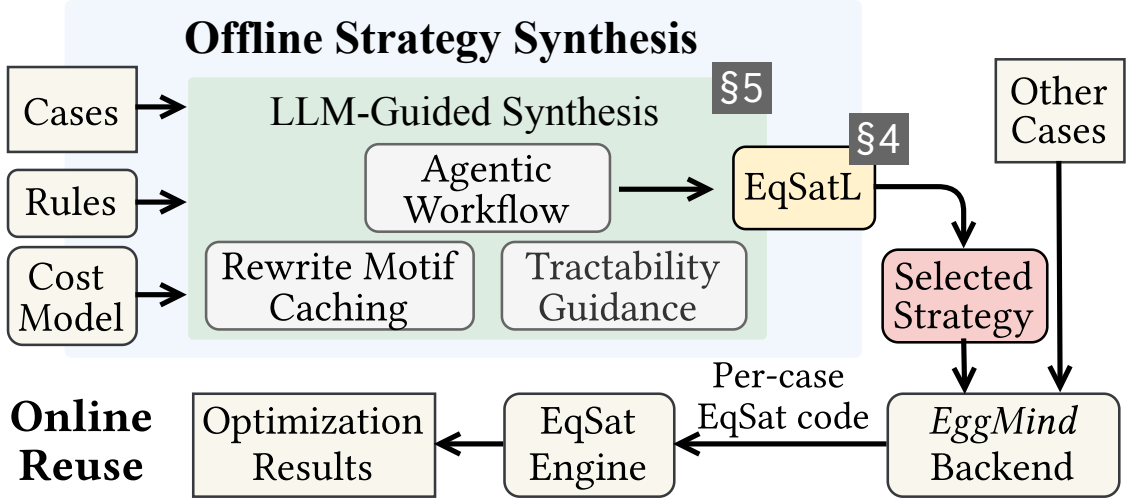


图 4.1 EggMind 总体流程。离线阶段通过智能体流程合成 EqSatL 策略，在线阶段由后端将策略降低为可执行等价饱和代码并复用到新优化任务。

4.2 等价图策略自动化

4.2.1 研究背景与设计目标

EggMind 直接回应了第一章和第三章共同提出的编译接口问题。等价图、等价饱和和基于重写的优化已经成为硬件综合、编译优化和指令集架构定制的重要方法，下一步需要解决这些优化过程中的策略设计如何进一步自动化。等价饱和的优势在于用等价图紧凑表示大量语义等价程序，避免过早承诺某条重写序列；ISAMORE 和 SkyEgg 分别利用这一思想发现可复用定制指令、联合探索硬件变换/映射/调度空间。但等价饱和的实用瓶颈同样明显。重写空间一旦变大，等价图会快速膨胀，执行时间和内存成为主要限制，最终结果很大程度上取决于规则如何分组、何时应用、应用多少轮、何时简化。

因此，EggMind^[122] 把自动化目标从“让大语言模型直接写 egglog 代码”转向“合成可复用等价饱和策略产物”。在 EggMind 中，策略指组织等价饱和搜索的控制结构，包括规则集划分、饱和调度、等价图简化和预算控制。这个策略表示位于重写规则和后端执行之间。它不改变语义规则本身，但决定哪些规则可以互相作用，怎样分阶段暴露优化机会，以及如何在搜索过程中收缩冗余状态。这样，大语言模型围绕一个可检查、可复用、可评估的策略表示进行搜索，避免直接操作庞大的原始等价图或低层 egglog 细节。

4.2.2 可检查的策略表示

EggMind^[122] 的第一个核心抽象是 EqSatL。EqSatL 是表示等价饱和策略的领域专用语言，目标是在紧凑性、形式化验证和表达能力之间取得平衡。它把策略设计分解为

三个控制面。第一是规则集划分，用语义标签将重写词表分为具有含义的规则集，避免向大语言模型暴露冗长规则列表，使模型能在更高层级表达“哪些规则应当一起或分开作用”。第二是调度构造，用流树组织阶段、序列和 `repeat` 区域，使规则交互可以被分阶段、可重复但有界地执行。第三是简化控制，把 `simplify` 显式化为一等控制节点，通过强度参数和可选大语言模型提示决定何时、如何收缩等价图，避免无约束增长。

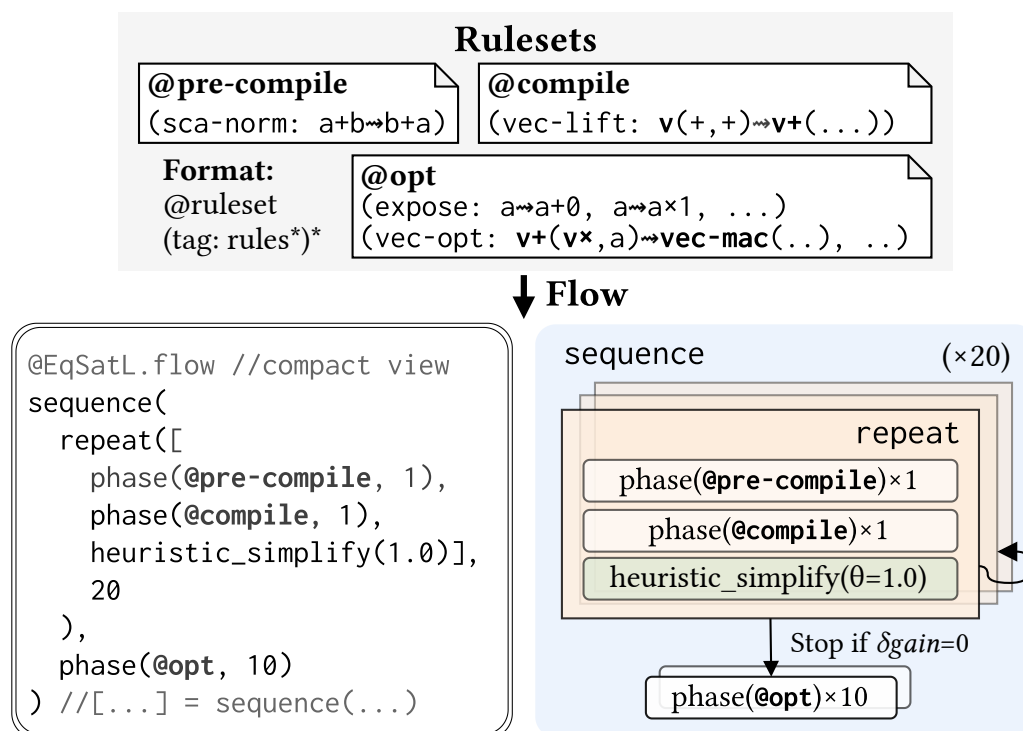


图 4.2 EqSatL 策略示例。策略表示将规则集划分、阶段化调度、repeat 结构和 simplify 控制显式化，便于大语言模型生成、工具验证和后端执行。

这一设计的关键在于，EqSatL 将策略意图与后端实现解耦。传统原始 `egglog` 脚本把策略、规则应用细节和执行机制混在一起，大语言模型需要反复修补低层代码，工具检查结果也难以定位责任边界。EqSatL 则把策略产物放在稳定边界上。它足够紧凑，便于大语言模型生成和修改；它足够显式，便于检查划分、调度和简化是否合理；它也足够有表达力，能控制规则互动、重复搜索和状态收缩。对软硬件协同而言，这种产物边界非常重要，因为受约束自动化必须操作稳定对象，不能依赖一段难以检查的低层脚本。

4.2.3 智能体流程：离线策略合成与在线复用

EggMind^[122] 的第二个核心是智能体流程。它把离线策略合成组织为 Strategist、Generator、Evaluator、Partitioner 和 Simplifier 的受控循环。流程记忆中维护策略仓库，包括当前最佳策略、有前景策略和待评估策略；每轮迭代中，Strategist 根据当前评估记录选择动作，Generator 产生或修改 EqSatL 策略，Evaluator 以不同预算执行等价饱和并返回质量、时间和内存结果，Partitioner 与 Simplifier 则提供规则集划分和简化提示。评

估后，系统更新最佳/有前景/待评估状态，并把成功运行中的记录写回记忆。这样，智能体流程依赖明确的策略表示、评估结果和可复用记录，不依赖开放式对话历史。

从系统结构看，EggMind 将策略发现分为离线合成和在线复用两个阶段。离线阶段以一组演化案例、重写规则和代价模型为输入，由智能体流程生成 EqSatL 策略产物；在线阶段则把选定策略应用到新案例，由后端将 EqSatL 降低为可执行等价饱和代码并运行。这个分离具有重要工程意义。昂贵的策略探索成本在离线阶段支付，并通过可复用策略摊销到后续优化任务；在线优化阶段不需要让大语言模型反复进入执行路径，从而降低不可控性和运行开销。对博士开题报告而言，这一点说明 EggMind 是一种把策略搜索、执行评价和经验记录分层组织的编译自动化系统，区别于一般意义的大语言模型调参。

4.2.4 证明驱动的经验记忆

第三个核心是证明派生重写 motif 缓存。等价饱和运行若产生更优结果，后端可以给出初始表达式到最终表达式的等价证明；但原始证明树通常庞大、包含大量同余/传递性记录，不适合作为大语言模型上下文。EggMind^[122] 因此从证明树中提取局部重写 motif，将具体规则链提升为语义标签链，例如某类向量提升后接某类向量优化。motif 缓存根据相对代价降低和跨案例频率保留高价值 motif，并在后续策略生成中提供紧凑结构记录。这样，系统记忆的是来自成功证明路径的可复用重写交互，区别于模糊经验描述。

这一机制解决了智能体编译优化中的一个关键问题，即经验如何避免变成难以检查的自然语言偏好。EggMind 首先定位证明树中的局部证明区域，用同余路径标记子表达式上下文；随后把局部重写链通过规则到标签映射提升为语义标签链，并对重复 motif 进行去重和排序。motif 的评分同时考虑相对收益和跨案例频率，因此缓存保留的是在成功优化中反复出现、且能带来实际代价降低的结构记录。后续 Generator 或 Strategist 使用这些 motif 时，看到的是来自等价证明的结构化引导，未经验证的人工经验不会直接进入策略生成依据。

EggMind 的证明派生记忆用相对收益更新 motif 评分。对候选案例 c 和 motif γ ，源文中的核心更新可写为：

$$g(c) = \max\left(0, 1 - \frac{\text{final-cost}(c)}{\text{previous-best-cost}(c)}\right) \quad (4.1)$$

$$\text{score}(\gamma) \leftarrow (1 - \alpha) \text{score}(\gamma) + \alpha g(c) \quad (4.2)$$

$$\text{best-gain}(\gamma) \leftarrow \max(\text{best-gain}(\gamma), g(c)) \quad (4.3)$$

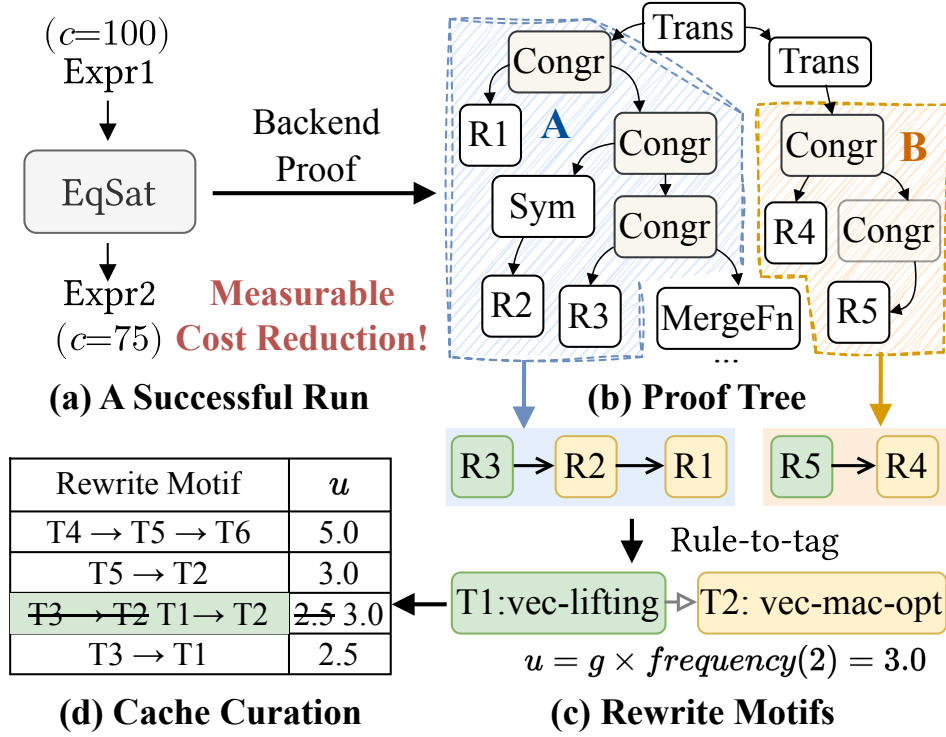


图 4.3 EggMind 证明派生 motif 记忆。系统从等价证明中抽取局部重写 motif，并将其转化为可复用、可检查的策略记录。

4.2.5 可处理性引导与简化提示

第四个核心是可处理性引导。EggMind^[122]用基于依赖的规则集风险模型为规则集划分提供稳定性指导。它把重写词表建模为规则依赖图，根据右侧/左侧模式激活、算子重叠和子树可匹配性估计规则之间的互相激活风险；划分目标奖励前向阶段依赖，惩罚回流、首阶段流入和同阶段纠缠，从而使规则交互更接近稳定的前向激活。与此同时，大语言模型引导简化提示为不同阶段提供偏好/剪除模式对，将简化从单一全局代价剪枝变成阶段局部结构偏好。二者共同服务一个目标，即让大语言模型搜索停留在稳定、可执行、可复用的策略空间中，避免不断触发等价图膨胀。

具体地，基于依赖的规则集风险模型将重写规则作为图节点，用有向边表示一条规则可能启发另一条规则的强度；阶段分配的目标是鼓励规则依赖从前一阶段流向后一阶段，同时抑制后向激活、首阶段反复被后续规则激活，以及同阶段规则强耦合。大语言模型引导简化提示则在每个阶段给出少量偏好/剪除模式对，形成局部简化偏序，并通过有界惩罚改变等价类内候选的保留优先级。这种做法将大语言模型提示作为可检验、可修正的软约束。评价过程继续检验提示是否过强或损害后续优化；如果提示不合适，后续迭代可以修正。因此，EggMind 对大语言模型的使用始终围绕可执行评价和可回退控制，模型不会直接决定最终正确性。

规则集风险模型把重写词表表示为有向依赖图 $G = (R, E, w)$ ，并为阶段分配 $\varphi: R \rightarrow \{1, \dots, k\}$ 求解如下目标：

$$\max_{\varphi} \alpha \sum_{(i,j) \in E} w_{i,j} 1[\varphi(i) + 1 = \varphi(j)] - \beta \sum_{(i,j) \in E} w_{i,j} 1[\varphi(i) > \varphi(j)] \quad (4.4)$$

$$-\gamma \sum_{(i,j) \in E} w_{i,j} 1[\varphi(j) = 1, \varphi(i) \neq 1] - \delta \sum_{(i,j) \in E} w_{i,j} 1[\varphi(i) = \varphi(j)] \quad (4.5)$$

四项分别奖励前向依赖，并惩罚回流、首阶段流入和同阶段纠缠。对大语言模型引导简化，阶段 t 中的偏好/剪除模式对定义局部偏序：

$$(p^+, p^-) \in \Pi_t \times \Pi_t, \quad p^- \prec_t p^+ \quad (4.6)$$

简化评分以基础代价加有界提示惩罚表示：

$$s(e) = c(e) + p_{\text{lim}}(e) \quad (4.7)$$

其中 θ 控制按评分剪除的强度。

4.2.6 实验评估与对软硬件协同的意义

EggMind^[122] 的评估说明，“大语言模型、形式化策略表示和验证约束体系”的组合确实改善了等价饱和和优化。向量化基准测试上，EggMind 相对完整等价饱和将最终代价几何平均降低 45.1%，相对专家调优 Isaria 策略降低 20.6%；在线执行相对 Isaria 获得 2.21x 几何平均加速，并相对完整等价饱和将峰值内存几何平均降低 69.1%。消融实验显示，motif 缓存、repeat 结构、Strategist 和 Partitioner 均对质量或稳定性有贡献，尤其缺少 motif 缓存或 repeat 会显著恶化代价、执行时间和内存。

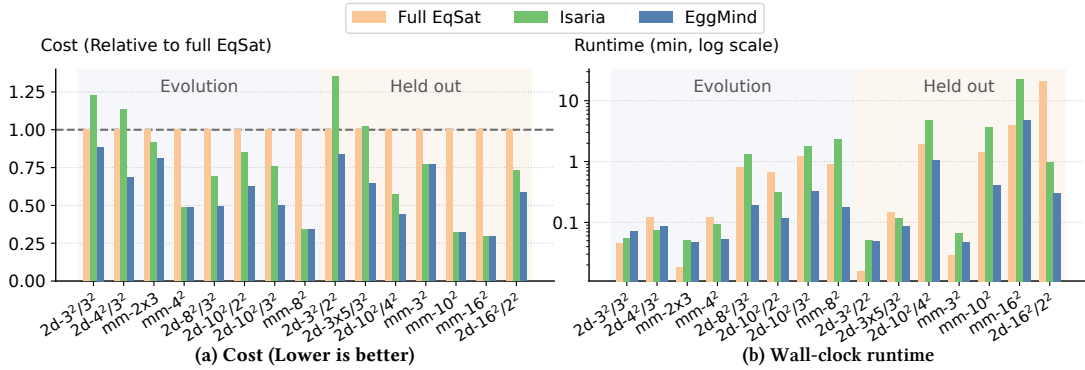


图 4.4 EggMind 在 Isaria/vectorization 基准上的主要结果。该结果支持正文关于策略质量、执行时间和内存占用改善的论断。

EggMind^[122] 还展示了跨领域迁移能力。在 XLA 张量编译器基准测试中，合成策略在 17 个案例中 13 个获得低于无引导基线的最终代价，其余 4 个持平，同时相对完整等价饱和获得 11.89x 几何平均执行时间加速。在 EqMap 逻辑综合案例研究中，EggMind 使用增强重写空间，在 carry-chain 基准测试上取得最好的质量效率权衡，相对原始规则 $i = 4$ 和 $i = 5$ 分别降低 33.76% 和 8.75% 代价，并相对无引导高预算搜索降低 51.94% 峰值内存。这些结果说明，EggMind 在不同等价图优化域中学习和复用策略组织方式，目标范围超出单个向量化基准测试的手工调参。

从本报告主线看, EggMind 是 ISAMORE 和 SkyEgg 的策略层延伸。ISAMORE 依赖等价图反合一和分阶段等价饱和发现可复用定制指令, SkyEgg 依赖等价图表达硬件变换与映射选择; 二者都面临重写空间如何组织、搜索如何收敛、策略如何迁移的问题。EggMind 的贡献是把这些策略从隐含经验提升为显式、可检查、可复用的 EqSatL 产物, 并用智能体流程、证明派生 motif 记忆和可处理性引导对大语言模型搜索施加结构化约束。这也是本报告中“大语言模型与形式化技术结合”的核心含义。大语言模型负责提出和改进策略, 形式化对象负责限定可操作边界, 证明和评价器负责提供检查记录, 编译基础设施负责执行和检查。

4.3 本章小结

本章回应第一章提出的第三个科学问题, 即编译接口如何组织程序语义、硬件能力和智能体行为, 形成跨硬件架构的可验证、可复用编译协同机制。EggMind 展示了在编译优化和等价图搜索中, 大语言模型应当通过 EqSatL、智能体流程、证明派生 motif 记忆和可处理性引导等结构化对象进入流程; OriGen 作为背景说明大语言模型进入寄存器传输级生成时必须面对数据质量、编译结果和自反思能力问题。二者共同说明, 大语言模型方法的价值在于帮助工具链进行策略生成、错误修复、经验积累和长程优化, 并始终留在硬件工具链能够检查和承载的边界内。

这也为第五章未来工作奠定了方法基础。后续研究需要把 OriGen 所体现的工具约束硬件生成背景、EggMind 所体现的可检查策略生成, 以及第二、三章中的架构接口和硬件综合机制结合起来, 进一步构建验证约束下的架构、硬件与编译协同生成, 智能体编译基础设施, 以及面向工业级异构架构的算子优化与智能体化编译协同。未来工作将约束智能体在明确的架构接口、中间表示产物、形式化约束、测试/基准测试和执行轨迹中进行协作, 避免自由生成设计。

第五章 未来工作

5.1 未来工作总体目标

前文已有工作展示了三类研究基础。第二章说明，应用需求可以通过架构接口、定制指令、微架构机制和加速器候选选择进入领域定制体系；第三章说明，架构能力只有经过硬件前端、中间表示、调度、综合和验证约束，才能成为质量可控的实现；第四章说明，大语言模型与智能体方法若要进入硬件设计和编译优化，就必须依附于形式化对象、编译基础设施和可执行检查，避免停留在自由文本生成。三类基础共同指向一个更完整的博士阶段研究目标，即构建由架构接口、硬件实现、编译接口和智能体能力共同支撑的跨硬件架构软硬件协同方法。

未来工作需要在已有系统基础上形成更完整的研究命题。APS/Aquas、ISAMORE、Cayman、Cement、Clay、SkyEgg、OriGen 和 EggMind 已经分别回答了“如何形成架构能力”“如何实现高质量硬件”“如何约束智能体优化”的局部问题，博士论文仍需要进一步回答这些局部能力如何组合成一个系统性研究命题。后续研究要把应用意图、架构接口、硬件生成、编译降低、目标执行环境、测试基准和智能体行为放到同一套可检查对象中，使敏捷芯片设计与创新性编译技术能够相互定义、相互约束并共同演化。

基于这一判断，后续研究分为三个方向。第一，研究验证约束下的架构、硬件与编译协同生成，回答生成对象如何在架构、硬件和编译层保持一致。第二，研究智能体编译基础设施，回答编译层如何为人和智能体提供可操作、可检查、可演化的基础对象。第三，研究面向工业级异构架构的算子优化与智能体化编译协同，回答上述方法如何进入真实异构系统中的算子、任务图、数据移动和性能优化问题。三个方向从协同生成方法、编译基础设施和产业级复杂场景三个侧面共同补全本报告的主线，三者关系并列互补，不构成线性上下游。

表 5.1 未来工作三方向的关系。三者分别从协同生成方法、编译基础设施和工业异构场景补全本报告的软硬件协同主线。

验证约束协同生成	智能体编译基础设施	异构算子与智能体编译
语义边界、类型化产物、可执行判定基准	目标事实、显式中间表示、语义契约、经验记录	算子优化、任务图、性能剖析、执行轨迹
架构、硬件与编译对象保持一致	编译器成为人和智能体共同使用的基础设施	工业级异构架构中的长期优化能力

5.2 验证约束下的架构、硬件与编译协同生成

5.2.1 问题来源与研究问题

第一项未来工作面向架构、硬件与编译支持的协同生成。第二章中的 APS/Aquas 已经说明，若应用接口、架构描述、编译中间表示和硬件生成流程能够被统一组织，领域定制处理器和近数据处理系统就可以从手工拼接工具链转向由接口驱动的端到端协同。第四章引言中讨论的 OriGen 则说明，大语言模型可以利用编译错误等工具信息修复寄存器传输级代码，但其对象仍主要停留在局部代码和局部工具结果。后续研究需要把这两类经验统一起来。大语言模型和智能体可以参与架构、硬件和编译的候选生成，但系统事实必须由可解析、可执行、可重放的对象承载。

该方向的核心研究问题是如何在架构描述、硬件实现和编译支持同时快速演化的情况下，定义一组跨层语义边界，使智能体提出的修改能够被类型化产物、可执行判定基准、仿真、等价性检查和综合结果共同约束。这里的“生成”指把人的应用意图、目标架构能力、硬件阶段性表示、编译降低计划和验证义务组织成一组相互引用的对象，区别于自然语言到寄存器传输级代码的一次性映射。只有这些对象之间可以持续相互校验，敏捷芯片设计才不会退化为不可检查的自动代码生成。

5.2.2 研究方法与设计

未来研究将采用阶段化协同生成方法。硬件侧从架构接口、并发语义、通信协议和存储结构开始，逐步进入微架构、寄存器传输级设计和综合约束，避免直接生成完整实现；编译侧从程序语义、目标事实、中间表示降低、编译过程和系统应用二进制接口逐步推进，避免直接生成目标后端。两个过程通过共同的可执行判定基准对齐。同一个任务或算子应当能够在高层语义模型、硬件原型、编译产物和目标执行环境之间产生可比较的执行轨迹；若执行轨迹不一致，系统应能定位责任边界，例如接口语义错误、降低错误、调度错误或硬件实现错误。

Spine 原型可以作为这一方向的重要基础，其位置更接近 APS/Aquas 式端到端协同框架的智能体化发展。应用意图进入架构接口，架构接口约束硬件生成，硬件生成结果需要编译降低和执行判定基准共同检查，智能体在其中提出有限范围内的接口修订、硬件结构候选、编译降低候选和修复建议。已有 Delta 执行主体/任务设计界面、生成器驱动执行、执行轨迹流、案例报告和 GCD Delta 到寄存器传输级设计的验证路径，说明小规模语义判定基准、任务执行和寄存器传输级对齐可以被组织为可重放记录。后续研究需要扩大覆盖面，使架构描述能够表达处理器-加速器接口、存储/缓存约束和领域专用操作，并使智能体行为始终受类型检查、仿真、等价性检查、综合结果和基准测试约束。

5.2.3 对全局故事的补全意义

这一方向补全的是“敏捷芯片设计”侧的系统能力。前文已有工作已经分别展示了可编程接口、可复用定制指令、微架构感知综合和形式化策略生成，但仍缺少把架构、硬件和编译共同生成、共同检查的统一方法。如果该方向能够成立，敏捷芯片设计将依赖一套带有语义边界、类型化产物、可执行判定基准和跨层检查的协同生成体系，从而超越单纯追求硬件生成器速度的路径。这一体系也为第二、三项未来工作提供对象来源。智能体编译基础设施需要承载这些生成对象，工业级异构算子优化需要在真实目标约束中使用这些对象。

5.3 智能体编译基础设施

5.3.1 问题来源与研究问题

第二项未来工作面向智能体编译基础设施。第一章已经指出，编译接口是衔接软件领域的重要接口；第二、三、四章进一步说明，无论是 APS/Aquas 的端到端处理器生成、ISAMORE 的分阶段等价饱和与定制指令复用、SkyEgg 的等价图高层综合优化，还是 EggMind 的 EqSatL 策略合成，真正困难的部分都在于如何让目标事实、中间表示表达、语义约束、变换过程、后端产物和评价记录保持一致，孤立实现某个编译过程并不足以解决这一问题。大语言模型和智能体方法进入这一过程后，编译基础设施必须成为智能体能力提升的结构化媒介，不能退化为可运行脚本集合。

该方向的核心研究问题是如何把编译器中的事实、语义、变换、检查和产物组织成智能体可以安全操作的对象。传统编译器往往把大量语义隐藏在编译遍实现、临时分析结果或后端约定中，人类专家可以凭经验理解这些隐含关系，但智能体若直接修改代码或编译流程，容易破坏布局假设、别名关系、同步语义、目标应用二进制接口或后端约束。因此，编译基础设施需要显式暴露目标事实、中间表示结构、语义契约、修改意图、检查门、后端记录和历史运行记录，使智能体行为有输入边界、输出形态和检查依据。

5.3.2 研究方法与设计

未来研究将以“目标事实、显式中间表示与语义契约、受约束变换、可执行后端产物、检查记录”为基本链条。目标事实包括并行层级、存储层级、通信与同步约束、指令语义、布局限制、系统应用二进制接口和性能剖析能力；显式中间表示负责承载程序结构与变换边界；语义契约说明操作、区域、方言和后端产物的含义；受约束变换把分析、重写、编译过程、检查门、语义执行和智能体行为统一到可记录的编译器行为机制中；后端产物包括可执行代码、调度计划、计算内核、系统配置和验证脚本；检查记录

包括 TraceDB 中的事实/事件、重写记录、检查门结果、测试输出、性能剖析数据和人工审阅记录。

IntelliC 的研究重点正是构建这样一类基础设施。其基础设计将中间表示拆分为 $S_y + S_e$ 。 S_y 负责语法、结构、身份、验证、规范文本和解析， S_e 负责由类型化语义定义、语义层级键和 TraceDB 承载的语义定义。这样的拆分避免把语义隐藏在解析器或编译过程的临时状态中，也避免把智能体的解释当作编译事实。一个操作可以有具体值、抽象范围、符号、后端记录等多个语义层级；同一个编译流程可以通过 TraceDB 记录匹配、修改意图、语义事实、诊断、义务和记录链接，从而让人和智能体都能追踪某次变换依赖哪些事实、通过哪些检查、产生哪些后端记录。

在智能体参与方面，编译基础设施需要区分三类行为。第一类是固定编译器行为，例如传统编译过程、验证检查或后端交接；第二类是智能体行为，例如在受限输入、受限输出和明确记录模式下提出候选变换、候选流程或诊断解释；第三类是经过检验后可以固化的行为，即智能体提出的可复用流程组件在测试、语义检查和人工审阅后被转化为固定编译器能力。这样的边界使智能体沿着编译器原生对象参与优化，始终保留在编译基础设施能够检查和承载的范围内。

5.3.3 对全局故事的补全意义

这一方向补全的是“创新性编译技术”侧的系统能力。第一项未来工作需要能够承载架构、硬件和编译生成对象的编译层；第三项未来工作需要能够记录目标事实、优化动作和性能记录的编译层。智能体编译基础设施正是二者之间的中介，它把已有等价图、MLIR、EqSatL、TraceDB、语义契约和后端记录组织成智能体可用的编译接口。由此，编译器成为软硬件协同中组织事实、限制行动、积累经验和推动智能体能力提升的基础设施，其作用超出硬件能力调用工具。

5.4 面向工业级异构架构的算子优化与智能体化编译协同

5.4.1 问题来源与研究问题

第三项未来工作面向工业级异构架构的算子优化与智能体化编译协同。该方向承接前两个方向所形成的协同生成和编译基础设施能力，同时面对更复杂的产业系统场景。当目标从单个可控硬件原型扩展到真实异构系统时，编译器不仅要生成高性能计算内核，还要面对动态图捕获、任务调度、数据移动、多设备通信、服务延迟、吞吐、性能剖析和回归稳定性等问题。PyTorch^[87] torch.compile/TorchDynamo 已经把动态图程序、图断点、后端编译和即时执行回退组织为编译与动态执行之间的边界；TensorRT-LLM^[88]

则展示了大语言模型推理系统中引擎构建、内核融合、量化、KV 缓存、连续批处理、分页注意力和服务后端的共同作用。

与此同时, NVIDIA GB200 NVL72^[2]、CloudMatrix384^[3] 和 UB-Mesh^[4] 等系统把优化对象推进到机架级互连、跨芯片通信和真实服务拓扑, 使算子优化必须与通信路径、数据移动和系统边界共同考虑。NVIDIA AVO^[23] 进一步说明, 在 Blackwell B200 图形处理器注意力内核优化中, 自主代码智能体可以结合谱系、领域知识和执行结果作为变异算子, 进入性能敏感的内核/编译优化流程。这些材料共同支撑一个判断。工业级异构架构上的优化对象天然跨越编译、硬件拓扑和执行系统, 智能体方法必须围绕目标事实、性能剖析、测试基准和长期经验组织行动。

5.4.2 研究方法与设计

本方向采用并列的两条子线。第一条子线研究工业级异构架构上的芯片算子自动优化。其目标是构建面向复杂融合算子、Torch 训练/推理核心算子和长程优化任务的智能体协作框架, 目标范围超过单个算子实现的自动生成。该框架的输入包括芯片架构描述、指令集架构与存储层级、编译约束、算子语义和性能目标; 行为空间覆盖候选实现生成、中间表示重写、编译过程组合、性能剖析调用、形式化辅助建模、回归测试和经验记忆更新; 输出不仅是一段计算内核代码, 还应包括正确性记录、性能记录、适用配置、失败案例和可复用优化经验。

AVO 在这里是重要趋势材料, 不作为狭义基线。它说明“智能体、评价器和执行结果”在工业硬件附近是技术上真实的方向, 但本报告拟研究的问题更宽。目标算子不局限于注意力内核, 目标硬件不局限于单一图形处理器, 优化过程也需要结合目标中间表示、编译过程、性能剖析轨迹、形式化约束、回归验证和长期经验记忆, 而不能停留在代码变异。也就是说, AVO 支撑方向合理性, 剩余科学问题是如何把智能体搜索嵌入可维护、可检查、可迁移的编译和算子优化基础设施。

第二条子线研究分布式执行系统协同。PTO 层级执行原型提供了重要基础。系统用 L0–L6 层级对应 Ascend 神经网络处理器集群拓扑, 其中 L2 是芯片边界, L3 以上由编排器、调度器和执行单元递归组合; 编排器通过 TensorMap 从 TaskArgs 的输入、输出和读写标签推导依赖, Ring 提供带反压的槽位池, 调度器负责连接队列、就绪队列和完成队列, 工作线程在线程或进程模式下把 Callable、TaskArgs 和 CallConfig 分发给下一级执行单元。L2 芯片级执行系统又包含主机侧程序、AICPU 调度器和 AICore 执行单元, 形成从主机编排到设备任务执行的明确边界。

基于这一基础, 后续研究需要把分布式执行系统从可执行任务图推进到可被编译器和智能体使用的执行记录系统。一方面, TensorMap、RingBuffer、就绪/完成协议、任

务组、CallConfig 和 mailbox/blob 应用二进制接口可以扩展为描述数据依赖、设备边界、执行配置和任务完成状态的结构化对象；另一方面，性能剖析、张量转储、性能监测单元、执行轨迹和失败报告应进入统一记录层，并回传给算子优化和编译决策。这样，执行系统不仅被动执行编译产物，也能向编译器暴露数据移动瓶颈、任务粒度失衡、设备边界开销、存储层级压力和调度等待原因。

5.4.3 对全局故事的补全意义

两条子线在“工业级异构架构”处汇合，其关系是并列互补。算子自动优化需要执行系统提供真实运行记录和部署约束，分布式执行系统也需要编译器提供任务图、内核、布局和配置决策；二者共同面向的是跨硬件架构的软硬件协同生态。未来研究将优先选择能够被自动验证和重复评估的核心场景，例如 Torch 训练算子、大语言模型推理融合算子、多设备任务图和调度案例，通过正确性测试、性能剖析、轨迹重放、回归基准测试和人工检查共同评价。

这一方向补全的是报告主线的产业级复杂性。第二章到第四章主要建立方法与基础设施，第三项未来工作则检验这些方法能否面对真实异构系统中的长程优化问题。若该方向成立，单点内核优化、编译过程组合和执行调度信息将成为可长期演化的系统能力，摆脱彼此分离的工程细节状态，从而把“敏捷芯片设计与创新性编译技术驱动的软硬件协同”落到复杂产业场景。

5.5 本章小结

本章提出的三项未来工作分别补全全局故事中的三个缺口。验证约束下的架构、硬件与编译协同生成，使应用意图、架构接口、硬件生成和编译降低能够在共同语义边界下相互校验；智能体编译基础设施，使编译器成为人和智能体共同使用的事实、语义、变换和记录组织系统；面向工业级异构架构的算子优化与智能体化编译协同，则把上述方法推进到真实多芯片、多层执行系统和长程性能优化场景中。

这三个方向共同服务于博士论文的核心目标。后续研究关注如何在跨硬件架构的软硬件协同生态中形成可复用架构能力、可控硬件实现、可演化编译接口和受约束智能体能力，使生成器、综合器和优化器的单点改进进入统一的协同体系。它们与第二至第四章的已有工作形成连续关系，也使后续研究能够围绕明确的科学问题、方法对象和验证方式展开。

第六章 结论

本报告围绕“敏捷芯片设计与创新性编译技术驱动的软硬件协同”展开，核心判断是领域应用、人工智能模型和产业级异构系统正在快速演进，但芯片设计、编译基础设施和系统软件生态的生产力仍难以匹配这种演进速度。传统依赖人工经验、固定工具边界和局部优化的研发方式，已经难以支撑从算法需求到架构接口、硬件实现、编译优化和系统部署的跨层协同。软硬件协同研究因此需要同时关注接口抽象、生成方法、验证记录、编译基础设施和系统检查，不能只追求某个单点工具的性能提升。

已有工作构成了本研究继续推进的技术基础。面向领域定制的架构接口与端到端协同工作表明，应用语义、架构描述和编译支持可以通过明确接口被共同组织，APS/Aquas、ISAMORE 和 Cayman 分别从处理器生成、可复用定制指令和端到端接口协同等角度展示了领域定制的可行路径。面向高质量硬件实现的前端抽象与综合优化工作表明，硬件前端、中间表示设计和综合优化能够显著影响设计质量和生产效率，HECTOR、Cement、Clay 和 SkyEgg 分别提供了多层中间表示积累、硬件建模、应用专用指令集处理器设计和高层综合优化的技术支撑。大语言模型与形式化技术驱动的软硬件协同工作进一步说明，大语言模型与智能体的价值在于进入等价饱和、寄存器传输级生成、编译检查结果和经验驱动优化等流程，在既有工具链约束下帮助策略生成、错误修复和经验迁移；EggMind 和 OriGen 分别展示了这一方向在编译优化与硬件生成中的潜力。

在此基础上，未来研究将从三个方向推进到更完整的系统研究路线。验证约束下的架构、硬件与编译协同生成，旨在让架构接口、硬件实现和编译支持在类型化产物、语义边界、可执行判定基准和跨层检查中共同演化。智能体编译基础设施，旨在把编译器建设为智能体方法的协作媒介，使目标事实、中间表示、语义契约、编译过程、检查门、后端产物和经验记录能够被人和智能体共同使用。面向工业级异构架构的算子优化与智能体化编译协同，旨在把复杂融合算子、Torch 训练/推理算子、任务图、数据移动和性能剖析轨迹纳入长期优化流程，使单点优化能力能够面向真实系统持续演化。

预期而言，本研究的贡献将体现在三个层面。方法层面，形成面向敏捷芯片设计的验证约束协同生成方法，以及面向编译优化的智能体化、形式化方法体系。基础设施层面，构建智能体编译协作框架，使软硬件协同过程中的事实、变换和检查记录能够被系统化管理。应用层面，在领域定制处理器、高质量硬件实现、编译优化和工业级异构架构算子协同场景中验证上述方法，最终支撑更高生产力、更强可验证性和更好可迁移性的软硬件协同研发流程。

参考文献

- [1] JOUPPI N P, KURIAN G, LI S, et al. Tpu v4: an optically reconfigurable supercomputer for machine learning with hardware support for embeddings[Z/OL]. 2023. ISCA 2023; arXiv:2304.01433. <https://arxiv.org/abs/2304.01433>.
- [2] NVIDIA. Nvidia gb200 nvl72[Z/OL]. 2026. Official NVIDIA product and architecture overview. <https://www.nvidia.com/en-us/data-center/gb200-nvl72/>.
- [3] LI Z, OTHERS. Serving large language models on huawei cloudmatrix384[Z/OL]. 2025. arXiv:2506.12708. <https://arxiv.org/abs/2506.12708>.
- [4] ZHAO Z, OTHERS. Ub-mesh: a hierarchically localized nd-fullmesh datacenter network architecture[Z/OL]. 2025. arXiv:2503.20377. <https://arxiv.org/abs/2503.20377>.
- [5] LI C, XUE C, REN Y, et al. A full-stack performance evaluation infrastructure for 3d-dram-based llm accelerators[A/OL]. 2026. <https://arxiv.org/abs/2604.08044>. DOI: 10.48550/arXiv.2604.08044
- [6] MO Z, LI G, CHEN H M, et al. Deepstack: scalable and accurate design space exploration for distributed 3d-stacked ai accelerators[A/OL]. 2026. <https://arxiv.org/abs/2604.04750>. DOI: 10.48550/arXiv.2604.04750
- [7] WATERMAN A, ASANOVIC K. The risc-v instruction set manual, volume i: unprivileged isa, document version 20190608-base-ratified[Z/OL]. 2019. <https://riscv.org/technical/specifications/>.
- [8] SANGHAVI H A, ANDREWS N B. Tie: an adl for designing application-specific instruction set extensions[M]//Processor Description Languages. Elsevier, 2008: 183–216.
- [9] OPPERMANN J, DAMIAN-KOSTERHON B M, MEISEL F, et al. Longnail: high-level synthesis of portable custom instruction set extensions for RISC-v processors from descriptions in the open-source CoreDSL language[C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. 2024: 591–606. <https://dl.acm.org/doi/10.1145/3620666.3651375>. DOI: 10.1145/3620666.3651375
- [10] CLARK N T, ZHONG H, MAHLKE S A. Automated custom instruction generation for domain-specific processor acceleration[J/OL]. IEEE Transactions on Computers, 2005, 54(10): 1258–1270. <https://ieeexplore.ieee.org/document/1501791>. DOI: 10.1109/TC.2005.156
- [11] TRILLA D, WELLMAN J D, BUYUKTOSUNOGLU A, et al. Novia: a framework for discovering non-conventional inline accelerators[C/OL]//MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture. 2021: 507–521. <https://doi.org/10.1145/3466752.3480094>. DOI: 10.1145/3466752.3480094
- [12] ZACHAROPOULOS G, FERRETTI L, ANSALONI G, et al. Compiler-assisted selection of hardware acceleration candidates from application source code[C]//2019 IEEE 37th International Conference on Computer Design. 2019: 129–137. DOI: 10.1109/ICCD46524.2019.00024.
- [13] OPENAI. Introducing codex[Z/OL]. 2025. Official product and technical overview. <https://openai.com/index/introducing-codex/>.
- [14] ASANOVIC K, AVIZIENIS R, BACHRACH J, et al. The rocket chip generator[J/OL]. EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016. <https://people.eecs.berkeley.edu/~krste/papers/EECS-2016-17.pdf>.
- [15] OPENHW GROUP. The openhw group core-v-xif interface[Z/OL]. 2025. <https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/latest/>.

- [16] DAMIAN M, OPPERMANN J, SPANG C, et al. SCAIE-v: an open-source SCALable interface for ISA extensions for RISC-v processors[C/OL]//Proceedings of the 59th ACM/IEEE Design Automation Conference. 2022: 169–174. <https://dl.acm.org/doi/10.1145/3489517.3530432>. DOI: 10.1145/3489517.3530432
- [17] XILINX INC. Vitis high-level synthesis user guide[EB/OL].. <https://docs.amd.com/r/en-US/ug1399-vitis-hls>
- [18] SOHRABIZADEH A, YU C H, GAO M, et al. AutoDSE: enabling software programmers design efficient FPGA accelerators[C/OL]//The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2021: 147. <https://doi.org/10.1145/3431920.3439464>. DOI: 10.1145/3431920.3439464
- [19] YE H, HAO C, CHENG J, et al. ScaleHLS: a new scalable high-level synthesis framework on multi-level intermediate representation[Z/OL]. 2021. arXiv:2107.11673. <http://arxiv.org/abs/2107.11673>. DOI: 10.48550/arXiv.2107.11673
- [20] LATTENER C, AMINI M, BONDHUGULA U, et al. Mlir: a compiler infrastructure for the end of moore's law[C/OL]//2021 IEEE/ACM International Symposium on Code Generation and Optimization. 2021: 2–14. <https://arxiv.org/abs/2002.11054>.
- [21] WILLSEY M, NANDI C, WANG Y R, et al. Egg: fast and extensible equality saturation[J/OL]. Proceedings of the ACM on Programming Languages, 2021, 5(POPL). <https://arxiv.org/abs/2004.03082>.
- [22] NOVIKOV A, VU N, EISENBERGER M, et al. Alphaevolve: a coding agent for scientific and algorithmic discovery[Z/OL]. 2025. arXiv:2506.13131. <https://arxiv.org/abs/2506.13131>.
- [23] CHEN T, YE Z, XU B, et al. Avo: agentic variation operators for autonomous evolutionary search[Z/OL]. 2026. arXiv:2603.24517. <https://arxiv.org/abs/2603.24517>.
- [24] GREEN B, TODD D, CALHOUN J C, et al. Tigra: a tightly integrated generic risc-v accelerator interface[C]//2021 IEEE International Conference on Cluster Computing (CLUSTER). IEEE, 2021: 779–782.
- [25] LATTENER C, ADVE V. Llvm: a compilation framework for lifelong program analysis & transformation[C]//Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization. IEEE Computer Society, 2004: 75.
- [26] VANHATTUM A, NIGAM R, LEE V T, et al. Vectorization for digital signal processors via equality saturation[C/OL]//Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2021: 874–886. <https://dl.acm.org/doi/10.1145/3445814.3446707>. DOI: 10.1145/3445814.3446707
- [27] THOMAS S, BORNHOLT J. Automatic generation of vectorizing compilers for customizable digital signal processors[C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. 2024: 19–34. <https://dl.acm.org/doi/10.1145/3617232.3624873>. DOI: 10.1145/3617232.3624873
- [28] CHENG J, COWARD S, CHELINI L, et al. SEER: super-optimization explorer for high-level synthesis using e-graph rewriting[C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2024: 1029–1044. <https://dl.acm.org/doi/10.1145/3620665.3640392>. DOI: 10.1145/3620665.3640392
- [29] SMITH G H, KUSHIGIAN B, CANUMALLA V, et al. FPGA technology mapping using sketch-guided program synthesis[C/OL]//Proceedings of the 29th ACM International Conference on Architectural

- Support for Programming Languages and Operating Systems, Volume 2. 2024: 416–432. <https://dl.acm.org/doi/10.1145/3620665.3640387>. DOI: 10.1145/3620665.3640387
- [30] AMID A, BIANCOLIN D, GONZALEZ A, et al. Chipyard: Integrated Design, Simulation, and Implementation Framework for Custom SoCs[J/OL]. IEEE Micro, 2020, 40(4): 10–21. <https://doi.org/10.1109/MM.2020.2996616>. DOI: 10.1109/MM.2020.2996616
- [31] WOLF C. Picorv32 - a size-optimized risc-v cpu[Z/OL]. 2020. <https://github.com/YosysHQ/picorv32>.
- [32] NUCLEI SYSTEM TECHNOLOGY. NICE (Nuclei Instruction Co-unit Extension)[Z/OL]. 2024. <https://doc.nucleisys.com/hbirdv2/core/core.html#nice>.
- [33] CADENCE DESIGN SYSTEMS. Tensilica processor IP[Z/OL]. 2025. https://www.cadence.com/en_US/home/tools/ip/tensilica-ip.html.
- [34] SYNOPSYS. Synopsys ASIP designer[Z/OL]. 2025. <https://www.synopsys.com/dw/ipdir.php?ds=asip-designer>.
- [35] CODASIP. Codasip studio[Z/OL]. 2025. <https://codasip.com/products/codasip-studio/>.
- [36] LIU G, PRIMMER J, ZHANG Z. Rapid generation of high-quality RISC-v processors from functional instruction set specifications[C/OL]//Proceedings of the 56th Annual Design Automation Conference 2019. 2019. <https://doi.org/10.1145/3316781.3317890>. DOI: 10.1145/3316781.3317890
- [37] CLARK N, ZHONG H, MAHLKE S. Processor acceleration through automated instruction set customization[C/OL]//36th Annual IEEE/ACM International Symposium on Microarchitecture. 2003: 129–140. <http://ieeexplore.ieee.org/document/1253189/>. DOI: 10.1109/MICRO.2003.1253189
- [38] POZZI L, ATASU K, IENNE P. Exact and approximate algorithms for the extension of embedded processor instruction sets[J/OL]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2006, 25(7): 1209–1229. <https://ieeexplore.ieee.org/document/1634620>. DOI: 10.1109/TCAD.2005.855950
- [39] ADITYA S, RAU B R, KATHAIL V. Automatic architectural synthesis of VLIW and EPIC processors[C/OL]//Proceedings of the 12th International Symposium on System Synthesis. 1999: 107–113. <https://ieeexplore.ieee.org/document/814268>. DOI: 10.1109/ISSS.1999.814268
- [40] VENKATESH G, SAMPSON J, GOULDING N, et al. Conservation cores: reducing the energy of mature computations[J/OL]. ACM SIGARCH Computer Architecture News, 2010, 38(1): 205–218. <https://dl.acm.org/doi/10.1145/1735970.1736044>. DOI: 10.1145/1735970.1736044
- [41] GOULDING-HOTTA N, SAMPSON J, VENKATESH G, et al. The GreenDroid mobile application processor: an architecture for silicon's dark future[J/OL]. IEEE Micro, 2011, 31(2): 86–95. <https://ieeexplore.ieee.org/document/5719585>. DOI: 10.1109/MM.2011.18
- [42] VENKATESH G, SAMPSON J, GOULDING-HOTTA N, et al. QSCORES: trading dark silicon for scalable energy efficiency with quasi-specific cores[C]//2011 44th Annual IEEE/ACM International Symposium on Microarchitecture. 2011.
- [43] GNANASAMBANDAPILLAI V, PEDDERSEN J, RAGEL R, et al. FINDER: find efficient parallel instructions for ASIPs to improve performance of large applications[J/OL]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2020. DOI: 10.1109/TCAD.2020.3012211.
- [44] ZACHAROPOULOS G, EJJEH A, JING Y, et al. Trireme: exploration of hierarchical multi-level parallelism for hardware acceleration[J/OL]. ACM Transactions on Embedded Computing Systems, 2023, 22(3). <https://doi.org/10.1145/3580394>. DOI: 10.1145/3580394

- [45] BACHRACH J, VO H, RICHARDS B, et al. Chisel: constructing hardware in a scala embedded language[C]//DAC Design Automation Conference 2012. 2012: 1212–1221. DOI: [10.1145/2228360.2228584](https://doi.org/10.1145/2228360.2228584).
- [46] NIKHIL R. Bluespec system verilog: efficient, correct rtl from high level specifications[C]//Proceedings of the Second ACM and IEEE International Conference on Formal Methods and Models for Co-Design. 2004: 69–70. DOI: [10.1109/MEMCOD.2004.1459818](https://doi.org/10.1109/MEMCOD.2004.1459818).
- [47] IZRAELEVITZ A, KOENIG J, LI P, et al. Reusability is FIRRTL ground: hardware construction languages, compiler frameworks, and transformations[C]//2017 IEEE/ACM International Conference on Computer-Aided Design. 2017: 209–216. DOI: [10.1109/ICCAD.2017.8203780](https://doi.org/10.1109/ICCAD.2017.8203780).
- [48] CANIS A, CHOI J, ALDHAM M, et al. LegUp: high-level synthesis for fpga-based processor/accelerator systems[C/OL]//Proceedings of the 19th ACM/SIGDA International Symposium on Field Programmable Gate Arrays. 2011: 33–36. <https://doi.org/10.1145/1950413.1950423>. DOI: [10.1145/1950413.1950423](https://doi.org/10.1145/1950413.1950423)
- [49] JOSIPOVIĆ L, GHOSAL R, IENNE P. Dynamically scheduled high-level synthesis[C/OL]//Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2018: 127–136. <https://doi.org/10.1145/3174243.3174264>. DOI: [10.1145/3174243.3174264](https://doi.org/10.1145/3174243.3174264)
- [50] NIGAM R, ATAPATTU S, THOMAS S, et al. Predictable accelerator design with time-sensitive affine types[C/OL]//Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. 2020: 393–407. <https://doi.org/10.1145/3385412.3385974>. DOI: [10.1145/3385412.3385974](https://doi.org/10.1145/3385412.3385974)
- [51] KOEPLINGER D, FELDMAN M, PRABHAKAR R, et al. Spatial: a language and compiler for application accelerators[C/OL]//Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation. 2018: 296–311. <https://doi.org/10.1145/3192366.3192379>. DOI: [10.1145/3192366.3192379](https://doi.org/10.1145/3192366.3192379)
- [52] DURST D, FELDMAN M, HUFF D, et al. Type-directed scheduling of streaming accelerators[C/OL]//Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. 2020: 408–422. <https://doi.org/10.1145/3385412.3385983>. DOI: [10.1145/3385412.3385983](https://doi.org/10.1145/3385412.3385983)
- [53] LAI Y H, CHI Y, HU Y, et al. Heterocl: a multi-paradigm programming infrastructure for software-defined reconfigurable computing[C/OL]//Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. Association for Computing Machinery, 2019: 242–251. <https://doi.org/10.1145/3289602.3293910>. DOI: [10.1145/3289602.3293910](https://doi.org/10.1145/3289602.3293910)
- [54] CHEN H, ZHANG N, XIANG S, et al. Allo: a programming model for composable accelerator design[J/OL]. Allo: A Programming Model for Composable Accelerator Design, 2024. <https://dl.acm.org/doi/10.1145/3656401>. DOI: [10.1145/3656401](https://doi.org/10.1145/3656401)
- [55] YE H, CHEN D. Streamtensor: make tensors stream in dataflow accelerators for llms[C/OL]//Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture. 2025. <https://dl.acm.org/doi/10.1145/3725843.3762817>. DOI: [10.1145/3725843.3762817](https://doi.org/10.1145/3725843.3762817)
- [56] NIGAM R, THOMAS S, LI Z, et al. A compiler infrastructure for accelerator generators[C/OL]//Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2021: 804–817. <https://arxiv.org/abs/2102.09713>.
- [57] NIGAM R, Azevedo de AMORIM P H, SAMPSON A. Modular hardware design with timeline types[J/OL]. Proceedings of the ACM on Programming Languages, 2023, 7(PLDI). <https://doi.org/10.1145/3591234>. DOI: [10.1145/3591234](https://doi.org/10.1145/3591234)

- [58] CIRCT COMMUNITY. CIRCT: circuit IR compilers and tools[Z/OL]. 2022. <https://github.com/llvm/circt>.
- [59] GUO L, CHI Y, WANG J, et al. AutoBridge: Coupling Coarse-Grained Floorplanning and Pipelining for High-Frequency HLS Design on Multi-Die FPGAs[C/OL]//The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. ACM, 2021: 81–92. <https://dl.acm.org/doi/10.1145/3431920.3439289>. DOI: 10.1145/3431920.3439289
- [60] GUO L, CHI Y, LAU J, et al. TAPA: A Scalable Task-parallel Dataflow Programming Framework for Modern FPGAs with Co-optimization of HLS and Physical Design[J/OL]. ACM Transactions on Reconfigurable Technology and Systems, 2023, 16(4): 1–31. <https://dl.acm.org/doi/10.1145/3609335>. DOI: 10.1145/3609335
- [61] GUO L, MAIDEE P, ZHOU Y, et al. RapidStream: Parallel Physical Implementation of FPGA HLS Designs[C/OL]//Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. ACM, 2022: 1–12. <https://dl.acm.org/doi/10.1145/3490422.3502361>. DOI: 10.1145/3490422.3502361
- [62] LAU J, XIAO Y, XIE Y, et al. RapidStream IR: Infrastructure for FPGA High-Level Physical Synthesis[C/OL]//Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design. ACM, 2024: 1–11. <https://dl.acm.org/doi/10.1145/3676536.3676649>. DOI: 10.1145/3676536.3676649
- [63] DU L, LIANG T, SINHA S, et al. Fado: floorplan-aware directive optimization for high-level synthesis designs on multi-die fpgas[C/OL]//Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays. Association for Computing Machinery, 2023: 15–25. <https://doi.org/10.1145/3543622.3573188>. DOI: 10.1145/3543622.3573188
- [64] MOSES W S, CHELINI L, ZHAO R, et al. Polygeist: raising c to polyhedral mlir[C/OL]//2021 International Conference on Parallel Architectures and Compilation Techniques. 2021: 45–59. <https://research.google/pubs/polygeist-raising-c-to-polyhedral-mlir/>.
- [65] CHEN T, MOREAU T, JIANG Z, et al. TVM: an automated End-to-End optimizing compiler for deep learning[C/OL]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). USENIX Association, 2018: 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>.
- [66] TILLET P, KUNG H T, COX D. Triton: an intermediate language and compiler for tiled neural network computations[C/OL]//Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages. Association for Computing Machinery, 2019: 10–19. <https://doi.org/10.1145/3315508.3329973>. DOI: 10.1145/3315508.3329973
- [67] WANG L, CHENG Y, SHI Y, et al. Tilelang: a composable tiled programming model for ai systems[Z/OL]. 2025. <http://arxiv.org/abs/2504.17577>. DOI: 10.48550/arXiv.2504.17577
- [68] JAIN D, PARDESHI A, FRIGO M, et al. Act: automatically generating compiler backends from tensor accelerator isa descriptions[Z/OL]. 2025. <http://arxiv.org/abs/2510.09932>. DOI: 10.48550/arXiv.2510.09932
- [69] LARSEN S, AMARASINGHE S. Exploiting superword level parallelism with multimedia instruction sets[C/OL]//Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation. Association for Computing Machinery, 2000: 145–156. <https://doi.org/10.1145/349299.349320>. DOI: 10.1145/349299.349320
- [70] NUZMAN D, ROSEN I, ZAKS A. Auto-vectorization of interleaved data for simd[C/OL]//Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation.

- Association for Computing Machinery, 2006: 132–143. <https://doi.org/10.1145/1133981.1133997>. DOI: 10.1145/1133981.1133997
- [71] TATE R, STEPP M, TATLOCK Z, et al. Equality saturation: a new approach to optimization[C]// Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. 2009: 264–276. DOI: 10.1145/1480881.1480915.
- [72] NANDI C, WILLSEY M, ZHU A, et al. Rewrite rule inference using equality saturation[J/OL]. Proceedings of the ACM on Programming Languages, 2021, 5(OOPSLA). <https://doi.org/10.1145/3485496>. DOI: 10.1145/3485496
- [73] CAO D, KUNKEL R, NANDI C, et al. Babble: learning better abstractions with e-graphs and anti-unification[Z/OL]. 2022. arXiv:2212.04596. <http://arxiv.org/abs/2212.04596>. DOI: 10.48550/arXiv.2212.04596
- [74] PAL A, SAIKI B, TJOA R, et al. Equality saturation theory exploration à la carte[J/OL]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA2): 1034–1062. <https://dl.acm.org/doi/10.1145/3622834>. DOI: 10.1145/3622834
- [75] PAL A, SAIKI B, TJOA R, et al. Equality saturation theory exploration à la carte[J/OL]. Proc. ACM Program. Lang., 2023, 7(OOPSLA2). <https://doi.org/10.1145/3622834>. DOI: 10.1145/3622834
- [76] ZHANG Y, WANG Y R, FLATT O, et al. Better Together: Unifying Datalog and Equality Saturation[J/OL]. Artifact for "Better Together: Unifying Datalog and Equality Saturation", 2023, 7(PLDI): 125:468–125:492. <https://dl.acm.org/doi/10.1145/3591239>. DOI: 10.1145/3591239
- [77] MERCKX J, LOPOUKHINE A, COWARD S, et al. Eqsat: an equality saturation dialect for non-destructive rewriting[Z/OL]. 2025. <http://arxiv.org/abs/2505.09363>. DOI: 10.48550/arXiv.2505.09363
- [78] ZAYED A E A, DUBACH C. Dialegg: dialect-agnostic mlir optimizer using equality saturation with egglog[C/OL]//Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization. 2025. <https://dl.acm.org/doi/10.1145/3696443.3708957>. DOI: 10.1145/3696443.3708957
- [79] VOHRA A, LEE L S, BACHURSKI J, et al. Mind the abstraction gap: bringing equality saturation to real-world ml compilers[J/OL]. Artifact for "Mind the Abstraction Gap: Bringing Equality Saturation to Real-World ML Compilers", 2025. <https://dl.acm.org/doi/10.1145/3763062>. DOI: 10.1145/3763062
- [80] CHEN C, HU G, ZUO D, et al. E-syn: e-graph rewriting with technology-aware cost functions for logic synthesis[Z/OL]. 2024. arXiv:2403.14242. <https://arxiv.org/abs/2403.14242>.
- [81] CHEN C, HU G, YU C, et al. E-morphic: scalable equality saturation for structural exploration in logic synthesis[Z/OL]. 2025. arXiv:2504.11574. <https://arxiv.org/abs/2504.11574>.
- [82] HOFMANN M, GOKMEN B, ZHANG Z. Eqmap: FPGA LUT remapping using e-graphs[C]//2025 IEEE/ACM International Conference on Computer Aided Design. 2025: 1–9. DOI: 10.1109/ICCAD66269.2025.11240672.
- [83] SMITH G H, KNIZEK C, PETRISKO D, et al. Scaling program synthesis based technology mapping with equality saturation[Z/OL]. 2024. arXiv:2411.11036. <http://arxiv.org/abs/2411.11036>. DOI: 10.48550/arXiv.2411.11036
- [84] CAI Y, YANG K, DENG C, et al. Smoothe: differentiable e-graph extraction[C]//Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. 2025: 1020–1034. DOI: 10.1145/3669940.3707262.

- [85] ARORA J, LU S, JAIN D, et al. Tensorright: automated verification of tensor graph rewrites[J/OL]. Artifact for "TensorRight: Automated Verification of Tensor Graph Rewrites", 2025. <https://dl.acm.org/doi/10.1145/3704865>. DOI: 10.1145/3704865
- [86] PARK J, KIM Y, AN H, et al. Trinity: three-dimensional tensor program optimization via tile-level equality saturation[C/OL]//Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2026. <https://dl.acm.org/doi/10.1145/3779212.3790240>. DOI: 10.1145/3779212.3790240
- [87] PYTORCH CONTRIBUTORS. Torch.compile[Z/OL]. 2026. PyTorch 2.11 official documentation. <https://docs.pytorch.org/docs/stable/generated/torch.compile.html>.
- [88] NVIDIA. Nvidia tensorrt-llm documentation[Z/OL]. 2026. Official TensorRT-LLM documentation. <https://docs.nvidia.com/tensorrt-llm/>.
- [89] KWON W, LI Z, ZHUANG S, et al. Efficient memory management for large language model serving with pagedattention[Z/OL]. 2023. <http://arxiv.org/abs/2309.06180>. DOI: 10.48550/arXiv.2309.06180
- [90] YE Z, CHEN L, LAI R, et al. Flashinfer: efficient and customizable attention engine for llm inference serving[Z/OL]. 2025. <http://arxiv.org/abs/2501.01005>. DOI: 10.48550/arXiv.2501.01005
- [91] ZHENG S, BAO W, HOU Q, et al. Triton-distributed: programming overlapping kernels on distributed ai systems with the triton compiler[Z/OL]. 2025. <http://arxiv.org/abs/2504.19442>. DOI: 10.48550/arXiv.2504.19442
- [92] HWANG C, CHENG P, DATHATHRI R, et al. Msccl++: rethinking gpu communication abstractions for ai inference[C/OL]//. 2026. <http://arxiv.org/abs/2504.09014>. DOI: 10.1145/3779212.3790188
- [93] GOLDMAN A, BOKER N, SHERAIZIN M, et al. Nccl ep: towards a unified expert parallel communication api for nccl[Z/OL]. 2026. <http://arxiv.org/abs/2603.13606>. DOI: 10.48550/arXiv.2603.13606
- [94] PEARCE H, TAN B, KARRI R. DAVE: deriving automatically verilog from english[C]//Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD. 2020: 27–32.
- [95] BLOCKLOVE J, GARG S, KARRI R, et al. Chip-chat: challenges and opportunities in conversational hardware design[C]//2023 ACM/IEEE 5th Workshop on Machine Learning for CAD. 2023: 1–6.
- [96] CHANG K, WANG Y, REN H, et al. ChipGPT: how far are we from natural language hardware design[J]. arXiv preprint arXiv:2305.14019, 2023.
- [97] LIU M, PINCKNEY N, KHAILANY B, et al. VerilogEval: evaluating large language models for verilog code generation[C]//2023 IEEE/ACM International Conference on Computer Aided Design. 2023: 1–8.
- [98] LU Y, LIU S, ZHANG Q, et al. RtlLm: an open-source benchmark for design rtl generation with large language model[C]//2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC). IEEE, 2024: 722–727.
- [99] THAKUR S, AHMAD B, PEARCE H, et al. Verigen: a large language model for verilog code generation[J]. ACM Transactions on Design Automation of Electronic Systems, 2023.
- [100] LIU S, FANG W, LU Y, et al. Rtlcoder: outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution[J/OL]. arXiv preprint arXiv:2312.08617, 2023. <https://arxiv.org/abs/2312.08617>.
- [101] PEI Z, ZHEN H L, YUAN M, et al. BetterV: controlled verilog generation with discriminative guidance[J]. arXiv preprint arXiv:2402.03375, 2024.
- [102] TSAI Y, LIU M, REN H. Rtlfixer: automatically fixing RTL syntax errors with large language models[J/OL]. arXiv preprint arXiv:2311.16543, 2023. <https://arxiv.org/abs/2311.16543>.

- [103] THAKUR S, BLOCKLOVE J, PEARCE H, et al. Autochip: automating HDL generation using LLM feedback[J/OL]. arXiv preprint arXiv:2311.04887, 2023. <https://arxiv.org/abs/2311.04887>.
- [104] LIU M, ENE T D, KIRBY R, et al. Chipnemo: domain-adapted llms for chip design[J]. arXiv preprint arXiv:2311.00176, 2023.
- [105] CHANG K, WANG K, YANG N, et al. Data is all you need: finetuning llms for chip design via an automated design-data augmentation framework[J]. arXiv preprint arXiv:2403.11202, 2024.
- [106] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: agent computer interfaces enable software engineering language models[Z]. 2024.
- [107] ANTHROPIC. Claude code overview[Z/OL]. 2026. Official documentation for Anthropic's agentic coding tool. <https://docs.anthropic.com/en/docs/claude-code/overview>.
- [108] CAO S, MAO Z, GONZALEZ J E, et al. K-search: llm kernel generation via co-evolving intrinsic world model[Z/OL]. 2026. <http://arxiv.org/abs/2602.19128>. DOI: 10.48550/arXiv.2602.19128
- [109] MAI H, GUO X, DING X, et al. Argus: agentic gpu optimization guided by data-flow invariants[Z/OL]. 2026. <http://arxiv.org/abs/2604.18616>. DOI: 10.48550/arXiv.2604.18616
- [110] HARI S K S, BALAJI V, DAMANI S, et al. Improving efficiency of gpu kernel optimization agents using a domain-specific language and speed-of-light guidance[Z/OL]. 2026. <http://arxiv.org/abs/2603.29010>. DOI: 10.48550/arXiv.2603.29010
- [111] LIN E, MODI S, HARI S K S, et al. Sol-execbench: speed-of-light benchmarking for real-world gpu kernels against hardware limits[Z/OL]. 2026. <http://arxiv.org/abs/2603.19173>. DOI: 10.48550/arXiv.2603.19173
- [112] ZHANG N, DENG C, KUEHN J M, et al. Aspen: LLM-guided e-graph rewriting for RTL datapath optimization[C/OL]//2025 ACM/IEEE 7th Symposium on Machine Learning for CAD. 2025: 1–9. <https://doi.org/10.1109/MLCAD65511.2025.11189222>. DOI: 10.1109/MLCAD65511.2025.11189222
- [113] XIAO Y, OTHERS. Aps: open-source hardware-software co-design framework for agile domain-specific processor research[Z]. 2025. Manuscript and local repository.
- [114] XIAO Y, OTHERS. Aquas: mlir-based automated hardware-software co-design for asip specialization[Z]. 2026. Manuscript and local repository.
- [115] XIAO Y, OTHERS. Isamore: finding reusable instructions via e-graph anti-unification[Z]. 2026. ASPLOS 2026 manuscript and local repository.
- [116] XIAO Y, CUI F, LUO Z, et al. Cayman: custom accelerator generation with control flow and data access optimization[C]//Proceedings of the 62nd ACM/IEEE Design Automation Conference. 2025: 1–7. DOI: 10.1109/DAC63849.2025.11132875.
- [117] XU R, XIAO Y, LUO J, et al. Hector: a multi-level intermediate representation for hardware synthesis methodologies[C]//Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design. 2022. DOI: 10.1145/3508352.3549370.
- [118] XIAO Y, OTHERS. Cement: streamlining fpga hardware design with cycle-deterministic ehdl and synthesis[Z]. 2024. Local manuscript.
- [119] PENG W, XIAO Y, ZOU Y, et al. Clay: high-level asip framework for flexible microarchitecture-aware instruction customization[C]//2025 IEEE/ACM International Conference On Computer Aided Design. 2025: 1–9. DOI: 10.1109/ICCAD66269.2025.11240669.
- [120] XIAO Y, OTHERS. Skyegg: joint implementation selection and scheduling for hardware synthesis using e-graphs[Z]. 2026. FPGA 2026 manuscript and local repository.

- [121] XIAO Y, OTHERS. Origen: open-source rtl code generation with dataset augmentation and self-reflection[Z]. 2024. Local manuscript.
- [122] XIAO Y, OTHERS. Eggmind: llm-guided synthesis of reusable equality saturation strategies[Z]. 2026. Local manuscript.